

Часть IV

Проектирование с использованием C++

Здесь представлены возможности языка C++ в более широком плане разработки программного обеспечения. Изложение фокусируется на методах проектирования и эффективной реализации проекта в терминах языковых конструкций.

Главы

- 23.** Общий взгляд на разработку программ. Проектирование
- 24.** Проектирование и программирование
- 25.** Роли классов

«Я только сейчас начинаю испытывать всю сложность реализации идей на бумаге. Пока речь идет об описании чего-либо, все просто; но как только в игру вступают рассуждения, так сразу взаимосвязи, ясность и беглость изложения становятся для меня весьма нелегкой задачей, о чем ранее я и понятия не имел.»

— Чарльз Дарвин

Общий взгляд на разработку программ. Проектирование

*Не существует серебряной пули.
— Ф. Брукс*

Разработка программного продукта — цели и средства — процесс разработки — цикл разработки — этапы проектирования — выявление классов — определение операций — определение зависимостей — определение интерфейсов — реорганизация иерархии классов — модели — экспериментирование и анализ — тестирование — сопровождение программного продукта — эффективность — руководство разработкой — повторное использование — масштаб — важность личностей — гибридное проектирование — аннотированная библиография — советы.

23.1. Обзор

Данная глава является первой из трех глав, посвященных процессу разработки программ, начиная с относительно высокоуровневого взгляда на проектирование и заканчивая специфическими для языка C++ концепциями и программными технологиями, призванными непосредственно поддерживать проектные решения. После введения и короткого обсуждения целей и средств программирования в §23.3, настоящая глава сосредотачивается на двух главных темах, которые можно охарактеризовать как:

- §23.4 Общий взгляд на разработку программ.
- §23.5 Практические советы по поводу организации процесса разработки программ.

В главе 24 обсуждается взаимосвязь проектирования и языка программирования. Глава 25 рассказывает о той роли, которую классы играют в организации программ на этапе проектирования. В целом эти три главы, составляющие часть IV на-

стоящей книги, призваны навести мосты между двумя крайними методиками: между высокоуровневым проектированием, мнящим себя независимым от деталей языка, и сугубо приземленным программированием, близоручко цепляющимся за мелкие детали языка. Оба конца столь широкого спектра находят свое место в рамках обширного программного проекта, но во избежание излишне высокой цены проекта или его неудачи их нужно тщательно согласовывать на предмет соответствия задач и предлагаемых методик.

23.2. Введение

Разработка и создание любого нетривиального программного продукта является весьма сложной задачей. Даже для программиста-индивидуала непосредственное написание программных конструкций является лишь частью процесса. В типичном случае, анализ исходной задачи, создание общего проекта программы, ее документация, тестирование и поддержка, а также управление всеми этими аспектами разработки намного превосходят по сложности написание и отладку отдельных фрагментов кода. Можно, конечно, назвать всю такую деятельность программированием и заявить, что «нет, я не занимаюсь проектированием, а только лишь программированием», но все равно, независимо от названий, нужно уметь и фокусироваться на отдельных частях, и уметь рассмотреть все это в целом. Ни детали, ни общая картина не должны теряться из-за стремления поскорее сдать готовую систему — хотя часто именно так и происходит.

Данная глава посвящена тем аспектам разработки программных продуктов, которые не связаны с написанием и отладкой конкретного кода для отдельных частей программы. Обсуждение указанных вопросов менее точно и менее подробно, чем рассмотрение конкретных деталей языка C++ или специфических приемов программирования, рассмотренных в нашей книге. Однако это неизбежно, ибо не существует готовых рецептов создания хороших программных продуктов. Подробные рекомендации в стиле «сделай так-то и так-то» возможны лишь для отдельных типов приложений, а не для общего случая. Ничем невозможно заменить интеллект, опыт и вкус в программировании. Поэтому данная глава предлагает лишь общие советы, альтернативные подходы и некоторые полезные предостережения.

Обсуждение затрудняется абстрактной природой программного обеспечения и тем фактом, что приемы, хорошо работающие для небольших проектов (скажем, для одного-двух человек на 10000 строк кода), не обязательно пригодны для проектов среднего или большого масштаба. По этой причине ряд положений формулируется на примере родственных тем из менее абстрактных инженерных дисциплин, а не программирования. Вам следует помнить, что «доказательства по аналогии» часто обманчивы, так что мы используем аналогии лишь для наглядной иллюстрации (а не доказательства). Переформулирование общих утверждений непосредственно в терминах языка C++ и сопутствующих примеров кода осуществляется в главах 24 и 25. Идеи же данной главы неявно подкрепляются языковыми средствами и примерами, рассмотренными во всех остальных главах книги.

Помните также, что из-за огромного разнообразия прикладных областей, людей и применяемых средств разработки не следует ожидать, что каждое высказывание непосредственно применимо к вашей конкретной задаче. Приведенные в настоя-

шей главе высказывания вытекают из опыта разработки реальных проектов и применимы к широкому кругу задач, но они не являются абсолютно универсальными. Относитесь к ним со здоровой долей скептицизма.

Известно, что C++ можно использовать как улучшенный C. Однако в таком случае останутся незадействованными самые мощные средства этого языка, и лишь малая часть выгод от применения C++ будет достигнута. Данная глава фокусируется на подходе к проектированию, при котором удастся эффективно применить механизм абстракции данных и средства объектно-ориентированного программирования языка C++; такой подход часто называют *объектно-ориентированным проектированием* (*object-oriented design*).

Несколько важнейших мыслей насквозь пронизывают данную главу:

- Самое важное в разработке программ — ясно понимать, что именно вы пытаетесь сделать.
- Успешный процесс разработки программного продукта — это длительный процесс.
- Разрабатываемые системы всегда эволюционируют в направлении предела сложности, который доступен нам самим и применяемым средствам разработки.
- Не существует готовых рецептов проектирования и программирования, которые могли бы заменить интеллект, опыт и вкус.
- Экспериментирование незаменимо для любого нетривиального программного проекта.
- Проектирование и программирование — итеративные процессы.
- Невозможно четко отделить друг от друга такие фазы разработки, как проектирование, программирование и тестирование.
- Не стоит рассматривать проектирование и программирование в отрыве от вопросов управления этими процессами.

Легко недооценить эти положения (это может дорого обойтись в результате). Трудно превратить эти абстрактные идеи в реальную практику. Поэтому здесь крайне важно экспериментирование. Как в самых разных областях деятельности — в судостроении, в велосипедном спорте или программировании, твердые навыки проектирования невозможно получить на основе одной лишь теории.

Часто мы забываем про человеческий фактор в проектировании и рассматриваем процесс построения программных продуктов как «последовательность строго определенных шагов, каждый из которых порождает определенные действия над входными данными, соответствующими правилам их преобразования в требуемые выходные результаты». На самом деле, любой язык программирования, любые методики проектирования и построения на основе проекта программного кода обязаны принимать во внимание человеческий фактор. Забыть про это — загубить всю работу.

Данная глава нацелена на рассмотрение вопросов проектирования систем, находящихся на пределе возможностей коллектива разработчиков (или индивидуалов). Кажется, что в самой природе людей заключается их стремление браться за задачи, находящиеся на последней грани их ресурсов. Менее амбициозные проекты слабо нуждаются в подробном рассмотрении вопросов проектирования. У них имеются

свои привычные среды и приемы разработки, от которых не стоит отказываться. Однако для совершенно новых и чрезвычайно сложных задач нужны новые, более мощные и изощренные средства и методики. А проекты, которые «мы знаем как делать», можно отдать относительно новым новичкам, которые еще этого не знают.

Не существует единственно верного способа проектировать и строить любые системы. Я бы даже счел веру в единственно верный способ некоторой разновидностью детской болезни, если бы не изрядное количество подверженных этой вере опытных проектировщиков и разработчиков. Пожалуйста, помните, что если некоторая методика хорошо сработала у вас в прошлом году для некоторого проекта, это не значит, что она безо всяких изменений будет столь же хорошо работать у другого разработчика или для другого проекта. Очень важно, чтобы ваш ум был всегда открыт новым обстоятельствам и идеям.

Ясно, что большая часть обсуждения относится к разработке больших систем промышленного масштаба. Читатели, не имеющие отношения к таким разработкам, могут удобно развалиться в кресле и посмотреть на «ужасные картины», которых они избежали. Или они могут сосредоточиться на подмножестве вопросов, касающихся их индивидуальных проблем. Не существует столь малых программ, для которых были бы вредны общая проработка и создание проекта работы до ее выполнения. Однако существуют столь малые программные проекты, для которых подойдут любые частные приемы проектирования, программирования и документирования. См. §23.5.2 по поводу вопросов, связанных с масштабом программ.

Самой большой проблемой в разработке программ является сложность. Для борьбы со сложностью есть только одно универсальное средство — *разделяй и властвуй*. Задачу, которую удастся разделить на две относительно независимые подзадачи, можно считать более чем наполовину решенной. Этот простой принцип можно реализовывать разными способами. В частности, *применение модуля или класса* в проекте системы разделяет программу *две части* — на *реализацию* и ее *пользователей*, взаимодействующих (в идеале) исключительно через хорошо спроектированный *интерфейс*. Это фундаментальный подход к борьбе со сложностью программ. Аналогично, процесс проектирования программного продукта может быть разбит на отдельные действия с четко определенными взаимодействиями между участвующими в проекте людьми. Это фундаментальный подход к борьбе со сложностью процесса разработки и связанными с ним человеческими отношениями.

Для обоих случаев *вычленение отдельных частей и определение интерфейса их взаимодействия* требует максимального опыта и тонкого вкуса. Такое вычленение — это вовсе не простой механический процесс, а нечто, требующее проникновения в самую суть проблемы, для чего требуется глубокое понимание системы на подходящем уровне ее абстракции (см. §23.4.2, §24.3.1 и §25.3). Близорукий взгляд на процесс разработки программ часто приводит к их серьезным изъятиям. Отметим также, что *в реальной жизни любое разделение*, будь-то частей программы или людей, *выполняется значительно легче, чем их эффективное взаимодействие* по разные стороны барьера без разрушения этого барьера и с предоставлением средств коммуникации, достаточных для эффективной кооперации.

Настоящая глава рассматривает именно подход к проектированию, а не метод проектирования целиком, что очевидным образом выходит за рамки данной книги. Представленный нами подход может использоваться с разной степенью формализации, и он может служить основой для разных степеней формализации. Аналогич-

но, данная глава не служит литературным обзором по проектированию и разработке программных продуктов, так что в ней не затрагиваются все относящиеся к изучаемой проблеме вопросы и не перечисляются все известные точки зрения. Такой обзор представлен в книге [Booch, 1994]. Используемые в данной главе термины являются общепринятыми и широко употребляемыми. Особо повторяемые термины, такие как *проект (design)*, *прототип (prototype)* и *программист (programmer)* имеют в литературе множество разных и зачастую противоречащих друг другу определений. Пожалуйста, постарайтесь не попасться на удочку узко понимаемых определений и не прочтите в данной главе чего-нибудь такого, чего я вовсе не намеревался сказать.

23.3. Цели и средства

Целью профессионального программирования является изготовление и поставка законченного программного продукта, который удовлетворит заказчиков и/или иных его пользователей. Главным средством достижения такой цели является четкое понимание внутренней структуры программы, для чего требуется вырастить группу проектировщиков и программистов, достаточно квалифицированную и хорошо мотивированную для быстрого реагирования на новые обстоятельства и возможности.

Почему так? Ведь вроде бы внутренняя структура программы и детали процесса ее изготовления не должны волновать заказчика (пользователя). Более того, если конечный пользователь вынужден беспокоиться о таких деталях, то с программой явно что-то не в порядке. Тогда почему же внутренняя структура программы и качество изготавливаемой программы коллектива разработчиков так важны?

Ответ на этот вопрос таков — ясная внутренняя структура программы нужна для того, чтобы ее было легче (быстрее и надежнее):

- тестировать,
- портировать (переносить на другие системы),
- сопровождать и поддерживать,
- расширять,
- модифицировать,
- понимать.

Главная мысль состоит в том, что каждая успешная программа имеет определенное время жизни, в течение которого с ней работают разные группы проектировщиков и программистов, она переносится на новые виды аппаратуры, подстраивается под непредвиденные обстоятельства и, вообще, так или иначе периодически изменяется. За время жизни программного продукта своевременно появляются его новые версии с приемлемым уровнем ошибок. Не планировать указанных аспектов поведения программного продукта — заранее обрекать его на неудачу.

Далее, хотя конечные пользователи и не обязаны знать внутреннюю структуру программной системы, они вполне могут сами захотеть это сделать. Например, пользователь может захотеть узнать поподробнее внутренний дизайн системы для того, чтобы оценить ее надежность и/или способность к модификациям и расширению. Если программный продукт не представляет собой завершенной системы, например,

это может быть набор вспомогательных библиотек, то в этом случае пользователи захотят узнать побольше деталей с целью более эффективного их использования.

В реальных разработках приходится искать разумный баланс между отсутствием общего проекта (четкой внутренней структуры) и слишком большим упором на архитектурные аспекты. Первая крайность приводит к бесконечному «мы только сдадим эту программу, а все остальные проблемы решим в следующей версии». Вторая же крайность приводит к переусложненным проектам, в которых суть скрывается за формализмом и часто задерживается выпуск программ («новая структура программы *намного* лучше старой; люди с удовольствием подождут»). Кроме того, это часто приводит к излишним требованиям максимальных компьютерных ресурсов, которые пользователи не всегда могут себе позволить. *Такое балансирование — самый сложный аспект проектирования*, в котором ярче всего раскрываются настоящие таланты и действительно требуется изрядный опыт. Этот выбор труден даже для индивидуального проектировщика и программиста, и его тем более сложно сделать в больших программных проектах, в которых задействовано множество людей с разным опытом и квалификацией.

Программы должны производиться и обслуживаться организациями, способными делать это несмотря на смену персонала и менеджеров. Часто для этого стараются осуществлять разработку так, что вся работа разбивается на узкоспециализированные слои, жестко встраиваемые в общую структуру разработки. Для этого формируется множество быстро обучаемых (дешевых) и взаимозаменяемых программистов низкого уровня (кодировщиков — «coders») и множество не столь дешевых, но столь же взаимозаменяемых проектировщиков. От кодировщиков не требуется принимать архитектурных решений, в то время как проектировщики не обязаны разбираться в мелких и низкоуровневых деталях кода. Такой подход, однако, нередко ведет к неудачам. Там, где он все-таки работает, получаются переусложненные системы с плохой производительностью.

Проблемы этого подхода заключаются в следующем:

- *недостаточное взаимодействие и взаимопонимание* между проектировщиками и кодировщиками приводит к *упущенным возможностям*, задержкам, неэффективности и повторению проблем из-за того, что никто не накапливает предыдущий опыт и не учится на ошибках;
- *узкие рамки* работы специалистов *сковывают инициативу*, мешают профессиональному росту, способствуют разболтанности и высокой текучке кадров.

По сути дела, *такой системе не хватает каналов обратной связи*, чтобы люди могли плодотворно общаться и учиться на опыте других. Это растрата человеческого таланта. Создание среды и структуры осуществления процесса разработки программ, в которых люди проявляют таланты, развивают способности, предлагают свои идеи и радуются результатам — это не только благородная цель, достойная сама по себе, но и практическая вещь, приносящая экономическую выгоду.

С другой стороны, строить программную систему, документировать и поддерживать ее в безусловном порядке невозможно без некоторой бюрократической организационной структуры. Просто найти подходящих людей и дать им наброситься на задачу наилучшим с их точки зрения образом часто бывает идеальным началом для инновационных проектов. Дальше, по мере развития проекта, растет необходимость в планировании, специализации и установлении более формальных связей

между участниками проекта. Под словом «формальных» я не имел в виду нечто строго математическое (хотя в отдельных случаях и это не исключается), а скорее набор рекомендаций по именованию, документированию, тестированию и т.д. Тут, естественно, требуется вкус и чувство меры. Слишком жесткая организация может придушить рост и творчество. Здесь как раз и испытываются талант и опыт руководителя. Для индивидуальных программистов дилемма заключается в том, чтобы выбрать между изобретательством и копированием известных «книжных образцов».

Моя рекомендация — думать не только о ближайшем выпуске, но и о более долгосрочных проблемах. Беспокоиться исключительно о ближайшем выпуске программы — значит планировать неудачу. Нужно разрабатывать стратегию организации процесса разработки программных продуктов, нацеленную на множество выпусков множества продуктов, то есть нужно планировать целую серию успехов.

Цель проектирования состоит в построении ясной и относительно простой *внутренней структуры программы*, иногда называемой *архитектурой*. Другими словами, мы создаем инфраструктурную среду, в которую должны вписываться отдельные части программного кода, и которая сама направляла бы написание отдельных фрагментов.

Проект — это конечный результат проектирования (если, конечно, у итеративного процесса есть конечный результат). Именно проект организует взаимодействие между проектировщиком и программистом, а также между разными группами программистов. Тут возможны две крайности. Для индивидуального программиста и небольшой задачи, которую нужно решить за пару дней, проект может заключаться в нескольких набросках на обратной стороне почтового конверта. В то же время, для разработок, в которых участвуют сотни проектировщиков и программистов, проект может вылиться во многие тома спецификаций, тщательно выполненных в соответствии с устоявшимися (формальными или полуформальными) стандартами. Выявление подходящего уровня детализации, точности и формализации проекта — сама по себе непростая техническая и управленческая задача.

В данной и двух последующих главах книги я предполагаю, что проект системы заключается в наборе объявлений классов (в типичном случае с опущенными закрытыми секциями объявлений как несущественными подробностями) и указании связей между ними. Это, конечно, упрощение. Для реального проекта имеют значение огромное множество деталей, таких как возможность параллельного исполнения кода, группировка идентификаторов в пространства имен, применение глобальных функций и данных, параметризация классов и функций, реорганизация структуры программы с целью минимизации перекомпиляций, возможность работы в распределенной сетевой среде и т.д. Тем не менее, наше упрощение оправдано, так как позволяет сосредоточиться на том уровне проектной детализации, для которого ключевое значение имеют классы языка C++. Остальные проектные особенности частично затрагиваются в данной главе, а те из них, что напрямую связаны с особенностями языка C++ — в главах 24 и 25. За более детальными сведениями об объектно-ориентированном проектировании обратитесь к книге [Booch, 1994].

Я не сосредотачиваюсь на различиях между анализом и проектированием, поскольку эта тема выходит за рамки настоящей книги, и она, к тому же, очень чувствительна по отношению к разным проектным методикам. Важно так выбрать метод анализа, чтобы он соответствовал методу проектирования, а метод проектирования — чтобы соответствовал стилю программирования и языку программирования.

23.4. Процесс разработки

Разработка программного продукта — процесс итеративный. За время разработки многократно возвращаются к одним и тем же промежуточным стадиям, каждый раз усовершенствуя соответствующие им состояния проекта. Вообще говоря, такой процесс не имеет конца. Начиная работать, вы обычно обращаетесь к некоторой базе ранее выполненных другими людьми проектов, к библиотекам и прикладным средам разработки. Заканчивая проект, вы оставляете его другим для улучшений и расширений, а также для переноса на иные платформы. Естественно, конкретный проект должен иметь определенные начало и конец, и желательно (хотя это и трудно) четко планировать процесс разработки программного продукта во времени. Думая, что стартуете с чистого листа, вы лишь затрудняете себе работу. Полагая, что все в мире заканчивается с последним выпуском вашей программы, вы ставите преемников в затруднительное положение (и себя самого в этом качестве).

Последующие разделы можно читать почти что в произвольном порядке, так как аспекты проектирования и реализации могут произвольно перемежаться в реальных проектах. То есть проект почти всегда переделывается, отталкиваясь от опыта предыдущих проектов и их реализаций. Кроме того, его ограничивают календарные планы, квалификация персонала, необходимость обеспечить совместимость с предыдущими версиями или иными программами и т.д. Очень важно для проектировщиков, менеджеров и программистов внести, тем не менее, в процесс разработки определенный порядок, не удушающий творчество и не разрушающий обратные связи между людьми, способствующие конечному успеху дела.

Процесс разработки можно разделить на три этапа:

- *анализ*: выявление границ решаемой задачи (проблемы);
- *проектирование*: создание общей внутренней структуры системы;
- *реализация*: написание кода и тестирование программы.

Пожалуйста, не забывайте об итеративной природе процесса — важно, что перечисленные этапы не пронумерованы. Также стоит отметить, что некоторые аспекты разработки программ не подпадают под разбиение на этапы:

- Экспериментирование
- Тестирование
- Анализ проекта и реализации
- Документирование
- Управление проектом (менеджмент)

Поддержка и сопровождение программы сводятся к дополнительным проходам по всему циклу разработки (§23.4.6).

Очень важно, чтобы анализ, проектирование и реализация не слишком отдалялись друг от друга, и чтобы люди, ответственные за разные виды деятельности, могли при этом взаимодействовать и обмениваться мыслями и опытом. Слишком часто в больших проектах это отсутствует. В идеале, в процессе выполнения проекта люди должны изменять свое амплуа и переходить от одного этапа к другому — только так можно осуществить перенос важной и тонкой информации, хранящейся в голове человека. К сожалению, многие организации выставляют барьеры на пути

таких человеческих переходов, предоставляя, например, проектировщикам более высокий статус и/или зарплату, чем «простым программистам». Если уж среди сотрудников не практикуются переходы туда-сюда с целью учить и учиться, то желательно поощрять их беседовать с участниками «других» этапов разработки.

Для небольших и средних проектов часто не делается различия между анализом и проектированием; эти две фазы сливаются в одну. А для маленьких проектов не делается различия и между проектированием и реализацией. При этом, конечно, автоматически решаются «проблемы общения». Любому проекту важно придать подходящий уровень формализации и поддерживать соответствующую степень разделения этапов (§23.5.2). Тут не существует единственно верного решения.

Описанная здесь модель разработки программных продуктов радикально отличается от традиционной «модели водопада». В последней процесс разработки разворачивается в линейную последовательность шагов от анализа до тестирования. «Модель водопада» имеет серьезный изъян, заключающийся в том, что информация в рамках этой модели «течет» лишь в одном направлении. Когда проблемы обнаруживаются «внизу по течению», возникает сильное методологическое и организационное давление с тем, чтобы решать их на месте выявления, не затрагивая предыдущие этапы процесса. Такое намеренное игнорирование обратных связей обедняет проектное решение, а исправления «по месту возникновения» часто порождают ущербные программы. В случаях, когда информация с неизбежностью вытекает назад, приходится изменять проект, что замедляет разработку и вызывает нескладный волнообразный эффект в системе, предполагающей отсутствие таких изменений, а потому сопротивляющейся и медленно реагирующей на изменения. В качестве довода в пользу отсутствия изменений и решения проблем на месте приводят суждение о том, что одно подразделение не должно ради своего удобства перекладывать работу на плечи другого подразделения. Наконец, когда выявляется проблема, исключительно плохо преодолеваемая на месте, оказывается, что к этому моменту выполнена грандиозная бумажная работа по документированию, переделать которую столь сложно и накладно, что лучше уж любой ценой и как-нибудь все исправить по-прежнему «на месте». Таким образом, бумажная работа оказывается важнейшим фактором в разработке программного продукта. Разумеется, при любой организации работ такие проблемы могут и наверняка возникают в рамках больших программных проектов. Ясно, что какие-то бумаги действительно необходимы. Однако применение для разработки линейной модели (модели «водопада») увеличивает вероятность того, что бумажные проблемы могут выйти из под контроля.

Недостаток модели «водопада» заключается в недостаточности обратной связи и неспособности реагировать на изменения. Опасность же описанного ранее итеративного подхода заключается в возможной попытке подменить необходимость думать серией плохо сходящихся изменений. Обе проблемы легче описать, чем решить, и в любом случае имеется соблазн счесть бурную деятельность гарантией успешного продвижения к желаемой цели. Естественно, что по мере продвижения проекта разные его фазы становятся более или менее важными. В самом начале акцент делается на анализе и проектировании, а вопросы написания конкретного кода в этот момент не столь актуальны. Постепенно акцент перемещается на связку проектирование-программирование, а затем и на связку программирование-тестирование. Однако никогда нельзя 100%-но фокусироваться на отдельных стадиях, забывая про все остальные.

Помните, что *если вы не знаете, чего хотите достичь, ничего не поможет* — ни внимание к деталям, ни правильные приемы менеджмента, ни передовые методики проектирования, ничего. Большая часть проектов проваливается как раз по причине отсутствия четко определенной и реалистичной цели. Нужно ясно понимать свою цель, четко определять промежуточные фазы разработки и не пытаться подменять техническими приемами отсутствие перечисленного. При наличии четкой цели используйте все доступные методы, даже если они требуют серьезных инвестиций; люди работают лучше в условиях хорошего финансирования и благоприятной социальной и технологической среды. Однако не думайте, что следовать этому совету легко.

23.4.1. Цикл разработки

Разработка программной системы является итеративным процессом. Основной цикл заключается в повторе следующих шагов:

0. Исследование проблемы.
1. Создание общего проекта.
2. Нахождение стандартных компонентов. Приспособление компонентов к проекту.
3. Создание новых стандартных компонентов. Их приспособление к проекту.
4. Сборка проекта.

Для аналогии рассмотрим автомобильный завод. Чтобы запустить производство, нужен полный общий проект нового типа автомобиля. Самый первый набросок будет отталкиваться от предварительного анализа и состоять в словесном описании, относящемся скорее к области применения автомобиля, а не к техническим деталям его устройства, позволяющим добиться заявленных целей. Принятие решения о требуемых свойствах автомобиля является самым трудным решением. *Правильно принятое решение* часто является *плодом озарения одной сверхпроницательной личности*. Многие проекты испытывают дефицит четко заявленных целей, и по этой причине они часто терпят неудачу.

Допустим, что мы хотим построить среднего размера седан с четырьмя дверями и довольно мощным двигателем. После этого совершенно необязательно набрасываться на выполнение проекта автомобиля (и его узлов) с нуля. Аналогично, программисту на этом этапе также нет смысла сразу же составлять программный проект и начинать писать код.

Намного лучше сначала посмотреть, какие компоненты уже имеются на заводе или у надежных поставщиков. Найденные таким образом компоненты не обязаны в точности подходить для нового автомобиля. Их всегда можно будет в дальнейшем приспособить для этой цели. Неплохо сразу же написать спецификации для будущих версий этих компонентов, чтобы они лучше подходили для нового автомобиля. Например, пусть имеется двигатель со всеми подходящими характеристиками, кроме слегка недостаточной мощности, но есть возможность слегка турбировать этот двигатель для достижения желаемой мощности без внесения изменений в базовую конструкцию. Выполнить такое турбирование без внесения изменений в базовый проект двигателя можно лишь тогда, когда такой аспект модификаций был учтен при выполнении исходного проекта двигателя. Ясно, что эта форма доработки двигателя выполняется во взаимодействии с поставщиком двигателя.

Проектировщик программного продукта и программист находятся в схожем положении. Например, они могут *использовать полиморфные классы и шаблоны для подгонки готовых компонентов*. Но, конечно, не следует думать, что можно осуществить произвольную подгонку, если только разработчик компонента *заранее не заложил в него эту возможность*.

Выявив наличие и определив возможности готовых стандартных компонентов, разработчик не бросается на проектирование новых (недостающих) компонентов для автомобиля. Например, оказалось, что нигде нет подходящего кондиционера, а под капотом есть свободное пространство L-образной формы. Прямолинейным решением будет разработка кондиционера L-образной формы. Но вероятность приспособить в дальнейшем такую экзотику под другие машины (даже после сильной подгонки) крайне мала. В результате не удастся снизить совокупную стоимость разработки за счет тиражирования решения на большую партию самых разных машин, а эффективная жизнь нашего кондиционера будет весьма короткой. В результате более мудрым решением будет разработка собственного кондиционера традиционных форм и габаритов с широкими возможностями приспособления к разным конкретным случаям и нуждам. Это, конечно, займет больше времени и сил, чем проработка частного L-решения, и возможно повлечет за собой внесение изменений в первоначальный дизайн разрабатываемого нового автомобиля, чтобы он мог принять кондиционер более традиционной формы и габаритов. Возможно даже, что более традиционный кондиционер после этого снова нужно будет довести (в соответствии с заложенными в его проект вариантами модификации), чтобы сделать его великолепно подходящим новому автомобилю. И снова отмечаем, что проектировщик системы и программист находятся в аналогичном положении, и им тоже не следует сразу же писать узкоспецифичный код вместо разработки универсального решения, которое имеет шанс стать общеупотребительным стандартом в будущем.

Наконец, исчерпав потенциал готовых стандартных компонентов, мы собираем «окончательный» проект. Мы используем в нем столь малое количество специфических компонентов, насколько это возможно, ибо в уме мы держим простую мысль, что уже в следующем году нам придется все начинать сначала, разрабатывая очередную модель автомобиля, и специфические компоненты при этом придется либо сильно переделывать, либо выбросить совсем. Печально, но опыт разработки программных компонентов говорит о том, что немногие части программы удастся хотя бы выделить в четко очерченные компоненты, и еще меньше из них находит свое применение за пределами оригинального проекта.

Я не утверждаю, что все проектировщики автомобилей поступают столь рационально, как мы это описали, и не все проектировщики программ допускают перечисленные выше ошибки. Я, просто, твердо полагаю, что описанную (автомобильную) модель можно и нужно с разумными поправками использовать в процессе разработки программных продуктов. Данная глава и последующая рассматривают технологии, позволяющие применять описанную модель в рамках языка C++. Но я все же подчеркиваю, что из-за нематериальной природы компьютерных программ трудно полностью избежать указанных ошибок (§24.3.1, §24.3.4), и в §23.5.3 я показываю, что от использования приведенной здесь модели людей часто удерживают корпоративные стандарты.

Отметим, что эта модель разработки хороша именно в долгосрочной перспективе. Если ваш горизонт распространяется только до ближайшего выпуска, разработ-

ка и создание стандартных компонентов теряет для вас всякий смысл. Это будет выглядеть для вас излишними проблемами (и затратами). Наша модель предлагается для организаций, трудовая жизнь которых распространяется на многие выпуски крупных программных продуктов, чтобы могли оправдать себя дополнительные инвестиции в инструменты (менеджмента, проектирования и программирования) и обучение персонала (менеджеров, проектировщиков и программистов). Можно считать это наброском фабрики по производству программных продуктов. Забавно, но он отличается лишь масштабом от реальной практики талантливых и предусмотрительных индивидуальных программистов, которые с годами набирают отличный запас приемов, проектов, инструментов и библиотек, резко усиливающих их персональную производительность. Похоже, что многим организациям не удастся воплотить в жизнь работающие методики индивидуальных программистов из-за близорукости или неумения управлять ими в более крупных масштабах.

Отметим, что совершенно *неразумно ожидать* от стандартных компонентов *абсолютной универсальности*. Могут появиться несколько международных стандартных библиотек, но большинство из них останется стандартными лишь в рамках конкретных стран, отраслей, организаций, отделов и прикладных областей. Мир просто слишком велик, чтобы ограничиваться немногими универсальными стандартами.

Стремление к чрезмерной универсальности на начальных стадиях самых первых проектов почти наверняка повлечет за собой то, что эти проекты никогда не будут завершены. Цикл разработки потому и цикл, что нужно черпать опыт из полученных в рамках очередной итерации работающих систем (§23.4.3.6).

23.4.2. Цели проектирования

Каковы задачи, стоящие перед проектированием? Безусловно достижение простоты, но простоты в каком смысле, ведь проект должен развиваться? Он будет расширяться, портироваться, настраиваться и вообще меняться столь большим числом способов, что их невозможно предвидеть. Следовательно, мы должны стремиться к тому, чтобы в проект системы и ее реализацию было легко вносить изменения. Нет ничего удивительно в том, что требования к системе и изменения в проект будут несколько раз вноситься даже при разработке самого первого выпуска продукта.

Из всего этого следует, что система должна проектироваться таким образом, чтобы оставаться сколь возможно простой под воздействием серии модификаций. Мы должны закладывать изменения в проект, то есть мы должны стремиться к:

- гибкости,
- расширяемости,
- переносимости.

Лучше всего это достигается попыткой изоляции и *инкапсулирования тех частей системы*, которые *более всего подвержены потенциальным изменениям*, с тем чтобы в дальнейшем проектировщики и программисты вносили изменения в четко очерченные отдельные части системы. Это выполняется путем выявления ключевых концепций разработки и соотнесения их с определенным классом, в исключительное ведение которого передается вся работа с информацией, относящейся к концепции. В таком случае изменения можно сосредоточить в единственном классе.

В идеале, необходимые изменения в концепции можно было бы выполнять с помощью введения производного класса (§23.4.3.5) или передачей нового параметра шаблону. Конечно, *идеал легче провозгласить, чем реально осуществить*.

Рассмотрим пример. В программе моделирования погодных явлений возникла необходимость визуализировать тучу. Как нам сделать это? Мы не можем написать внешнюю процедуру для показа тучи, поскольку ее вид зависит от ее внутреннего состояния, а о нем должна знать только сама туча.

Первое решение — пусть туча сама себя показывает. Во многих ограниченных случаях такой стиль решения проблемы является приемлемым. Он, однако, не является универсальным, ведь можно по-разному показывать тучу; детальным образом, схематично, в виде иконки на карте и т.д. Другими словами, вид тучи зависит не только от самой тучи, но и от окружающего контекста.

Второе решение — пусть туча знает окружающий контекст и сама себя показывает. Это решение более универсально. Но оно по-прежнему не является общим решением. Оно нарушает принцип, заключающийся в том, что туча знает все о себе и ни о ком другом, а за каждую отдельную концепцию в программе отвечает какой-то определенный класс. Также вряд ли возможно согласованное определение понятия «окружение тучи», поскольку внешний вид тучи зависит даже от наблюдателя. Даже в обычной жизни то, как я вижу тучу, зависит от того, смотрю ли я на нее невооруженным глазом, или через поляризационный фильтр. Также этот вид зависит и от определенного общего фона, например от положения солнца. Добавление иных объектов, таких как другие облака или самолеты, еще более усложняет проблему. А чтобы еще более усложнить задачу проектировщику, добавьте возможность присутствия нескольких наблюдателей.

Третье решение — пусть туча и все остальные объекты, такие как солнце или самолеты, докладывают о себе наблюдателю. Это решение еще более универсально. Но оно может оказаться слишком сложным, в том числе с точки зрения необходимости огромных вычислительных мощностей и иных компьютерных ресурсов. Например, как понятным образом организовать данные, передаваемые наблюдателю перечисленными выше объектами?

Тучи не являются частыми гостями программных проектов (для примера см. §15.2), но вот другие объекты, которые вовлекаются во множество операций ввода/вывода, встречаются часто. Это делает пример с тучей вполне уместным в контексте разработки программ и особенно в проектировании библиотек. Код C++ для логически похожего примера встречается в контексте манипуляторов, используемых для форматированного вывода в библиотеке потоков (§21.4.6, §21.4.6.3). Заметьте, что третье решение не является самым «правильным» решением — оно лишь самое универсальное. Проектировщик должен балансировать между множеством различных требований, чтобы выбрать оптимальный уровень общности/абстрактности, подходящий для конкретной проблемы в рамках данной системы. Выскажем грубое эмпирическое соображение, что оптимальным уровнем абстракции для программ длинного цикла жизни является такой, который предоставляет наибольшую универсальность, которую вы только можете себе позволить и воспринять. Чрезмерная универсальность, выходящая за рамки возможностей индивидуального понимания и технических рамок проекта, причинит один лишь вред: вызовет задержки, приведет к неприемлемо низкой производительности, неуправляемому проекту и, в конце концов, просто к катастрофе.

Чтобы сделать приемы проектирования экономически приемлемыми и управляемыми, мы должны учитывать необходимость повторного использования проекта (§23.5.1) и не забывать совсем уж об эффективности (§23.4.7).

23.4.3. Этапы проектирования

Рассмотрим проектирование отдельного класса. В общем случае, это не слишком хорошая идея. *Концепции не существуют в полной изоляции*; скорее, концепции определяются в контексте других концепций. Точно так же, и *классы не существуют изолированно*. Они определяются совместно с логически связанными с ними классами. Как правило, мы работаем с набором логически связанных классов. Такой набор классов принято называть *библиотекой классов* или *компонентом* (*class library* или *component*). Иногда все классы компонента составляют единую иерархию классов, иногда — определяются в единственном пространстве имен, а иногда они являются просто произвольным набором объявлений (§24.4).

Набор классов объединяется в компонент согласно некоторому логическому критерию, часто по общему стилю, но еще чаще по общему сервису. Таким образом, *компонент* — это *единица проекта, документации и объект повторного использования*. Это не значит, что если применяете один класс из компонента, вы должны понимать код всех остальных классов и использовать их, загружая вхолостую память машины. Наоборот, мы стремимся использовать классы с минимальными затратами машинных ресурсов и человеческих усилий. Однако чтобы уверенно пользоваться каким-либо классом компонента, нам нужно понять объединяющий классы логический критерий (хорошо, если он ясно задокументирован), соглашения и стиль программирования, а также использование общих ресурсов (если это имеет место).

Итак, рассмотрим, как можно было бы спроектировать компонент. Поскольку часто это непростая задача, разобьем ее на отдельные шаги, чтобы сфокусироваться на отдельных подзадачах логически полным образом. Как обычно, не существует единственного способа сделать такое разбиение. Вот, однако, последовательность шагов, которая помогла многим:

1. Выявите концепции/классы и их фундаментальные взаимосвязи.
2. Уточните классы, определив набор их операций.
 - Классифицируйте эти операции, в том числе определите необходимость конструкторов, деструкторов и операций копирования.
 - Обеспечьте минимальное решение, достаточно полное и удобное.
3. Уточните классы в плане их зависимостей.
 - Параметризация, наследование и иные типы зависимостей.
4. Определите интерфейсы.
 - Разделите функции на открытые и защищенные.
 - Определите точный тип операций над классом.

Помните, что это шаги итеративного процесса. Как правило, чтобы получить удобный в работе проект, нужно неоднократно пройти по этим шагам. Одним из преимуществ качественно выполненного анализа и абстракции данных является относительная простота изменения взаимозависимостей классов даже после того,

как написан конкретный код программы. Это, конечно, не совсем тривиальная задача.

После этого мы реализуем код классовых методов и пересматриваем проект, учитывая опыт, полученный при реализации кода. В последующих подразделах данной главы мы обсудим все эти шаги последовательно, один за другим.

23.4.3.1. Этап 1: отражение концепций классами

Выявите концепции/классы и их фундаментальные взаимосвязи. Ключ к хорошему проекту — это непосредственное моделирование реальностей прикладной задачи в виде классов, представление их взаимосвязей известными способами, например в виде наследования, причем все это нужно периодически повторять на разных уровнях абстракции. Но как на практике нужно выискивать концепции/классы? Как понять, какие именно классы нам нужны?

Лучше всего начать такой поиск прямо в самом приложении, а не рыться в портфеле абстракций и концепций компьютерного ученого. Послушайте того, кто собирается стать профессиональным пользователем вашей системы, или того, кто недоволен существующей системой. Обратите внимание на термины, которыми они при этом оперируют.

Часто говорят, что существительные соответствуют классам и объектам программы (и это часто соответствует действительности). Ясно, однако, что это далеко не конец истории. Глаголы соответствуют операциям над объектами, традиционным (глобальным) функциям, возвращающим новые значения на базе их значений-аргументов, и даже классам. Последний случай иллюстрируется классами функциональных объектов (§18.4) и манипуляторами (§21.4.6). Такие глаголы, как «итерировать» и «выполнить (как единое целое)» соответствуют поведению итераторных объектов и объектов, отвечающих за целостное исполнение группы операций (транзакций) над базами данных. Иногда даже прилагательные можно с пользой для дела представлять классами. Например, прилагательные «хранимый», «параллельный», «зарегистрированный» или «связанный» могут быть представлены проектировщиком в виде классов, которые в дальнейшем в качестве виртуальных базовых классов (§15.2.4) помогут придать классам иерархии желательные дополнительные атрибуты.

Не всякий класс имеет прямое соответствие реальностям прикладной задачи. Некоторые представляют системные ресурсы и абстрактные детали реализации (§24.3.1). Также важно отойти от близкого соответствия моделям старой программной системы. Например, вряд ли в новой системе нужно подражать старой системе, которая для работы с данными помогала пользователям моделировать «перекладывание бумажек».

Наследование применяется для отражения общности концепций. Что самое важное, наследование должно отражать иерархические отношения классов, моделирующих индивидуальные концепции (§1.7, §12.2.6, §24.3.2). Иногда это называют *классификацией* (*classification*) или *таксономией* (*taxonomy*). Активно выискивайте общности. Обобщение и классификация — это абстрактная деятельность, требующая проникновения в самую суть вещей для получения полезных и надежных результатов. Общий базовый класс должен отражать самую общую концепцию, а не просто похожую концепцию с минимальными данными для представления.

Отметим, что классификации должны быть подвергнуты те аспекты концепций, которые мы моделируем в нашей системе, а не аспекты, имеющие значение в других случаях. Например, в математике окружность — это частный случай эллипса, но в большинстве программ окружность не моделируют в свете ее связи с эллипсом, и эллипс не представляют в виде типа, производного от типа окружности. Тут такие аргументы, что мол так обстоит дело в математике, не работают. Для большинства программ ключевое свойство окружности — это равноудаленность ее точек от центра окружности. Это свойство должно поддерживаться всеми операциями типа окружности (то есть это свойство является инвариантом; §24.3.7.1). С другой стороны, эллипс характеризуется наличием двух фокальных точек, которые во многих программах можно изменять независимым образом. Если эти точки совпадают, эллипс становится похож на окружность, но это не окружность, поскольку его операции не сохраняют инварианта окружности. В большинстве систем это отличие будет отражено в разных наборах операций для окружности и эллипса, так что один набор не является подмножеством другого.

Мы не просто определяем набор классов и отношений между ними и используем их в окончательной системе — мы на самом деле создаем первоначальный набор классов и их взаимоотношений с тем, чтобы многократно совершенствовать этот набор (§23.4.3.5) и в итоге получить универсальный, гибкий и стабильный набор, помогающий на всех будущих стадиях эволюции системы.

Наилучшим инструментом для поиска и выявления концепций/классов является обычная классная доска. Уточнение начального набора лучше всего выполнять в процессе дискуссий со специалистами в предметной области задачи и некоторыми друзьями. Дискуссии нужны для выявления *первоначального словаря задачи и базовой системы концепций*. Редкие люди могут делать это в одиночку. Дальше с целью уточнения и улучшения классов из первоначального набора стоит попытаться смоделировать систему, используя проектировщиков в роли классов. Это может выявить абсурдность некоторых первоначальных идей, стимулировать предложение альтернатив и создать согласованное понимание системы у всех участников. Для документирования дискуссии можно применить, например, CRC-карточки (Class, Responsibility, Collaborators) [Wirfs-Brock, 1990], которые так называются в соответствии с записываемой на них информацией (Класс, Ответственность, Сотрудники).

Примеры использования (use cases) — это описание частных случаев использования системы. Вот простой пример использования телефонной системы: поднимаем трубку, набираем номер, телефон на другом конце звонит, там снимают трубку. Разработка набора примеров (частных случаев) использования чрезвычайно полезна на всех этапах разработки. На начальном этапе выявление полезных частных случаев применения нашей системы помогает нам глубже понять, что именно мы хотим разработать. На этапе проектирования они помогают нам интроспектировать систему (например, через CRC-карточки) с целью убедиться, что относительно статическое описание системы в терминах классов и объектов имеет смысл с точки зрения пользователя системы. На этапах программирования и тестирования примеры использования становятся источниками тестов. Таким образом, примеры частных случаев использования вносят ортогональный взгляд на систему с точки зрения ее соответствия реальности.

Примеры использования описывают систему как (динамическую) работающую сущность. Поэтому они могут сконцентрировать внимание проектировщика на

функциональных аспектах системы и отвлечь его внимание от поиска важных концепций, отображаемых в классы. Это особенно опасно в том случае, когда проектировщик имеет большой опыт в структурном анализе и недостаточный опыт в объектно-ориентированном проектировании/программировании, так что упор на частных примерах использования может привести в итоге к функциональной декомпозиции. Вообще, набор примеров использования — это не проект. *Примеры использования системы должны быть уравновешены анализом структуры системы.*

Команда разработчиков также может попасться в ловушку погони за полным набором примеров использования. Такие ошибки дорого обходятся. Как и в случае поиска важных сущностей и соответствующих им классов приходит время остановиться и сказать: «Хватит! Наступил момент испробовать то, что мы уже получили и посмотреть, как это работает». Применяя подходящий набор классов и подходящий набор примеров использования можно двигаться вперед, приобретая опыт и обратную связь для корректировки проекта. Всегда трудно решить, когда же нужно остановиться. Особенно трудно остановиться тогда, когда известно, что позже все равно придется вернуться для завершения проекта.

Сколько нужно примеров? В общем случае на этот вопрос ответить невозможно. Тем не менее, в конкретном проекте всегда приходит время, когда становится ясно, что большая часть стандартной функциональности системы уже отражена в текущем варианте проекта, а некоторая часть специфического поведения и обработки ошибок до некоторой степени изучена. Здесь и наступает момент, когда нужно переходить к следующим этапам проектирования и программирования.

Когда вы пытаетесь оценить процент покрытия системы набором примеров использования, полезно разделить все имеющиеся примеры на первичные и вторичные. Первичные описывают наиболее общие и нормальные действия, а вторичные — менее обычные действия и сценарии обработки ошибок. Часто говорят, что когда 80% первичных примеров и некоторое количество вторичных описаны, можно заканчивать сбор примеров, но поскольку мы не знаем, что такое 100%, то эту рекомендацию следует воспринимать как ориентировочную. Опыт и вкус сослужат здесь неоценимую пользу.

Концепции, операции и взаимосвязи естественным образом вытекают из нашего понимания прикладной области задачи или появляются в процессе практической работы с существующей классовой структурой. Они составляют основу фундаментального понимания приложения. Часто они порождаются классификацией, такой как, например, «машина с раздвижной лестницей» есть частный случай пожарной машины, которая есть частный случай грузовика, который есть частный случай колесного транспортного средства. Разделы §23.4.3.2 и §23.4.5 знакомят с несколькими способами взгляда на классы и классовые иерархии с точки зрения их улучшения.

Остерегайтесь излишней графической инженерии. В некоторый момент вас попросят представить проект кому-нибудь в графической форме, раскрывающей структуру разрабатываемой системы. Это может оказаться для вас положительным опытом, так как заставит сосредоточить внимание на том, что действительно важно для системы и заставит представить все это в форме, понятной окружающим. Презентации можно рассматривать как ценное проектное средство. Подготовка презентации для людей, имеющих к проекту интерес и возможность порождать конструктивные замечания, является испытанием для качества разработанных вами концепций и способности ясного изложения идей.

В то же время, формальная презентация проекта также имеет и отрицательные стороны, ибо подталкивает к тому, чтобы представить проект идеальной системы, а не той, которая у вас уже есть или которую можно произвести в реальные сроки. Когда конкурируют разные подходы, а начальство на самом деле не понимает деталей или не заботится о них, презентация превращается в ложное соревнование, в котором побеждает та команда, которая представит наиболее грандиозный проект ради сохранения заказа для своего коллектива разработчиков. В таких случаях ясное изложение идей часто подменяется тяжеловесным жонглированием малопонятными аббревиатурами. Если вы на такой презентации представляете руководство, то вам чрезвычайно важно отличать благие пожелания от реалистичного планирования. Визуальное качество презентации не гарантирует такого же качества у проекта. Более того, зачастую организации, сосредоточенные на качественном выполнении проектов, проигрывают тем, кто меньше беспокоится о реальном производстве эффективных программ.

Наконец, у всех систем существуют такие свойства, которые не представимы классами. Например, надежность, производительность и удобство тестирования не отвечают никаким классам программы, хотя их и можно измерить. Такие свойства систем можно закладывать в проект и измерять в процессе работы программных продуктов. Забота об этих свойствах должна красной нитью проходить через все классы, и ее можно отразить в проектировании и реализации классов и компонентов (§23.4.3).

23.4.3.2. Этап 2: определение операций

Уточните классы, определив набор их операций. Конечно, невозможно полностью отделить выявление концепций и классов от определения операций над ними. Но в процессе поиска классов мы сосредотачиваемся в первую очередь на сущности концепций без упора на операции, в то время как на втором этапе уже можно сосредоточиться именно на операциях, определяя их полный и удобный набор. Действительно, трудно было бы делать это одновременно, особенно в случае параллельной разработки многих, связанных друг с другом классов. Здесь полезными могут оказаться CRC-карточки (§23.4.3.1).

Для выяснения набора необходимых операций могут оказаться полезными следующие соображения:

1. Рассмотрите, как объекты классов должны создаваться (конструироваться), копироваться и уничтожаться.
2. Определите минимальный набор операций, который требует отражаемая классом концепция. Как правило, эти операции реализуются функциями-членами (§10.3).
3. Подумайте, какие операции можно добавить для удобства. Включите только несколько наиболее важных операций. Часто такие операции реализуются в виде *вспомогательных функций* (*helper functions*) (§10.3.2).
4. Подумайте, какие операции должны быть виртуальными, то есть для которых класс обеспечивает интерфейс, а реализацию предоставляет производный класс.
5. Подумайте, какой общности именования и функциональности можно добиться для всех классов компонента.

Это — манифест минимализма. Ясно, что легче добавить, не думая, все функции, которые когда-либо могут понадобиться, и сделать их виртуальными, но чем больше функций, тем более вероятно, что они останутся неиспользованными и только затруднят реализацию системы и ее дальнейшие модификации. В частности, функции, которые непосредственно читают или записывают некоторую часть состояния объекта класса ограничивают класс единственной стратегией развития и сильнейшим образом снижают потенциал развития системы. Такие функции понижают уровень абстракции от концепции до конкретного варианта реализации. Большое количество функций лишь добавляет работы реализующему код программисту и проблемы проектировщику системы при ее дальнейшем совершенствовании. *Гораздо лучше* добавить функцию в тот момент, когда четко выявилась необходимость в этой функции, чем удалять функцию при осознании ее ненужности.

Нужно принимать решение о виртуальности функции самым что ни на есть явным образом (а не в виде детали реализации), ибо это критически влияет на применение класса и на его взаимосвязи с другими классами. Объекты класса всего лишь с одной виртуальной функцией имеют нетривиальную структуру по сравнению с объектами таких языков, как С или Fortran. Класс с единственной виртуальной функцией потенциально может действовать как интерфейс к еще не определенному классу, а виртуальные функции подразумевают зависимость от еще не определенных классов (§24.3.2.1).

Отметим также, что минимализм на самом деле *требует от проектировщика больше работы*, а не меньше.

При выборе операций важно сосредоточиться на том, что нужно сделать, а не на том, как это сделать. То есть нам нужно сосредоточиться на желательном поведении, а не на деталях реализации.

Бывает полезно классифицировать операции с точки зрения того, как они используют внутреннее состояние объектов:

- **Фундаментальные операции:** конструкторы, деструкторы и операции копирования.
- **«Инспекторы»:** операции, не модифицирующие состояние объектов.
- **Модификаторы:** операции, модифицирующие состояние объектов.
- **Преобразования:** операции, производящие объект другого типа, отталкиваясь от значения (состояния) объекта, к которому они применяются.
- **Итераторы:** операции, служащие для доступа к последовательности объектов, хранящихся в контейнере.

Перечисленные классификационные категории не ортогональны друг другу. Например, итератор может быть также инспектором или модификатором. Эти категории являются просто классификацией, которая помогла многим людям совершенствоваться в проектировании интерфейсов. Естественно, возможны и другие классификации. Такие классификации помогают согласовывать классы компонента.

Язык С++ помогает отличать инспекторов от модификаторов за счет поддержки константных и неконстантных функций-членов. Аналогично, непосредственно в самом языке поддерживаются конструкторы, деструкторы, операции присваивания и операции приведения (преобразования) типов.

23.4.3.3. Этап 3: выявление зависимостей

Уточните классы в плане их зависимостей. Различные виды зависимостей обсуждаются в §24.3. В контексте проектирования ключевыми являются зависимости параметризации, наследования и использования. Для каждой из них важно ответить на вопрос о том, что означает, что класс ответственен за какое-то одно из свойств системы. Чтобы классу быть ответственным за что-то, ему не обязательно самому содержать все необходимые для этого данные, или непосредственно выполнять все требуемые операции. Более того, классы с фиксированной ответственностью за отдельные свойства системы чаще всего переправляют идущие на них запросы другим классам, которые отвечают за решение частных подзадач. Естественно, тут можно переборщить и создать чрезвычайно неэффективную и труднопонимаемую архитектуру, в рамках которой никакая работа не делается иначе, как порождением каскада обращений объектов за сервисом друг к другу. То, что можно сделать на месте, должно делаться сразу на этом месте.

Необходимость рассмотрения вопросов наследования и отношений использования на этапе проектирования (а не реализации) вытекает из перехода от простого использования классов к представлению концепций задачи. Отсюда следует, что *единицей проекта является компонент (§23.4.3, §24.4), а не индивидуальный класс.*

Параметризация — часто приводящая к применения шаблонов — это способ явного представления неявных зависимостей, позволяющая представить альтернативы без введения новых концепций. Часто имеется возможность выбора между тем, чтобы оставить что-то зависеть от контекста, или представить в виде ветви дерева наследования, или воспользоваться параметром (§24.4.1).

23.4.3.4. Этап 4: определение интерфейсов

Определите интерфейсы. Закрытые (private) функции обычно не рассматриваются на этапе проектирования. Вопросы реализации на этапе проектирования по большей части связаны с зависимостями, возникшими на этапе 3. Кроме того, я вывел эмпирическое правило, гласящее: «если для класса не возможны по крайней мере две реализации, то с этим классом что-то не так». Это, скорее всего, не представление концепции, а замаскированная определенная реализация. Во многих случаях, рассмотрение возможности медленной эволюции класса есть приближение к ответу на вопрос «достаточно ли независим интерфейс класса от реализации?».

Открытые базовые классы, а также друзья класса являются частью интерфейса; см. также §11.5 и §24.4.2. Обеспечение отдельных интерфейсов для наследования и для обычных клиентов путем предоставления защищенных и открытых интерфейсов является упражнением, заслуживающим поощрения.

На данном шаге выявляются и специфицируются точные типы аргументов. В идеале нужно стремиться к тому, чтобы максимально полно статически типизировать интерфейсы в рамках типов прикладной задачи; см. §24.2.3 и §24.4.2.

В процессе определения интерфейсов выявляйте классы, чьи операции, как кажется, поддерживают несколько уровней абстракции. Например, некоторые функции-члены класса *File* могут принимать аргументы типа *File_descriptor*, в то время как другие — строковые описания имен файлов. Операции, принимающие *File_descriptor*, и операции, принимающие имена файлов, работают на разных уровнях абстракции, так что может возникнуть вопрос, относятся ли они к одному

и тому же классу. Может лучше организовать два класса, каждый из которых поддерживает либо файловый дескриптор, либо имена файлов. В типичном случае, все операции класса должны поддерживать одинаковый уровень абстракции. Если это не так, то стоит подумать о реорганизации этого класса и родственных ему классов.

23.4.3.5. Реорганизация иерархии классов

На этапах 1 и 3 мы изучаем структуру классов и классовую иерархию с целью убедиться, что они адекватны нашим нуждам. В общем случае 100%-ой адекватности нет, и мы вынуждены заняться реорганизацией этих структур с целью их улучшения.

Наиболее общими типами реорганизации классов являются вычленение общих частей двух классов и вынос этой общей части в новый класс, или разделение старого класса на два новых класса. В обоих случаях мы приходим к трем классам: одному базовому и двум производным. Когда такую реорганизацию следует делать? Каковы признаки того, что такая реорганизация принесет пользу?

К сожалению, простых и однозначных ответов на эти вопросы нет. Ничего удивительного, ибо мы ищем ответы не на частные вопросы о мелких деталях реализации, а на вопросы об изменениях базовых концепций системы. Здесь самая нетривиальная задача заключается в том, чтобы найти общность между классами и вычленить их общую часть. Точного критерия общности не существует, но этот критерий в любом случае должен затрагивать концепции, например, общность моделей использования, сходство наборов операций, сходство реализаций и даже то обстоятельство, что оба класса одновременно рассматриваются при обсуждении проекта. И наоборот, класс может стать кандидатом для расщепления на два класса, если разные подмножества операций этого класса имеют разные модели использования, если они имеют доступ к разным частям представления данных и даже если по поводу класса возникают разные дискуссии. Заметим также, что иногда превращение схожих классов в шаблон является систематизированным способом предоставления альтернатив (§24.4.1).

Часто проблемы с организацией классовых иерархий возникают из-за плохого именования классов и вытекающих отсюда трудностей в использовании этих имен в процессе обсуждения проекта. Действительно, если при обсуждении проекта ощущается, что имена из иерархии классов звучат нелепо, имеется необходимость и потенциальная возможность организовать иерархию по-другому. Заметьте, что тем самым я хочу сказать, что два человека при обсуждении и анализе иерархии классов намного лучше, чем один. В последнем же случае нужно попытаться составить письменное руководство по проекту с явным выписыванием всех имен классов иерархии.

Одна из самых важных задач проектирования — предоставить интерфейсы, которые могут оставаться стабильными при изменении самих классов (§23.4.2). Часто для этого нужно класс, от которого зависят множество других классов и функций, делать абстрактным классом, предоставляющим самые общие операции. Детали лучше оставить более специализированным производным классам, от которых зависят существенно меньше классов и функций. Чем больше классов зависят от данного класса, тем более общим он должен быть и тем меньше деталей он должен раскрывать.

Существует сильный соблазн — добавлять и добавлять все новые операции (и данные) в класс, используемый многими. Часто это выглядит, как попытка сделать класс более полезным и менее подверженным необходимости в будущих модификациях. Результатом такого образа мыслей является класс с так называемым жирным интерфейсом (§24.4.3) и с полями данных, поддерживающими слабо связанные между собой функции. Это опять-таки подразумевает, что при значительных изменениях в зависимых классах, этот класс также придется переделывать. А это порождает необходимость изменений и в других пользовательских и производных классах. Вместо усложнения основного класса проекта лучше делать его как можно более общим и абстрактным. Особые возможности при необходимости могут быть предоставлены производными классами. Примеры см. в [Martin, 1995].

При этом порождается иерархия абстрактных классов, в которой классы, более близкие к вершине иерархии, являются более общими, и от которых зависят большинство других классов и функций системы. Классы-листья представляют собой самые специализированные классы, и от них непосредственно зависит лишь малая толика кода. В качестве примера рассмотрите окончательную версию иерархии *Ival_box* (§12.4.3, §12.4.4).

23.4.3.6. Использование модельных образцов

Собираясь написать статью, я обычно стараюсь подыскать подходящую модель, которой можно было бы следовать. То есть вместо того, чтобы сразу набирать текст, я просматриваю предыдущие статьи на эту тему и прочие публикации, чтобы выбрать из них что-нибудь, что можно взять за начальный образец будущей статьи. Если выбранным образцом (моделью) является моя собственная статья на схожую тему, я могу даже оставить в ней многое в неизменном виде, что-то подправить и добавить лишь что-то новое только там, где этого требует логика новой статьи. Например, данная книга написана на основе первого и второго (предыдущих) изданий. Крайней степенью такой формы работы являются шаблоны писем — я просто заполняю имя и, может быть, добавляю пару строк, чтобы персонализировать послание. По сути дела, в таких письмах я лишь вношу необходимые отличия от стандартного модельного образца.

Такое *использование предыдущих систем* (наработок) в качестве моделей для новых разработок *есть скорее правило*, чем исключение в *любых видах творческой деятельности*. Всюду, где только возможно, проектирование и программирование также должны опираться на ранее выполненные работы. Это позволяет дизайнеру сузить «горизонты» (фронт работ) и сосредоточиться лишь на нескольких новых вопросах. Начинать новый проект с чистого листа — это может быть и вдохновляющий старт, но при попытке все точно специфицировать вы попадаете в ловушку множества альтернатив проектирования, среди которых блуждаете как пьяный. Отталкиваться от существующей модели не столь уж и ограничительно, так как не требуется, чтобы вы перед ней раболепствовали; она лишь освобождает дизайнера от необходимости думать обо всем сразу и позволяет ему рассматривать по одному аспекту системы за раз.

На самом деле, любой проект неявным образом использует модели (предыдущий опыт задействованных проектировщиков). Но явный сознательный выбор конкретной модели делает явными предпочтения, предположения, набор терминов, предоставляет начальную среду разработки и увеличивает вероятность того.

что разные проектировщики будут работать согласованно на базе одного и того же подхода.

Естественно, сам *выбор модели является ответственным проектным решением* и его следует делать только после тщательного поиска, изучения и отбора имеющихся альтернатив. Более того, во многих случаях модель подходит только в том случае, когда необходимые изменения соответствуют лишь адаптации основных идей к специфике нового приложения. Проектирование программ — сложная работа, и нам нужна любая помощь, которую только можно получить: мы не должны отвергать применение моделей из-за неуместного презрения к «подражательству». Подражание (имитация) есть наиболее искренняя форма лести, и применение моделей и результатов ранее выполненных работ для вдохновения (конечно, без нарушения авторских прав и прав собственности) является допустимым приемом творческой работы во всех областях: что было хорошо для Шекспира, то хорошо и для нас. Иногда такое применение моделей в проектировании называется *повторным использованием проектов (design reuse)*.

Документирование общих элементов множества проектов совместно с описанием решаемых ими проблем проектирования, а также условий, в которых они применимы, является отличной идеей. Задокumentированные таким образом наиболее общие и особо полезные элементы проектов называют *образцами* или *паттернами (patterns)*, и существует литература по документированию образцов и их использованию (например, [Gamma, 1994] и [Coplien, 1995]).

Проектировщику неплохо познакомиться с наиболее известными проектными образцами его предметной области. Как программист, я предпочитаю такие проектные образцы, которые сопровождаются примерами кода. Как и большинство людей, я лучше понимаю основную идею (в данном случае это проектный образец), когда я имею конкретный пример ее применения (здесь это фрагмент кода, иллюстрирующий используемый образец). Люди, интенсивно использующие проектные образцы, применяют специальную терминологию, помогающую им точнее понимать друг друга. К сожалению, это же может превратиться в закрытый язык, отсеивающий непосвященных, а ведь, как всегда, очень важно установить тесное общение между разными людьми, участвующими в проекте (§23.3) и, в первую очередь, между проектировщиками и программистами.

Каждый успешный программный проект является результатом перепроектирования некоторых работающих программных систем меньшего размера. Я не знаю исключений из этого правила. Можно привести множество примеров неудачных проектов, над которыми работали годами, потратили огромные средства, и, наконец, добились успеха, но много позже первоначальных сроков. Как правило, в таких проектах сначала строилась (непреднамеренно и неосознанно) неработающая система, которая затем трансформировалась в систему работающую, и, наконец, эта система перепроектировалась и реорганизовывалась так, что начинала удовлетворять первоначально заявленным целям. Это говорит о том, что наивно пытаться разрабатывать большие системы «с нуля», лишь следуя новейшим принципам. Чем больше и амбициознее проект, за который вы беретесь, тем важнее для него подобрать исходную модель, с которой можно спокойно начать работу. Для больших систем в качестве моделей следует выбирать нечто меньшее, но уже работающее.

23.4.4. Экспериментирование и анализ

Начиная разрабатывать крупный амбициозный проект, мы не знаем лучшего способа его построения. Часто мы даже не до конца понимаем, что именно система должна делать, поскольку частности проявляются лишь после построения системы, ее тестирования и эксплуатации. Как же нам все-таки получить информацию, необходимую для понимания того, какие проектные решения важнее всего для разрабатываемой системы, и какие последствия могут быть у проектных решений?

Нужно проводить эксперимент. Нужно сразу же анализировать проект и реализацию, как только мы получаем хоть что-то для анализа. Часто большую пользу дает обсуждение альтернативных проекта и реализации. Во всех случаях, кроме редчайших, проектирование — это коллективная деятельность, в рамках которых проекты представляются и анализируются. Часто важнейшим инструментом визуализации концепций программы является классная доска — без нее они пребывают втуне.

Похоже, что самая распространенная форма эксперимента состоит в построении *protomuna* (*prototype*), то есть уменьшенной версии всей системы или некоторой ее части. Прототипу не предъявляют жестких требований к быстродействию и ресурсам системы (они полагаются априорно достаточными). В результате в кратчайшие сроки получается работающая версия, с которой можно исследовать проект более предметно и выбирать разные реализации.

При хорошем исполнении этот подход может принести много пользы. Его также можно использовать и для оправдания небрежности. Трудность состоит в том, что цель прототипа может сместиться от исследования проектных альтернатив просто к получению «хоть какой-то работающей реализации системы как можно скорее». В результате может потеряться интерес к внутренней структуре прототипа («в конце концов, это всего лишь прототип») и возникнуть пренебрежение к проектным усилиям, когда все сведется к возне вокруг реализации прототипа. Серьезная опасность таится в том, что все может выродиться в худший образец пожирателя ресурсов и кошмар для сопровождения, при этом создавая иллюзию «почти завершенной» системы. По определению, прототип не имеет ясной внутренней структуры, эффективности и инфраструктуры сопровождения, допускающих его разумное постепенное масштабирование до уровня конечного продукта. Поэтому прототип, превращенный в конечный продукт, лишь впустую отнимает время и энергию у разработчиков. Существует соблазн для проектировщиков и менеджеров превратить прототип в конечный продукт и оставить вопросы эффективности до следующего выпуска. Столь порочная тактика перечеркивает все принципы проектирования.

Другая проблема состоит в том, что разработчики прототипа влюбляются в использованные ими ограниченные приемы программирования и забывают, что этого не может позволить себе вся группа программистов и проектировщиков, и что свобода их маленькой группы от ограничений и формальностей не выдержит столкновения с суровой реальностью жестких временных графиков и взаимозависимостей внутри большой группы разработчиков.

Тем не менее, прототипы могут принести неоценимую пользу. Рассмотрим, например, проектирование пользовательского интерфейса. В этом случае внутренняя структура тех частей системы, которые напрямую не взаимодействуют с пользователем, действительно не важна, и на первый план выступает желание как можно

быстрее прототипировать действующую систему, позволяющую наглядно отработать детали взаимодействия с пользователем. Другой пример — прототип, призванный оказать практическую помощь в исследовании и отработке внутренней структуры программы. Тут уже не важен истинный пользовательский интерфейс и его можно смело отбросить, заменив минимально достаточными рудиментарными средствами имитации взаимодействия с пользователем.

Прототипирование является разновидностью экспериментирования. Желаемым результатом прототипирования является проникновение в проблему в процессе построения прототипа, а не сам по себе прототип. Может быть, самым важным критерием хорошего прототипа является то, что он должен оставаться настолько незавершенным, чтобы было абсолютно очевидно, что это некий экспериментальный аппарат, который просто невозможно превратить в готовый продукт без серьезной переделки проекта и реализации. Оставляя прототип в таком незавершенном состоянии, мы поневоле фокусируемся на эксперименте и минимизируем опасности, связанные с превращением его в конечный продукт. Это также минимизирует искушение делать окончательный проект продукта слишком похожим на проект прототипа, забывая о присущей ему ограниченности. После использования прототип нужно выбросить.

Также следует помнить, что во многих случаях существуют методы экспериментирования, альтернативные прототипам. Ими следует пользоваться во всех случаях, когда есть такая возможность, ибо эти альтернативные методы строже прототипов и налагают меньше нагрузки на проектировщика и на ресурсы системы. В качестве примера можно упомянуть математические модели и разного вида симуляторы. На самом деле, можно представить себе практически непрерывный переход от математической модели через все более подробные имитации, через прототипы, через частичные реализации к завершенной системе.

Отсюда возникает идея о «выращивании» системы путем последовательных повторяющихся переходов от исходного проекта к последующим его стадиям посредством перепроектирования и повторного программирования. Это идеальная стратегия, но она может предъявить слишком большие требования к качеству средств проектирования и реализации. На сегодня эта стратегия применима в случае умеренных по размеру проектов, в которых кардинальные изменения структуры проекта маловероятны, а также для перепроектирования уже выпущенных программных продуктов, для которых означенная стратегия просто неизбежна.

В дополнение к экспериментированию ради понимания возможных способов проектирования актуально исследование особенностей существующего проекта и реализации ради выявления путей дальнейшего совершенствования системы. Например, это может быть исследование зависимостей между классами (§24.3), в котором не следует пренебрегать такими традиционными инструментами разработчика, как графы вызовов функций, замер производительности и т.д.

Отметим, что проект и проектные спецификации столь же подвержены ошибкам, что и реализация. И по сути дела даже больше подвержены, ибо они менее конкретны, определено менее точны, их нельзя исполнить и, как правило, они не поддержаны столь изощренными средствами, которые доступны для тестирования и анализа реализации. Увеличение уровня абстракции (формализации) языка/обозначений для изложения проекта, способствует применению специальных программных инструментов для поддержки проектирования, что может в определен-

ной степени помочь проектировщику. Но нужно следить за тем, чтобы это не привело к обеднению средств языка программирования, применяемого в реализации (§24.3.1). К тому же, формализованные обозначения сами по себе могут стать источником путаницы и ошибок. Это часто происходит тогда, когда используемый формализм в принципе плохо согласуется с природой решаемой задачи, когда строгость формализма превышает математическую подготовку и опыт проектировщиков и программистов, и когда теряется связь формального описания системы с реальной системой.

Проектированию неизбежно свойственны ошибки. Его трудно поддерживать автоматизирующими инструментами. Главное в проектировании — это опыт и непрерывная учеба на результатах. Поэтому абсолютно неправильно рассматривать разработку программного продукта как линейный процесс, начинающийся анализом и заканчивающийся финальным тестированием. Важно сфокусироваться на итеративной природе проектирования и реализации, чтобы получить максимальную отдачу от обратной связи и опыта предыдущих итераций.

23.4.5. Тестирование

Можно считать, что если программа не тестировалась, то она не работает. Идеальное проектирование и такого же качества реализация системы, в результате которых программа абсолютно корректно работает с самого первого запуска, достижимы разве что в самых тривиальных случаях. Стремиться к идеалу надо, но нужно не забывать о необходимости тестирования.

«Как тестировать?» — на этот вопрос нет общего ответа. А вот на вопрос «Когда тестировать?» общий ответ существует: как можно раньше (и как можно чаще). Стратегия тестирования должна быть частью общего проекта или хотя бы разрабатываться параллельно с ним. *Как только система может быть запущена, следует начинать ее тестирование.* Откладывать серьезное тестирование до момента, когда будет получена почти окончательная реализация — значит обрекать систему на серьезные изъяны, а сроки разработки проекта на срыв.

Если только это вообще возможно, систему нужно *проектировать с прицелом на удобство тестирования*. В частности, механизмы, облегчающие тестирование, могут встраиваться в саму систему. Часто этого не делают из опасений перегрузить систему и что средства, необходимые для тестирования, чрезмерно расширят структуры данных. Такие опасения совершенно необоснованны, ибо дополнительный тестировочный код и расширения структур данных могут быть удалены перед выпуском окончательной версии. Здесь особо удобен механизм *проверочных утверждений (assertions)* (§24.3.7.2).

Но еще более важным, чем конкретные тесты, является разработка такой структуры системы, в рамках которой можно быть уверенным в том, что ошибки будут устранены путем комбинации статических проверок, статического анализа и тестирования. Когда разрабатывается общая стратегия устойчивости к ошибкам (§14.9), стратегия тестирования может разрабатываться как дополнительный и тесно связанный с ней аспект общего проектирования.

Если на стадии проектирования вопросы тестирования были забыты, то готовьтесь к тому, что процесс тестирования затянется, сроки сдачи продукта сорвутся, а поддержка продукта усложнится. Обычно хорошим местом для начала работы над

стратегией тестирования является этап проектирования, связанный с определением интерфейсов классов и их зависимостями (§24.3, §24.4.2).

Обычно трудно сказать, каков достаточный объем тестирования. На практике недостаточное тестирование встречается гораздо чаще избыточного. Точное количество ресурсов (в том числе человеческих), которые должны быть выделены под тестирование по сравнению с проектированием и реализацией, зависят от природы задачи и от методов, которыми ее решают. Однако в качестве эмпирического правила я могу предположить, что *на тестирование нужно выделить больше ресурсов (времени, талантов и т.д.), чем на ее первоначальную реализацию*. Тестирование должно сосредотачиваться на ошибках, влекущих за собой катастрофические последствия, а также на менее серьезных проблемах, но которые встречаются чаще других.

23.4.6. Сопровождение и поддержка программ

«Сопровождение (поддержка) программ» — это фраза, вводящая в заблуждение. Слово поддержка вызывает аналогию с обслуживанием техники. Но программу не нужно смазывать, заменять в ней износившиеся части, удалять застрявшую в полостях воду, вызывающую ржавчину. Программные продукты можно абсолютно точно реплицировать и перемещать на гигантские расстояния за минимальную цену. Программы — это не техника (аппаратура, железо).

Под сопровождением программ понимается деятельность, направленная на перепроектирование и повторную реализацию; таким образом, она вполне укладывается в обычный цикл разработки программ. Когда гибкость, расширяемость и возможность переноса на иные платформы закладываются в проект с самого начала, серьезных проблем с сопровождением не возникает.

Как и в случае тестирования, рассмотрение вопросов сопровождения системы не следует откладывать на потом и их не нужно отделять от всей разработки в целом. В частности, важно сохранять некоторую преемственность в коллективе разработчиков системы. Нелегко передать вопросы сопровождения готовой системы коллективу, который не принимал участия в ее разработке и не имеет связи с разработчиками. Когда смена персонала неизбежна, нужно особо внимательно относиться к передаче знаний о структуре системы и ее особенностях (целях). Если группа поддержки окажется в ситуации, когда ей самой придется гадать о целях системы и о ее внутреннем устройстве, исходя из кода реализации, структура системы может стремительно ухудшиться под натиском локальных исправлений. Обычно тут мало помогает документация, ибо последняя в основном нацелена на освещение деталей, чем на раскрытие ключевых идей и принципов функционирования системы.

23.4.7. Эффективность

Дональд Кнут заметил, что «слишком ранняя оптимизация — корень всех зол». Некоторые люди усвоили этот тезис слишком буквально и считают злом любую заботу об эффективности. Разумеется, это не так, и вопросы эффективности нельзя упускать из виду в процессе проектирования и реализации. Но проектировщик не должен рассматривать приемы достижения микроэффективности, а должен сосредотачивать свое внимание на высокоуровневых аспектах эффективной работы системы.

Наилучшая стратегия достижения эффективности — это создание ясного и не переусложненного проекта. Только такие проекты остаются стабильными на протяжении жизни системы и служат базой для повышения ее быстродействия. Не нужно добавлять в проект все новые возможности «просто на всякий случай» (§23.4.3.2, §23.5.3) — это заканчивается удвоением или утроением первоначального размера и сопоставимым снижением быстродействия. Такие переусложненные системы трудно анализировать, так что становится нелегко понять, где можно избежать затрат, а где нельзя. В результате пропадает желание анализировать и повышать эффективность. Оптимизация должна быть результатом тщательного анализа и замеров производительности, а не результатом мелкой возни с кодом программы. Особенно в больших программах интуиция проектировщика или программиста не является надежным средством повышения производительности.

Конечно, важно избегать в программе особо неэффективных конструкций или таких конструкций, для достижения эффективной работы которых потребуется много времени и усилий. Аналогично, важно минимизировать применение плохо переносимых конструкций и средств, поскольку при этом проект завязнет на старых аппаратных средствах, снижая, тем самым, результирующую эффективность работы системы.

23.5. Отдельные аспекты управления проектами

Большинство людей делают в первую очередь то, за что их поощряют. В частности, если кто-то получает при разработке проекта определенные рычаги управления работой других людей, то лишь редкостные личности будут, рискуя карьерой, до конца отстаивать свои взгляды на то, что они считают правильным, перед лицом оппозиции, равнодушия и бюрократии. Отсюда следует, что организации должны вводить систему поощрений, соответствующую заявленным целям проектирования и программирования. Серьезного изменения в программировании можно добиться лишь в связи с серьезным изменением стиля проектирования, а все это достигается серьезными изменениями в стиле управления проектом. Интеллектуальная и организационная инерция способствуют осуществлению сугубо локальных изменений, в то время как для серьезного успеха обычно нужны глобальные изменения. Типичным примером является ситуация, когда осуществляется переход на язык объектно-ориентированного программирования, такой как C++, но при этом по-прежнему используются старые проектные стратегии, не способствующие эффективному использованию новых средств языка (§24.2). Другой пример — переход на объектно-ориентированное проектирование при сохранении старого (не объектно-ориентированного) языка программирования.

23.5.1. Повторное использование кода

Увеличение степени повторного использования готовых проектов и программного кода часто называют в качестве главной причины перехода на новый язык программирования или новую стратегию проектирования. Но на практике часто бывает наоборот — организации скорее поощряют программистов «изобретать велосипед». Например, производительность программиста замеряют по числу строк кода; будет он после этого стремиться к фрагментам малого размера, активно ис-

пользующим стандартную библиотеку, за счет снижения собственного дохода и, возможно, статуса? Менеджеру могут платить в зависимости от количества людей в его подразделении; будет он после этого стремиться использовать готовые наработки других отделов вместо того, чтобы нанять еще несколько человек в свою группу? Компания может получить финансирование пропорционально величине предполагаемых затрат; будет эта компания стремиться минимизировать свою прибыль за счет применения наиболее эффективных средств разработки, позволяющих уменьшить затраты? Конечно, непосредственно платить за повторное использование проектов и кода нелегко, но если менеджмент не найдет возможности так или иначе поощрять участников проекта за такую экономию сил и средств, никакого повторного использования не будет.

Повторное использование предыдущих разработок — это социальная проблема. Воспользоваться готовыми программными проектами можно лишь тогда, когда:

1. Они работают: чтобы годиться к повторному использованию, они должны на самом деле использоваться.
2. Они понятны: важны программная структура, комментарии, документация и руководство.
3. Они могут сосуществовать с иными программами, специально не предназначенными для совместной работы с ними.
4. Они имеют поддержку.
5. Они экономичны (затраты на их разработку и сопровождение равномерно распределены между многими пользователями).
6. Они доступны.

Можно утверждать, что нельзя твердо считать программный компонент готовым для повторного использования до того момента, как кто-либо реально воспользуется им с этой целью. Задача подгонки компонента к новой среде работы требует, как правило, определенной корректировки его операций, универсализации поведения и улучшения способности сосуществовать с другими программами. Пока все это не будет сделано хотя бы раз, даже компоненты, спроектированные и реализованные с величайшим вниманием, будут «спотыкаться об острые углы».

Мой опыт говорит о том, что условия для реального повторного использования кода возникают лишь тогда, когда кто-либо всерьез возьмется за это дело. В малых коллективах это означает, что кто-то вдруг (намеренно или случайно) начинает хранить общие библиотеки и документацию. В больших организациях для этой цели выделяется отдел, призванный собирать, группировать, документировать, популяризировать и поддерживать программные фрагменты, предназначенные для использования многими другими отделами.

Важность таких групп по «стандартным компонентам» невозможно переоценить. Если организация не имеет механизма, поощряющего кооперацию и повторное использование наработок, то этого и не будет в ее работе. Группа «по стандартным компонентам» обязана пропагандировать эти компоненты, так как одной лишь хорошей документации недостаточно. Кроме того, эта группа должна составлять руководства, из которых потенциальные пользователи могут узнать, где найти компонент и какую пользу он может принести. То есть характер работы такой группы близок характеру работы групп маркетинга и повышения квалификации персонала.

По мере возможности члены группы «по стандартным компонентам» должны работать как можно ближе к разработчикам, ибо только в этом случае они смогут понять нужды своих пользователей и осознать возможности совместной работы разных приложений со стандартными компонентами.

Успех группы «по стандартным компонентам» должен измеряться успехом их клиентов. Если же его измерять количеством средств и инструментов, в полезности которых удалось убедить коллективы разработчиков, то такие группы деградируют и становятся простыми распространителями программных продуктов и поборниками изменений ради изменений.

Не любой код может повторно использоваться, так что способность к повторному использованию не является универсальной характеристикой. Когда говорят, что компонент годится к повторному использованию в рамках заданной среды разработки, это означает, что в этом случае не потребуется много дополнительной работы. В то же время, переход к другой среде разработки может потребовать значительных усилий, так что способность к повторному использованию напоминает свойство переносимости. Важно осознать, что повторное использование — это результат проектирования, направленного на достижение этого свойства, результат усовершенствования компонент на основе опыта их применения и результат сознательных усилий по поиску компонентов, пригодных для повторного использования. Повторное использование не возникает волшебным образом из бездумного применения отдельных языковых черт или технологий кодирования. Такие средства языка C++, как классы, виртуальные функции и шаблоны помогают спроектировать систему так, что ее повторное использование становится более легким (и более вероятным), но сами по себе эти средства ничего не гарантируют.

23.5.2. Масштаб

Отдельные программисты или организации легко поддаются идее «делать все правильно». На специфическом жаргоне это звучит как «разработка в соответствии со строгими процедурами». Но при этом может пострадать здравый смысл, будучи потесненным пылким стремлением улучшить обычный ход дел. К сожалению, когда теряется здравый смысл, ничем не ограничивается тот вред, что наносится неограниченным «новаторством».

Рассмотрим стадии процесса разработки, перечисленные в §23.4, и этапы проектирования, перечисленные в §23.4.3. Относительно легко переделать перечисленные моменты в методику проектирования, где каждый этап точно определен и имеет четко обозначенные вход и выход, а также полуформальное описание этих входа и выхода. Можно разработать список проверочных мероприятий, гарантирующих обязательное применение методики, а также инструменты, помогающие контролировать соглашения по процедурам и формальной нотации. Глядя на классовые взаимосвязи (как они представлены в §24.3), можно оценить некоторые из них как хорошие, а другие — как плохие, но такие оценки неплохо подкрепить специальным инструментом, гарантирующим однородность этих оценок для всего проекта. Для завершения процедуры «укрепления проекта» можно определить стандарты документации (включая орфографические и грамматические правила и способ набора текста) и правила приведения кода к общему виду (включая спецификации применяемых средств языка, применяемых библиотек, соглашений по отступам и по именам функций, переменных, типов и т.д.).

Многое из этого может способствовать успеху проекта. Во всяком случае, безрассудно приступать без четко определенной и достаточно строгой схемы деятельности на основе перечисленных выше правил к разработке системы, которая в результате работы сотни людей в конечном итоге будет содержать порядка десяти миллионов строк кода и которая будет поддерживаться еще большим количеством людей в течении десяти и более лет.

По счастью, большинство систем не попадает в эту категорию. И тем не менее, как только принимается некоторый метод проектирования или декларируется приверженность какому-либо методу кодирования и составления документации как «единственно правильному», это может подвигнуть к требованию применять метод всюду и неукоснительно. Для небольших проектов это может породить нелепые ограничения и бессмысленные затраты. В частности, это может стимулировать излишнее бумаготворчество и заполнение отчетов в качестве оценки работы вместо выполнения более полезной работы. В этом случае настоящие программисты и проектировщики покинут проект, а останутся лишь бюрократы.

Стоит такому нелепому применению (на самом деле вполне обоснованного) метода проектирования показать на практике плохие результаты, как тут же начинают шарахаться в противоположную сторону и полностью отказываться от какой-либо формализации проектов. А это, в свою очередь, порождает хаос и неудачи, предотвращать которые и призван регулярный метод проектирования.

Реальный вопрос состоит в том, как найти оптимальную степень формализации для конкретного проекта. Не ждите простых ответов на этот вопрос. Для маленьких проектов могут работать практически любые подходы. Но что существенно хуже — практически любые подходы используются и для больших проектов — неважно сколь плохи эти подходы сами по себе и по отношению к разработчикам — если есть желание потратить на разработку уйму времени и денег.

Ключевой проблемой каждого программного проекта является забота о сохранении целостности проекта. Эта проблема нелинейно нарастает с ростом размера проекта. Только лишь индивидуальный программист или небольшой коллектив программистов может охватить взором и удерживать в памяти все детали и цели проекта. В общем же случае, приходится тратить столь много времени на подпроекты, технические детали, ежедневное администрирование и т.д., что общепроектные цели забываются или подчиняются более локальным и злободневным целям. Нужно во избежание неудач выделять одного человека или группу людей со специальной задачей по поддержке целостности проекта. И, конечно, такой группе нужно дать полномочия влиять на весь проект в целом.

Отсутствие долговременной стратегии наносит проекту больше вреда, чем какие-либо другие, более частные недостатки. Работа нескольких людей должна выражаться в формулировании общей цели, в постоянном обсуждении этой цели, в написании ключевых документов по проекту в целом, и в том, чтобы помочь всем остальным участникам проекта не забывать про эту общую цель никогда.

23.5.3. Личности

Описанный метод проектирования выводит на первое место мастерство и квалификацию проектировщиков и программистов, так что их выбор критически важен для успеха всей организации.

Руководители часто забывают, что организация в своей основе состоит из личностей. Распространено мнение, что все программисты более-менее одинаковы и вполне взаимозаменяемы. Это заблуждение может развалить организацию, вытолкнув из нее наиболее эффективных личностей и заставив остальных работать ниже их потенциальных возможностей. Личности взаимозаменяемы лишь тогда, когда им не дают применить свои таланты выше того минимума, который требует решаемая задача. Миф о взаимозаменяемости личностей негуманен и расточителен по своей сути.

Большинство методик оценки производительности труда программиста поощряют расточительность и не в состоянии учесть критически важный вклад конкретной личности. Наиболее очевидным примером является практика оценки труда в количестве строк кода, количестве страниц документации, количестве проведенных тестов и т.д. Такие цифры неплохо смотрятся на диаграммах менеджеров, но к реальности они имеют весьма отдаленное отношение. Например, если производительность измеряется числом строк кода, успешное повторное использование кода оказывает отрицательное влияние на оценку программиста. Успешное применение лучших принципов перепроектирования больших программ производит, как правило, такой же эффект.

Измерить качество работы намного сложнее, чем применить валовой принцип подсчета, но все равно крайне важно попытаться поощрить качественную работу индивидуального программиста или групп программистов, вместо того, чтобы тупо подсчитывать количество строк кода. К сожалению, насколько я знаю, никаких попыток разработки мер оценки качества не ведется. В результате, ограниченные методики измерения выполнения работы и состояния проекта лишь искажают реальное представление о разработке. Люди приспосабливаются к установленным промежуточным срокам и подгоняют под них свою работу. В результате страдают как целостность проекта, так и совокупная производительность труда. Например, если постулируется, что к такому-то сроку нужно устранить столько-то ошибок, то это, скорее всего, будет сделано за счет снижения производительности системы и повышения ее требований к аппаратуре. И наоборот, если за критерий выполнения работы будет положено ее быстроедействие (то есть измерениям будет подвергнуто исключительно быстроедействие), то результат будет достигнут за счет увеличения числа сопутствующих ошибок. Отсутствие объективных критериев измерения качества налагает высокие требования к технической квалификации менеджеров, ибо в ее отсутствие наблюдается неминуемая тенденция награждать за «бурную деятельность», а не за реальные успехи. Не забывайте, что менеджеры — тоже люди. Им нужны знания и квалификация не меньшие, чем у людей, которыми они управляют.

Как и в других областях человеческой деятельности, при разработке программ требуется видеть перспективу. Трудно оценить отдельного человека на основе его деятельности за один всего лишь год. Однако большинство работников имеют достаточно долгий срок работы, чтобы на этом базисе можно было достаточно уверенно судить об их технической зрелости и средней производительности. Отказ от учета сведений за достаточно долгие периоды рабочей деятельности, как это делается там, где людей считают взаимозаменяемыми винтиками рабочего колеса организации, оставляет менеджеров на милость бессмысленных количественных измерений.

Одно из последствий долгосрочного подхода и отказа от «школы управления взаимозаменяемыми идиотами» состоит в том, что личностям (и разработчикам, и руководителям) приходится дольше вращаться в серьезную и ответственную работу. Это отбивает охоту к легкомысленным переходам с одной работы на другую или к смене отделов ради карьерного роста. Важной целью является понижение текучести как среди технического персонала, так и среди ключевых менеджеров. Ни один руководитель не может добиться успеха без взаимопонимания с основными проектировщиками и программистами, а также без обновляемых технических знаний. Аналогично, никакая группа проектировщиков и программистов не сможет добиться успеха в долгосрочной перспективе без поддержки грамотных управленцев и без выхода за технические рамки их деятельности.

Там, где требуются новации, старший технический персонал, аналитики, проектировщики, ведущие программисты и т.д. вынуждены начать знакомиться с новыми для них вещами. Эти люди должны изучать новые технологии и отказываться от старых привычек. Это нелегко. Такие люди, как правило, внесли большой вклад в предыдущие успехи организации, опиравшиеся на старые методы, в которых они достигли большого мастерства, и они приобрели на этом неплохую репутацию. Это же относится и к руководителям.

Естественно, такие люди боятся перемен. Это приводит к переоценке сложностей, сопровождающих внедрение новых методов, и к недооценке трудностей, порождаемых упорным стремлением придерживаться старых методов работы в новых условиях. Так же естественно, что люди, ратующие за перемены, склонны недооценивать трудности перехода к новым методам, и к переоценке проблем, порождаемых старыми приемами работы. Эти группы должны общаться между собой, должны учиться говорить на одном языке и выработать в итоге оптимальную модель перехода. Иначе — организационный паралич и уход наиболее способных личностей из обеих групп. Обе группы должны помнить, что самые ярые консерваторы — это вчерашние ярые радикалы. Получив возможность приобрести новый опыт без унижения, ведущие проектировщики и программисты станут наиболее успешными и разумными сторонниками перемен. Их здравый скептицизм, знание запросов конечных пользователей и умение справляться с организационными барьерами бесценны. Ратующие за скорые и бескомпромиссные изменения должны понять, что переход к новым технологиям требует постепенного привыкания. А те, у кого нет никакого стремления к переменам, должны поискать иные сферы приложения талантов, а не вести арьергардные бои в условиях, когда необходимость перемен объективно диктуется новыми условиями достижения успеха.

23.5.4. Гибридное проектирование

Внедрение новых методов работы всегда болезненно. Потрясение для организации и работающих в ней людей могут быть значительными. В частности, резкие перемены, когда через день опытные разработчики «старой школы» превращаются в неэффективных новичков «новой школы», абсолютно неприемлемы. В то же время, серьезные цели редко когда достигаются без серьезных изменений, которые связаны с определенным риском.

Язык C++ разрабатывался таким образом, чтобы минимизировать этот риск, позволяя постепенно вовлекать новые средства. Очевидно, что наибольших выгод

от C++ можно добиться, применяя абстракцию данных, объектно-ориентированное программирование и объектно-ориентированное проектирование, но не столь очевидно, что скорейший способ добиться этого — радикально порвать с прошлым. Иногда такой резкий переход возможен. Но чаще бывает так, что желание перемен сдерживается заботой о том, как такой переход совершить управляемым образом. Учтите следующие соображения:

Проектировщикам и программистам нужно время для того, чтобы приобрести новые навыки.

- Нужно, чтобы старый код работал совместно с новым кодом.
- Старый код нужно продолжать поддерживать.
- Незавершенные старые проекты нужно завершить (в срок).
- Инструменты для новых технологий должны быть адаптированы в текущую среду разработки.

Все это естественным образом порождает гибридный стиль проектирования и программирования, хотя у разработчиков и не было такого намерения.

Легко недооценить первые два из перечисленных пунктов. Поддерживая разные парадигмы программирования, язык C++ помогает организациям совершить плавный переход к новым методам работы из-за того, что:

- Изучая язык C++, программисты могут одновременно выполнять работу.
- Язык C++ дает значительные выгоды даже в условиях недостаточного инструментального окружения.
- Фрагменты программы на C++ могут отлично сочетаться с фрагментами, написанными на C и других традиционных языках.
- Язык C++ имеет большое подмножество, совместимое с языком C.

Центральная идея заключается в том, что программист может постепенно переходить к C++ от традиционных языков, сохраняя традиционный (процедурный) стиль программирования. Затем можно будет воспользоваться абстракцией данных. И, наконец — когда язык C++ и связанные с ним инструменты будут тщательно изучены — можно переходить к объектно-ориентированному и обобщенному программированию. Отметим, что использовать хорошо разработанные библиотеки намного легче, чем спроектировать их и реализовать, поэтому даже новички могут пользоваться преимуществами абстракции данных уже на самых ранних стадиях в целом длительного перехода.

Возможность постепенного изучения языка C++, объектно-ориентированного программирования и объектно-ориентированного проектирования поддерживается возможностью смешивать код на C++ с кодом на других языках, которые не поддерживают абстракцию данных и объектно-ориентированное программирование (§24.2.1). Многие интерфейсы вполне можно оставить процедурными, так как превращение их во что-нибудь более сложное не даст мгновенных преимуществ. Часто библиотеки так и организованы, и поэтому использующий их программист может оставаться в неведении о настоящем языке реализации библиотеки. Применение библиотек, написанных на C, являлось для языка C++ первой и самой важной формой повторного использования кода.

Следующая стадия (нужная там, где требуется более изощренная техника программирования) — представить процедуры, написанные, например, на C или

Fortran, в виде интерфейсных классов C++, инкапсулирующих структуры данных и функции. Простейший пример такого повышения семантики от «структуры данных плюс функции» к абстрактным данным служит строковый класс из §11.12: там инкапсуляция C-строк и строковых функций стандартной библиотеки языка C позволила создать абстрактный строковый тип, пользоваться которым и проще, и удобнее.

Аналогично, любой пользовательский или встроенный тип можно внедрить в иерархию классов (§23.5.1). Это позволяет проектам на C++ эволюционировать в направлении абстракции данных и классовых иерархий, несмотря на то, что в них используется некоторое количество кода на процедурных языках, не поддерживающих перечисленных концепций, и даже в направлении готового продукта, который можно будет использовать в проектах на процедурных языках.

23.6. Аннотированная библиография

Поскольку настоящая глава лишь поверхностно затрагивает сложнейшие вопросы проектирования программ и управления процессом их разработки, я привожу здесь краткую аннотированную библиографию. Более полную аннотированную библиографию можно найти в [Booch, 1994].

- [Anderson, 1990] Bruce Anderson and Sanjiv Gossain: *An Iterative Design Model for Reusable Object-Oriented Software*. Proc. OOPSLA'90. Ottawa, Canada. Описание модели итеративного проектирования на конкретном примере и обсуждением опыта его применения.
- [Booch, 1994] Grady Booch: *Object-Oriented Analysis and Design with Applications*. Benjamin/Cummings. 1994. ISBN 0-8053-5340-2. Детальное рассмотрение всех аспектов проектирования вместе с графическими инструментами и практическими примерами, поддержанными кодом на C++. Превосходная книга — настоящая глава обязана ей многим. В ней содержатся более глубокие трактовки вопросов, затронутых в данной главе.
- [Booch, 1996] Grady Booch: *Object Solutions*. Benjamin/Cummings. 1996. ISBN 0-8053-0594-7. Описание процесса разработки объектно-ориентированных систем с точки зрения менеджмента. Содержит подробные примеры на C++.
- [Brooks, 1982] Frederick P. Brooks, Jr.: *The Mythical Man-Month*. Addison-Wesley. 1982. Переиздана с дополнениями в 1997. ISBN 0-201-83595-9. Эту книгу нужно перечитывать каждую пару лет — прививка от высокомерия. И хотя она немного устарела в техническом плане, это абсолютно не касается человеческого и корпоративного факторов, а также вопросов масштабирования.
- [Brooks, 1987] Frederick P. Brooks, Jr.: *No Silver Bullet*. IEEE Computer, Vol. 20, No. 4. April 1987. Суммирующее изложение подходов к разработке программ промышленного масштаба с предостережениями против веры в «серебряную пулю».

- [Coplien, 1995] James O. Coplien and Douglas C. Schmidt (editors): *Pattern Languages of Program Design*. Addison-Wesley. 1995. ISBN 0-201-60734-4.
- [DeMarco, 1987] T. DeMarco and T. Lister: *Peopleware*. Dorset House Publishing Co. 1987. Одна из немногих книг, которые фокусируются на человеческом факторе в производстве компьютерных программ. Каждый менеджер должен с ней ознакомиться. Удобна для непринужденного чтения. Антибиотик от глупости.
- [Gamma, 1994] Erich Gamma, et. al: *Design Patterns*. Addison-Wesley. 1994. ISBN 0-201-63361-2. Практический каталог технологий изготовления гибкого программного обеспечения с широкими возможностями повторного использования. Подробно рассматриваются нетривиальные примеры вместе с кодом на C++.
- [Jacobson, 1992] Ivar Jacobson et. al.: *Object-Oriented Software Engineering*. Addison-Wesley. 1992. ISBN 0-201-54435-0. Глубокое практическое описание разработки программ промышленного уровня с упором на примерах использования (use cases). К сожалению, содержит примеры на устаревшем варианте C++ от 1987 года.
- [Kerr, 1987] Ron Kerr: *A Materialistic View of the Software «Engineering» Analogy*. In SIGPLAN Notices, March 1987. Рассмотрение разных аналогий в данной и последующих главах основано на этой статье и личных беседах с ее автором.
- [Liskov, 1987] Barbara Liskov: *Data Abstraction and Hierarchy*. Proc. OOPSLA'87 (Addendum). Orlando, Florida.
Рассматривается вопрос о том, как с помощью наследования можно скомпрометировать абстракцию данных. Отметим, что стандарт C++ располагает специальными средствами, помогающими избежать подобного рода проблем.
- [Martin, 1995] Robert C Martin: *Designing Object-Oriented C++ Applications Using the Booch Method*. Prentice-Hall. 1995. ISBN 0-13-203837-4. Показывает, как можно систематическим образом выполнять переход от постановки задачи к коду на C++. Представляет альтернативные решения и принципы выбора между ними. Более практична и более конкретна по сравнению с другими книгами по проектированию. Содержит примеры программ на C++.
- [Meyer, 1988] Bertrand Meyer: *Object Oriented Software Construction*. Prentice Hall. 1988. Стр. 1-64 и 323-334 этой книги содержат хороший взгляд на объектно-ориентированное программирование и проектирование, а также множество здравых практических советов. Остальная часть книги описывает язык Eiffel.
- [Parkinson, 1957] C N. Parkinson: *Parkinson's Law and other Studies in Administration*. Houghton Mifflin. Boston. 1957. Одно из самых забавных и язвительных описаний тех проблем, которые несут с собой административные процессы.

- [Shlaer, 1988] S. Shlaer and S. J. Mellon *Object-Oriented Systems Analysis and Object Lifecycles*. Yourdon Press. ISBN 0-13-629023-X and 0-13-629940-7. Представляет взгляд на анализ, проектирование и программирование, отличающийся от представленного в нашей книге. Однако делает это с применением терминологии, похожей на нашу.
- [Snyder, 1986] Alan Snyder: *Encapsulation and Inheritance in Object-Oriented Programming Languages*. Proc. OOPSLA'86. Portland, Oregon. Возможно, это первое хорошее описание взаимосвязи инкапсуляции и наследования. Содержит также хорошее обсуждение некоторых понятий множественного наследования.
- [Wirfs-Brock, 1990] Rebecca Wirfs-Brock, Brian Wilkenson, and Lauren Wiener *Designing Object-Oriented Software*. Prentice Hall. 1990. Описывает метод антропоморфического проектирования, основанного на CRC-карточках. Текст книги имеет склонность к языку Smalltalk.

23.7. Советы

1. Поймите, чего вы хотите добиться; §23.3.
2. Не забывайте, что разработка программ — это человеческая деятельность; §23.2, §23.5.3.
3. Часто доказательство по аналогии — это ошибка (или обман); §23.4.
4. Ставьте четкие и осязаемые цели; §23.4.
5. Не пытайтесь использовать технические приемы для решения проблем, связанных с людьми и обществом; §23.4.
6. Создавая проекты и управляя людьми, старайтесь планировать на долгосрочную перспективу; §23.4.1, §23.5.3.
7. Создавать проекты полезно даже для сколь угодно малых программ; §23.2.
8. В проектировании очень важна обратная связь; §23.4.
9. Не путайте бурную деятельность с реальным прогрессом; §23.3, §23.4.
10. Не пытайтесь обобщать сверх меры, больше того, что можно протестировать и что позволяет ваш опыт; §23.4.1, §23.4.2.
11. Концепции представляйте в виде классов; §23.4.2, §23.4.3.1.
12. У систем имеются свойства, не представимые классами; §23.4.3.1.
13. Иерархические взаимосвязи понятий представляйте в виде иерархии классов; §23.4.3.1.
14. Выискивайте общность в концепциях приложения и представляйте более общие понятия в виде базовых классов; §23.4.3.1, §23.4.3.5.
15. Классификация предметной области не всегда полезна для модели наследования в приложении; §23.4.3.1.

16. Проектируйте классовые иерархии, отталкиваясь от поведения моделируемой системы и от инвариантов; §23.4.3.1, §23.4.3.5, §24.3.7.1.
17. Рассмотрите типичные примеры использования; §23.4.3.1.
18. Рассмотрите возможность использования CRC-карточек; §23.4.3.1.
19. Используйте готовые системы в качестве моделей, вдохновляющих примеров и отправных точек; §23.4.3.6.
20. Остерегайтесь излишней графической инженерии; §23.4.3.1.
21. Отбросьте прототип до того, как он станет помехой; §23.4.4.
22. Проектируйте с учетом неизбежных изменений, обращая внимание на гибкость, расширяемость, переносимость и повторное использование; §23.4.2.
23. Сфокусируйтесь на проектировании компонентов; §23.4.3.
24. Желательно, чтобы каждый интерфейс представлял концепцию на одном уровне абстракции; §23.4.3.1.
25. Проектируйте так, чтобы была определенная стабильность при неизбежности изменений; §23.4.2.
26. Повышайте стабильность проекта, делая интенсивно используемые интерфейсы минимальными, общими и абстрактными; §23.4.3.2, §23.4.3.5.
27. Придерживайтесь принципа минимализма — не добавляйте ничего «просто на всякий случай»; §23.4.3.2.
28. Всегда рассматривайте альтернативные реализации классов. Если альтернатив нет — то, возможно, класс не отражает чистой концепции; §23.4.3.4.
29. Многократно пересматривайте и улучшайте проект и реализацию; §23.4, §23.4.3.
30. Применяйте самые лучшие доступные инструменты для тестирования и анализа проблем, для проектирования и реализации; §23.3, §23.4.1, §23.4.4.
31. Экспериментируйте, анализируйте и тестируйте как можно раньше и как можно чаще; §23.4.4, §23.4.5.
32. Никогда не забывайте об эффективности; §23.4.7.
33. Сопоставляйте уровень формализации масштабу проекта; §23.5.2.
34. Обязательно кто-то должен отвечать за проект в целом; §23.5.2.
35. Документируйте, рекламируйте и поддерживайте компоненты многократно-го использования; §23.5.1.
36. Документируйте не только детали, но и общие цели и принципы; §23.4.6.
37. В составе документации пишите и руководство для новых разработчиков; §23.4.6.
38. Поощряйте повторное использование проектов, библиотек и классов; §23.5.1.

Проектирование и программирование

*Стремитесь, чтобы все было просто:
просто, как только возможно, но не проще того.*
— А. Эйнштейн

Проектирование и язык программирования — классы — наследование — проверка типов — программирование — что представляют собой классы? — иерархии классов — зависимости — агрегация — агрегация и наследование — альтернативы проектирования — отношение использования — программируемые отношения — инварианты — утверждения — инкапсуляция — компоненты — шаблоны — интерфейсы и реализации — советы.

24.1. Обзор

В этой главе рассматривается, как языки программирования вообще и язык C++ в частности могут поддержать процесс проектирования:

- §24.2 Фундаментальная роль классов, классовых иерархий, проверок типов и собственно программирования.
- §24.3 Применение классов и классовых иерархий с акцентом на взаимозависимости отдельных частей программы.
- §24.4 Понятие *компонента* (*component*), который является основной единицей проекта, и практические соображения по определению интерфейсов.

Более общие вопросы проектирования рассматриваются в главе 23, а различные приемы использования классов более подробно обсуждаются в главе 25.

24.2. Проектирование и язык программирования

Если бы мне пришлось строить мост, я бы серьезно задумался над тем, из какого материала его строить. Ясно, что проект моста в сильнейшей степени зависел бы от выбранного материала, и наоборот. Разумный проект каменного моста отличается от разумного проекта стального моста, деревянного моста и т.д. Я не сумел бы сделать разумный выбор материала без необходимого минимума знаний о материалах и их применении в строительстве. Естественно, что для проектирования деревянного моста не нужно быть искусным плотником, но чтобы осуществить выбор между деревом и сталью, нужно знать основные характеристики деревянных конструкций. Более того, хотя лично вам и не требуется быть искусным плотником при проектировании деревянного моста, вам нужно знать детальные конструкционные характеристики дерева и приемы его обработки (которыми владеют плотники).

Аналогично, чтобы выбрать язык для реализации некоторого программного продукта, надо знать несколько языков программирования, а чтобы спроектировать некоторую часть системы, вам нужно знать выбранный для реализации язык — даже если вам лично и не пришлось писать на нем ни строчки. Хороший проектировщик мостов учитывает свойства применяемого материала и отталкивается от этих свойств для достижения высокого качества проекта. Аналогично, хороший проектировщик программной системы учитывает свойства выбранного языка программирования и старается по максимуму использовать его сильные стороны, а также старается, насколько возможно, избегать приемов его использования, которые вызовут излишние трудности у тех, кто будет реализовывать проект в коде.

Кто-то может подумать, что важность выбора языка актуальна в первую очередь в тех случаях, когда проектировщик/программист — это одно и то же лицо. В этом случае программист может соблазниться не тем языком, как из-за недостатка опыта, так и по причине чрезмерного увлечения некоторыми стилями программирования. Когда же проектируют и программируют разные люди, тогда различия в их опыте, предпочтениях, культуре и знаниях почти наверняка породят ошибки, неэлегантность и неэффективность результирующей системы.

Что может дать язык программирования проектировщику? Он может предоставить средства, позволяющие непосредственно выразить ключевые концепции системы с их помощью. Это упрощает процесс программирования, позволяет легче следить за соответствием между проектом и реализацией, улучшает взаимопонимание между проектировщиками и программистами и позволяет создать более качественные инструменты поддержки, как первых, так и вторых.

Например, большинство методов проектирования концентрируется на взаимозависимостях между разными частями программы (обычно, стараясь минимизировать и точно определить эти зависимости). Язык, позволяющий явным образом выразить интерфейсы взаимодействия между разными частями программы, непосредственно поддерживает такую озабоченность проектировщиков. Он гарантирует, что нет никаких других зависимостей, кроме явно оговоренных. Из-за того, что зависимости явно представлены в коде программы, облегчается создание автоматизированных инструментов, читающих код и строящих наглядные диаграммы зависимостей. Это радикально облегчает работу проектировщиков и всех людей, нуждающихся в изучении структуры программы. Язык C++ — как раз такой язык, который

помогает уменьшить естественный зазор между проектом и программным кодом, и тем самым, уменьшить путаницу и недоразумения.

Ключевое понятие языка C++ — класс. Класс — это тип. Наряду с пространствами имен классы являются механизмом сокрытия информации. Программы могут быть представлены в терминах пользовательских типов данных и их иерархий. И встроенные типы, и типы данных, определяемые пользователем, подчиняются правилам статической проверки типов. Виртуальные функции обеспечивают механизм динамического (позднего) связывания без нарушения правил статической типизации. Шаблоны поддерживают проектирование параметризованных типов. Исключения предоставляют регулярный способ обработки ошибок времени выполнения. И все эти возможности языка C++ не требуют дополнительных затрат памяти и времени по сравнению с программами на языке C. Перечисленные средства относятся к основным средствам языка, которые проектировщик должен хорошо знать и принимать во внимание. Но кроме этого, общедоступные основные библиотеки — библиотеки матриц, интерфейсы к базам данных, библиотеки графического интерфейса пользователя и библиотеки поддержки параллелизма — тоже могут сильно влиять на проектные решения.

Страх перед новизной часто порождает недостаточное использование средств языка C++. К этому же приводит неуместный перенос опыта из других предметных областей, сред разработки и языков программирования. Ухудшить проект могут и неадекватные инструменты проектирования. Стоит упомянуть пять ошибок проектирования, которые не позволяют надлежащим образом воспользоваться преимуществами языка и принять во внимание его ограничения:

1. Отказ от классов — в результате проект приходится выражать с помощью подмножества C++, совпадающего с возможностями языка C.
2. Отказ от производных классов и виртуальных функций — в результате проект использует лишь абстракцию данных.
3. Игнорирование всех возможностей статической проверки типов — в результате программисты вынуждены моделировать динамическую проверку типов.
4. Отказ от самостоятельного этапа программирования — в результате проект приходится подгонять под жесткие рамки, позволяющие обойтись без программистов.
5. Игнорирование возможностей, альтернативных или дополнительных по отношению к классовым иерархиям.

Перечисленные ошибки характерны, соответственно, для проектировщиков, которые:

1. Имеют опыт работы с C, case-средствами и структурным проектированием.
2. Имеют опыт работы с Ada83, Visual Basic и активно используют абстракцию данных.
3. Имеют опыт работы со SmallTalk или Lisp.
4. Имеют опыт работы в нетехнических или каких-либо иных узкоспециализированных областях.
5. «Зациклились» на «чистом» объектно-ориентированном программировании.

В каждом случае следует задуматься, правильно ли выбран язык программирования, правильно ли выбран метод проектирования и не ошибся ли проектировщик в использовании инструментов проектирования.

Любое перечисленное несоответствие — это не преступление и его не надо стыдиться. Это просто несоответствие, которое приводит к неоптимальным проектам и лишним проблемам для программистов, которых лучше было бы избежать. Проблемы возникают и у проектировщиков, когда их среда разработки беднее применяемой среды программирования. По мере возможностей лучше избегать таких несоответствий.

Последующие разделы посвящены более подробному анализу перечисленных выше ошибок.

24.2.1. Отказ от классов

Рассмотрим вариант проектирования с отказом от классов. Результирующая программа на C++ будет примерно такой же, как программа на C (или на COBOL), которая получилась бы в рамках того же самого проекта. Можно сказать, что в этом случае проект системы «не зависит от языка программирования», так как программисты вынуждены ограничивать себя общим подмножеством C и COBOL. У этого подхода есть свои преимущества. Например, строгое разделение данных и кода упрощает работу с традиционными базами данных, разработанными как раз для таких программ. Поскольку используется минимальный язык программирования, от программистов требуется минимальная квалификация. Для многих приложений, например тех, что последовательно работают с базами данных, такой образ мыслей вполне оправдан, а разрабатывавшиеся десятилетиями приемы программирования по-прежнему работоспособны.

Естественно, имеются приложения, которые существенно отличаются от программ последовательной работы с записями, например, более сложные интерактивные CASE-системы. Отсутствие поддержки абстракции данных, вызванной отказом от применения классов, принесет в таких случаях чистый вред. Внутренняя сложность системы обязательно выиграет где-нибудь в такой программе, а бедность языковых средств не позволит непосредственно отразить в ней элементы проекта. В результате, программа имеет излишне большой размер, плохую проверку типов, и плохо приспособлена к поддержке автоматическими инструментами.

Временным решением проблемы может быть построение некоторых частных инструментов, поддерживающих понятия проектного метода. Эти инструменты возьмут на себя проверку программы на более высоком уровне абстракции, а также проверку типов для компенсации недостаточных возможностей (намеренно) обедненного языка реализации. В результате, такой метод проектирования становится специализированным закрытым внутрифирменным языком создания программ. Подобного рода языки программирования в большинстве случаев служат плохой заменой для широко распространенных языков программирования общего назначения, поддерживаемых подходящими инструментами проектирования.

Самая типичная причина отказа от классов — простая инерция. Традиционные языки программирования не поддерживают понятия класса, так что традиционные методики проектирования вынуждены учитывать это обстоятельство. Главной целью проектирования было разбить задачу на ряд процедур, выполняющих заданные

ограниченные действия. Такой стиль, названный в главе 2 *процедурным программированием* (*procedural programming*), в контексте разработки проектов называется *функциональной декомпозицией* (*functional decomposition*). В связи с этим задают вопрос: «А можно ли применять язык C++ в проектах, основанных на функциональной декомпозиции?». Можно конечно, но в таком случае вы неизбежно придете к тому, что язык C++ будет использоваться просто как улучшенный C, и столкнетесь с проблемами, рассмотренными выше. Это может быть приемлемо в переходный период, для уже завершенных систем, а также для тех подсистем, где классы, возможно, не принесут больших выгод (с учетом опыта вовлеченных в проект исполнителей). В более долгосрочной перспективе широкомасштабная политика отказа от применения классов, обусловленная применением функциональной декомпозиции, не совместима с эффективным применением C++ и других языков, поддерживающих абстракцию данных.

Процедурно-ориентированный и объектно-ориентированный взгляды на процесс создания программ различаются фундаментально и, как правило, порождают принципиально разные способы решения одной и той же задачи. Это утверждение верно как для этапа проектирования, так и для процесса непосредственного программирования: вы можете сфокусировать проект либо на выполняемых действиях, либо на представляемых сущностях, но не на том и другом одновременно.

Так почему же стоит предпочесть объектно-ориентированные методики разработки программ традиционным методам, основанным на функциональной декомпозиции? В первую очередь потому, что функциональная декомпозиция приводит к недостаточной абстракции данных. А отсюда уже следует, что проект будет:

- менее приспособлен к изменениям,
- менее совместим с инструментальными средствами,
- менее приспособлен к параллельной разработке,
- менее удобен для поддержки.

Проблема заключается в том, что функциональная декомпозиция вынуждает делать важные данные глобальными, так как она порождает структуру системы в виде дерева функций, из-за чего данные, с которыми работают несколько функций, должны быть глобальными по отношению к этим функциям. Это приводит к тому, что «интересные» данные всплывают все выше и выше к корню дерева по мере того, как все большее число функций нуждается в доступе к этим данным (считаем, что дерево растет от корня вниз). Совершенно аналогично, в иерархии классов с одним корнем имеется тенденция поднимать «интересные» данные и функции ближе к корневому (базовому) классу (§24.4). Однако когда проект фокусируется на спецификациях классов и инкапсуляции данных, эта проблема решается таким образом, что зависимости между разными частями программы делаются явными и хорошо отслеживаются. Еще важнее то, что при этом вообще ослабляется зависимость разных частей программы друг от друга за счет того, что сильнее локализуются обращения к данным.

Тем не менее, все зависит от конкретных обстоятельств, и некоторые задачи лучше решаются посредством набора процедур. Суть объектно-ориентированного подхода вовсе не в том, чтобы в программе не было глобальных функций и не в том, чтобы в ней вовсе не было процедурно-ориентированных частей, а в том, чтобы программа была разделена на части, каждая из которых наилучшим образом соот-

ветствует своей прикладной области. Чаще всего это достигается в процессе проектирования, ориентированного на классы, а не на глобальные функции. Применять процедурный стиль нужно осознано и обосновано, а не просто так, по умолчанию. И классы, и процедуры должны соответствовать специфике задачи, а не жестко связанному методу проектирования.

24.2.2. Отказ от производных классов и виртуальных функций

Теперь рассмотрим методы проектирования, игнорирующие наследование классов. В этом случае программы просто отказываются использовать одно из самых основных преимуществ C++, хотя и продолжают получать выгоды от C++ по сравнению с C, Pascal, Fortran, COBOL и т.д. Часто в качестве причин такого отказа — кроме инерции — выдвигаются утверждения, что «наследование — это всего лишь деталь реализации», «наследование нарушает сокрытие информации» и что «наследование мешает взаимодействию с другими программами».

Взгляд на наследование как на деталь реализации не учитывает того, что с помощью наследственных иерархий классов можно непосредственно моделировать ключевые взаимоотношения между концепциями и понятиями приложения. Такие взаимоотношения должны также явно прописываться в проекте с тем, чтобы дизайнеры (проектировщики) могли обсуждать их и анализировать.

Есть определенные резоны в том, чтобы исключить наследование из частей программы, взаимодействующих с кодом, написанным на других языках программирования. Это, однако, не является достаточным обоснованием для полного отказа от наследования в иных частях программы. Нужно всего лишь более тщательно определить и изолировать интерфейс взаимодействия с «внешним миром». Аналогично, беспокойства о нарушении сокрытия информации посредством использования наследования (§24.3.2.1) являются причиной для более тщательного проектирования виртуальных функций и защищенных членов (§15.3). Это не причина отказываться от наследования вообще.

В определенных проектах наследование действительно может и не давать существенного выигрыша. Однако для больших проектов политика «никакого наследования» приводит лишь к менее ясным и гибким системам, в которых якобы отсутствующее наследование симулируется при помощи иных языковых конструкций. В дальнейшем сами программисты вынуждены будут применять наследование для улучшения кода, несмотря на отсутствие его в проекте системы. В результате, проектная политика «никакого наследования» приведет лишь к тому, что в итоге наследование будет встречаться в программе несогласованным, лоскутным образом, а единого согласованного проекта всей системы не получится.

24.2.3. Игнорирование возможностей статической проверки типов

Приумножение или игнорирование роли статической проверки типов часто обосновывается рассуждениями вроде того, что «типы — это артефакты языка программирования», «естественнее думать об объектах, а не о типах» и «статическая проверка типов вынуждает нас слишком рано задумываться о деталях реализации». На такие «мелочи», как проверка типов, действительно можно не обращать внимание на ранних стадиях проектирования и анализа. В то же время, классы и классовые иерархии также бесполезны на стадии проектирования: они помогают спе-

цифицировать концепции, выяснить их взаимосвязи и уточнить их смысл. По мере продвижения в разработке проекта эта точность все более обретает форму все более точных спецификаций классов и их интерфейсов.

Важно понять, что точно определенные и строго типизированные интерфейсы являются фундаментальным инструментом проектирования. Язык C++ разрабатывался с учетом этого обстоятельства. Строго типизированный интерфейс гарантирует, что согласованные куски кода могут быть откомпилированы и скомпонованы так, что это позволяет им делать обоснованные предположения друг о друге. Выполнение этих предположений гарантируется системой типов. В свою очередь, это уменьшает необходимость в интенсивном тестировании, упрощая тем самым фазу интеграции проекта, над которым работает множество людей. Интеграция не стала важнейшей частью настоящей главы лишь потому, что уже накоплен большой положительный практический опыт интеграции систем со строго типизированными интерфейсами.

Рассмотрим следующую аналогию. В физическом мире мы соединяем друг с другом различные приборы, и кажется, что число стандартов на разъемы соединений бесконечно. Очевидно, что разъемы специально спроектированы так, чтобы было невозможно соединить два устройства в том случае, если они не предназначены для этого, а соединения осуществлялись лишь надлежащим образом. Например, вам не удастся подсоединить электрическую бритву к высоковольтной сети. Если бы вам это удалось, результатом была бы обуглившаяся электробритва или труп ее владельца. Много усилий потрачено на то, чтобы гарантировать невозможность соединения частей аппаратуры, не предназначенных для этой цели. Иногда вместо несовместимых разъемов в аппаратуру встраивают внутреннюю защиту, например предохранители от скачков напряжения. Более высокую степень защищенности обеспечивает сочетание несовместимости разъемов и дополнительной встроенной динамической защиты от непредвиденных отклонений в режимах использования устройств.

В мире программ аналогия почти полная. Статическая проверка типов равносильна совместимости на уровне разъемов, а динамические проверки сродни динамической защите электрических цепей. Как в физическом мире, так и в программном мире, если не сработали оба вида проверок — жди беды. В больших системах нужно использовать оба вида проверок.

Согласно рассмотренной в §23.4.3 схеме этапов проектирования информация о типах возникает на этапе 2 (возможно, после поверхностного рассмотрения на этапе 1) и становится определяющей на этапе 4.

Статически проверяемые интерфейсы являются главной гарантией совместной работы программного кода, разработанного разными группами людей. Документация на интерфейсы (включая точную спецификацию всех вовлеченных типов) служит основным средством общения между разными группами программистов. Такие интерфейсы являются одним из важнейших результатов проектирования системы, и на них фокусируется общение между проектировщиками и программистами.

Игнорирование статической проверки типов при рассмотрении интерфейсов приводит к проектам, затуманивающим структуру программы и откладывающим обнаружение ошибок до стадии выполнения программы. Например, интерфейс можно определить в терминах самоидентифицирующих себя объектов:

// Пример, предполагающий динамическую проверку типов, а не статическую:
Stack *s*; *// стек может содержать указатели на объекты любых типов*
void *f()*
{
 s.push(new Saab900);
 s.push(new Saab37B);

 S.pop()->takeoff(); *// отлично: Saab37B - это самолет*
 s.pop()->takeoff(); *// run-time error: авто не может взлететь*
}

Недостаточная точность определения интерфейса (функции **Stack::push()**) привела к необходимости динамической проверки вместо статической. Стек *s* предназначен для хранения самолетов, но в коде это явным образом не отражено, так что теперь пользователь должен сам гарантировать соблюдения этого требования.

Более точная спецификация — шаблон плюс виртуальные функции вместо негарантированной динамической проверки — позволяет выявить ошибку на стадии компиляции:

Stack<**Plane***> *s*; *// Stack может содержать только указатели на самолеты*
void *f()*
{
 s.push(new Saab900); *// error: Saab900 - не самолет*
 s.push(new Saab37B);

 S.pop()->takeoff(); *// чудесно: Saab37B - это самолет*
 s.pop()->takeoff();
}

Этот вопрос рассматривался также в §16.2.2. Разница в производительности для статической и динамической проверок может быть значительной (в 3-5 раз хуже для динамической проверки).

Не следует, однако, бросаться в другую крайность. Исключительно статическими проверками невозможно выявить все ошибки. Например, очевидно, что никакими статическими проверками невозможно защититься от аппаратных сбоев. Посмотрите также пример из §25.4.1, из которого видно, что полная статическая проверка не всегда достижима. Все же нужно стремиться к тому, чтобы большинство интерфейсов для типов уровня приложения проверялись статически; см. §24.4.2.

Иная проблема состоит в том, что идеальный с абстрактной точки зрения проект может породить проблемы из-за того, что не принимает во внимание ограниченные возможности базового средства — языка программирования, в данном случае C++. Например, функция *f()*, которой требуется выполнять операцию **turn_right()** над своим аргументом, может делать это только при условии общности типов ее аргументов:

```
class Plane
{
  // ...
  void turn_right();
}
```

```

class Car
{
    // ...
    void turn_right() ;
};

void f( $X^*$  p)           // каков должен быть тип X?
{
    p->turn_right() ;
    // ...
}

```

Некоторые языки (вроде Smalltalk или CLOS) позволяют использовать два типа взаимозаменяемым образом (если у них операции одинаковые), связывая их через общий базовый класс и откладывая разрешение имени до выполнения программы. Язык C++ намеренно выполняет это с помощью шаблонов, разрешая имена во время компиляции. Нешаблонные функции допускают разные типы аргументов, только если они неявным образом приводятся друг к другу. Таким образом, в предыдущем примере X должен быть общим базовым классом для *Plane* и для *Car* (то есть общим базовым классом для управляемых транспортных средств).

Как правило, примеры, инспирированные идеями из языков, чуждых C++, могут отражаться в нем путем явного отражения неявных допущений. Например, для классов *Plane* и *Car* в отсутствие у них общей базы можно создать такую классовую иерархию, которая позволит нам все-таки передавать функции $f(X^*)$ объекты, содержащие *Car* или *Plane* (§25.4.1), что впрочем весьма непросто. Шаблоны обычно помогают встраивать в код на C++ чуждые для этого языка идеи. Несоответствие проектных идей возможностям C++ обычно порождает неестественный и неэффективный код, который трудно понимать и поддерживать.

Несоответствие техники проектирования языку реализации можно сравнить с дословным переводом с одного естественного языка на другой. Например, английский язык с немецкой грамматикой так же неуклюж, как немецкий с английской, и оба могут оказаться непонятными для человека, владеющего лишь одним из них.

Классы в программе являются конкретными представлениями концепций проекта. Таким образом, делая менее четкой разницу между классами, мы затуманиваем ключевые концепции проекта.

24.2.4. Отказ от традиционного программирования

Программирование весьма дорого и непредсказуемо по сравнению со многими другими видами деятельности, а результирующий код не на 100% надежен. Трудоемкость ручного программирования высока — большая часть задержек серьезных проектов происходит как раз из-за неготовности кода. Почему бы тогда совсем не отказаться от программирования как вида деятельности?

Для многих руководителей избавление от наглых, недисциплинированных, ненадлежащим образом одетых, слишком высокооплачиваемых и слишком увлекающихся техническим жаргоном программистов кажется безусловным благом. Для программистов это может показаться абсурдным. Однако на самом деле имеется возможность избавиться от традиционного программирования в ряде важных прак-

тических областей, автоматически сгенерировав конечный код исходя из одних лишь высокоуровневых спецификаций. Или можно генерировать код, отталкиваясь от показанных на мониторе компьютера изображений геометрических фигур. Последнее типично для проектирования графического интерфейса пользователя, когда с помощью автоматизированных средств можно получить готовый код во много раз быстрее, чем при традиционном ручном программировании. Аналогично, схема базы данных и доступ к этим данным согласно имеющейся схеме намного проще и быстрее запрограммировать автоматически на основе формальных спецификаций, чем писать для этого код на C++ или иных языках программирования общего назначения. Конечные автоматы, которые будут меньше, быстрее и корректнее чем то, что может произвести большинство программистов, могут генерироваться из спецификаций или на основе визуального манипулирования графическими образами.

Подобные приемы хорошо работают в тех областях, где-либо имеется строгий и однозначный теоретический фундамент (математика, конечные автоматы, реляционные базы данных), либо есть готовый каркас, в который можно добавлять (встраивать) небольшие фрагменты кода (графический интерфейс пользователя, моделирование сетей, схемы баз данных). Очевидная польза от такого приема в ограниченных (чаще всего, принципиально ограниченных) областях может навести кое-кого на мысль о том, что полный отказ от традиционного программирования уже не за горами. Это не так. Фундаментальная причина состоит в том, что за полный отказ от программирования придется заплатить тем, что в относительно несложный в частных случаях язык формальных спецификаций придется внести сложность языков программирования общего назначения. И где в таком случае будет выигрыш?

Иногда также забывают, что сам стандартный каркас приложения, позволяющий избавиться от традиционного программирования в какой-либо конкретной предметной области, был спроектирован, запрограммирован и оттестирован традиционным способом. Язык C++ и описанные в нашей книге приемы проектирования и программирования широчайшим образом применяются как раз для разработки подобного рода систем.

В произвольных предметных областях попытка дизайнеров, ради борьбы со сложностью, придерживаться ограниченных высокоуровневых спецификаций приводит к тому, что обедняются используемые средства языка программирования общего назначения и получается ужасный результирующий код. Для улучшения такого кода программисты вынуждены демонстрировать героизм и отказываться от навязанных высокоуровневых моделей.

Лично я не вижу никаких признаков того, что традиционное программирование может быть успешно устранено отовсюду, кроме областей, где имеется твердый математический базис или приемлемые готовые каркасы приложений. Но даже в последнем случае эффективность разработок тормозится и сходит на нет, когда приходится выходить за первоначальные рамки и решать более общие задачи. Ясно, что вредно и полностью отрицать пользу от каркасов и кодогенераторов на основе высокоуровневых спецификаций, и приписывать этим средствам универсальную силу.

Разработка инструментов, библиотек и каркасов является одной из высших форм проектирования и программирования. Построение полезной математической

модели предметной области является одной из высших форм анализа. Наконец, самостоятельная разработка инструментов, библиотек и каркасов, которые будут доступны тысячам людей, есть еще и способ избежать зависимости от единственного инструмента, абсолютная привязанность к которому превратит любого программиста в ремесленника.

Очень важно, чтобы система формализованных спецификаций или библиотека поддержки автоматизированной разработки могли эффективно взаимодействовать с языком программирования общего назначения. Иначе получающиеся каркасы слишком уж ограничительны. Большое преимущество имеют системы, которые из высокоуровневых спецификаций генерируют код на общедоступном языке программирования общего назначения. Собственные внутренние языки программирования являются долгосрочным преимуществом лишь для поставщика такого решения. Также плохи автоматизированные решения, генерирующие слишком низкоуровневый код. В оптимальном случае нужен компромисс между удобством и простотой спецификаций высокого уровня (или графических манипуляций со зрительными образами) и выразительностью и богатством языка программирования общего назначения. Пожертвовать здесь чем-либо одним в ущерб другому, значит пожертвовать интересами пользователей ради интересов непосредственных разработчиков системы. Успех больших разработок, которые всегда являются многоуровневыми и модульными, обуславливается привлечением и гармоничным сочетанием самых разных языков, библиотек, инструментов и технологий.

24.2.5. Применение исключительно классовых иерархий наследования

Когда мы обнаруживаем, что нечто новое действительно хорошо работает, мы теряем чувство меры и применяем это уже без разбору. Другими словами, удачное решение некоторых проблем кажется удачным решением для почти всех проблем. Классовые иерархии и полиморфные операции над их объектами действительно служат удачным решением многих проблем. В то же время, не каждую концепцию задачи можно удачно отобразить некоторой частью иерархии, и не каждый программный компонент реализуем в виде иерархии классов.

Почему? Класс отражает некоторую концепцию, а классовая иерархия отражает взаимоотношение между классами. Теперь скажите, что общего между улыбкой, CD-ROM приводом, записью оперы, строчкой текста, спутником, моей медицинской картой и часами реального времени? Помещение всего этого в единую классовую иерархию, когда общность состоит лишь в том, что все это артефакты программирования (то есть просто объекты), не имеет фундаментального значения и ведет лишь к путанице (§15.4.5). Действительно, загоняя все в единую иерархию, мы создаем искусственное сходство и затушевываем реальное. На самом деле, развивать такие классовые иерархии нужно лишь тогда, когда анализ сущностей приложения выявил фундаментальную общность концепций, или когда в процессе проектирования и программирования открылась полезная общность в применяемых для реализации концепций структурах. В последнем случае нужно четко различать истинную общность (подтипизация с применением открытого наследования) и полезные упрощения реализации (с помощью закрытого наследования; §24.3.2.1).

Такой образ мыслей приводит к программе, содержащей несколько несвязанных или слабо связанных классовых иерархий, каждая из которых представляет множество тесно связанных между собой концепций. Это также приводит к понятию конкретного класса (§25.2), не входящего в иерархию, ибо помещение его в иерархию лишь испортит производительность и нарушит независимость такого класса от остальных частей системы.

Чтобы быть эффективной, важная операция класса, входящего в иерархию, должна быть виртуальной функцией. Кроме того, большинство данных такого класса должны быть защищенными, а не закрытыми. Это порождает их незащищенность от изменений в производных классах и затрудняет тестирование. Там, где с проектной точки зрения нужна большая инкапсуляция, следует применять неvirtуальные функции и закрытые данные (§24.3.2.1).

Операции, у которых аргументы не равноправны (один из операндов может трактоваться особо как «объект»), искажают логику проектов, ориентированных исключительно на классовые иерархии. Если же при этом аргументы должны трактоваться равноправным образом, операцию лучше не делать функцией-членом. Это не значит, что она должна стать абсолютно глобальной — ее можно ввести в пространство имен (§24.4).

24.3. Классы

Наиболее фундаментальной идеей объектно-ориентированного проектирования и программирования является взгляд на программу, как на модель определенных аспектов реальности. Классы в программе соответствуют фундаментальным концепциям приложения, и, в частности, фундаментальным концепциям моделируемой реальности. Объекты реальности и артефакты реализации представляются с помощью объектов этих классов.

Анализ взаимоотношений между классами и между разными частями классов — центральная часть проекта системы:

- §24.3.2 Отношения наследования
- §24.3.3 Отношения включения
- §24.3.5 Отношения использования
- §24.3.6 Программируемые отношения
- §24.3.7 Отношения внутри класса

Поскольку класс языка C++ — это тип, классы и их взаимоотношения получают значительную поддержку компилятора и в общем случае поддаются статическому анализу.

Чтобы класс можно было удобным образом вписать в проект, он не только должен представлять собой полезную концепцию, но еще он должен предоставить подходящий интерфейс. Идеальный класс имеет четко определенную минимальную зависимость от остального мира и интерфейс, который выдает минимум информации, необходимой внешнему миру (§24.4.2).

24.3.1. Что представляют собой классы

В системе могут присутствовать два вида классов:

1. Классы, непосредственно представляющие концепции задачи; это концепции, с помощью которых конечные пользователи описывают свои проблемы и их решение.
2. Классы, являющиеся артефактами реализации; это классы, с помощью которых проектировщики и программисты описывают технологии реализации системы.

Некоторые из классов, отражающих артефакты реализации, могут соответствовать и понятиям реального мира. Например, аппаратные и программные ресурсы системы являются хорошими кандидатами в классы приложения. То, что для одного человека — деталь реализации, для другого — характерное понятие конкретной прикладной области.

Хорошо спроектированная система содержит классы, которые поддерживают логически разные взгляды на систему. Например:

1. Классы, непосредственно представляющие пользовательские концепции (например, легковые машины, грузовики и т.д.).
2. Классы, обобщающие пользовательские концепции (например, транспортные средства).
3. Классы, представляющие аппаратные ресурсы (например, память).
4. Классы, представляющие программные ресурсы (например, поток вывода).
5. Классы, помогающие реализовать другие классы (списки, очереди, блокировки).
6. Встроенные типы и управляющие конструкции.

В больших и сложных системах поддерживать четкое разделение между логически разными типами классов и не смешивать уровни абстракции весьма затруднительно. Для простых систем можно ограничиться тремя уровнями абстракции:

- 1+2. Прикладной взгляд на систему (взгляд со стороны пользователя).
- 3+4. Аппаратный взгляд на систему (взгляд со стороны компьютера).
- 5+6. Программный взгляд на систему (взгляд со стороны языка программирования).

Чем больше система, тем большее число уровней абстракции требуется для ее описания, и тем труднее становится разделять эти уровни. Эти уровни абстракции имеют прямые аналогии в природе и человеческой деятельности. Например, можно считать, что дом состоит из:

1. Атомов.
2. Молекул.
3. Бревен и кирпичей.
4. Полов, стен и потолков.
5. Комнат.

Пока эти уровни абстракции поддерживаются отдельно, нет противоречий в вашем взгляде на дом. Но если смешать их произвольным образом, возникают не-

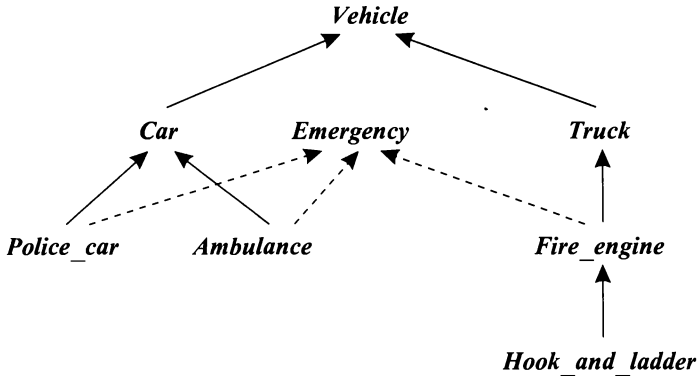
лепости. Например, высказывание «Мой дом состоит из нескольких тысяч фунтов углерода, нескольких видов полимеров, 5000 кирпичей, двух ваннных комнат и 13 потолков» звучит глупо. Из-за абстрактной природы компьютерных программ аналогичное по глупости утверждение не так легко распознается.

Переход от концепции прикладной области к классу программного проекта не является простой механической операцией. Тут нужно понимание прикладной области. Отметим, что концепции прикладной области сами являются значительными абстракциями. Например, «налогоплательщики», «монахи» и «сотрудники» не существуют в природе в чистом виде; это просто ярлыки, помогающие классифицировать людей в рамках некоторой системы отсчета (системы социальных понятий). Например, экран монитора не сильно-то напоминает поверхность рабочего стола, хотя он и разработан в виде метафоры рабочего стола, а окна на экране еще меньше напоминают окна в комнате, ветер из которых то и дело сдувает бумаги с реального рабочего стола. Смысл моделирования состоит не в том, чтобы рабски следовать за тем, что мы непосредственно видим и осязаем, а в том, чтобы отталкиваться от этого как от отправной точки в процессе создания проекта, и привязываться как к надежному якорю, когда неосозаемая природа программы грозит опрокинуть наши возможности понимать создаваемый продукт.

Начинающим обычно бывает трудно выявлять классы, но проблема эта рассасывается со временем без долговременных последствий. Затем, однако, наступает фаза, когда классы и классовые иерархии размножаются неконтролируемым образом. Это, как раз, может вызвать долговременные проблемы, усложнив программу и затруднив ее понимание и сопровождение. Не нужно каждую мелкую деталь системы обязательно представлять классом, и не всякую взаимосвязь нужно отражать в форме наследования классов. Цель проектирования — смоделировать систему с соответствующим уровнем детализации и на соответствующем уровне абстракции. Нахождение баланса между простотой и общностью — это не простое дело.

24.3.2. Иерархии классов

Рассмотрим задачу моделирования городского движения, чтобы выявить наиболее вероятное время, необходимое служебным машинам (полиция, скорая помощь, пожарные) для того, чтобы добраться до места назначения. Ясно, что нам потребуется смоделировать машины, грузовики, кареты скорой помощи, пожарные машины разных видов, полицейские машины, автобусы и т.д. Придется применить наследование, ибо перечисленные выше концепции реального мира не существуют изолированно; все они связаны друг с другом множеством связей. Без понимания этих связей нельзя адекватно воспринять концепции. Итак, в программах мы вводим классы для отражения концепций (понятий) и устанавливаем взаимосвязи между ними. Одним из наиболее общих средств представления этих связей является наследование. В нашем примере мы, возможно, захотим особо выделить машины экстренных служб, а также различать легковые машины и грузовые машины. Такой взгляд порождает следующую иерархию:



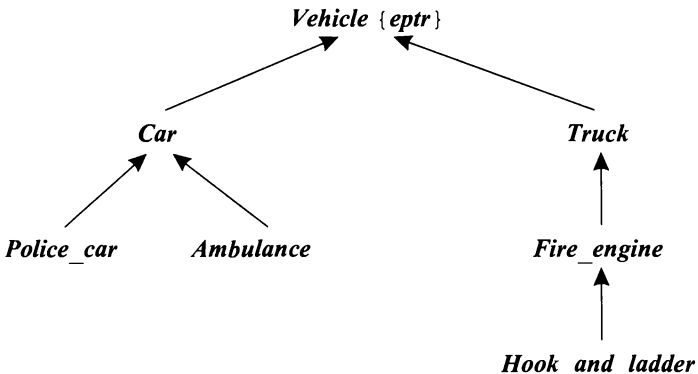
Здесь **Emergency** представляет понятие транспортного средства экстренной службы, уместное в нашей задаче моделирования: оно может нарушать обычные правила дорожного движения, имеет преимущества на перекрестках, оно находится на связи с диспетчером и т.д.

Вот соответствующая версия на C++:

```

class Vehicle { /* ... */ };
class Emergency { /* ... */ };
class Car: public Vehicle { /* ... */ };
class Truck: public Vehicle { /* ... */ };
class Police_car: public Car, protected Emergency { /* ... */ };
class Ambulance: public Car, protected Emergency { /* ... */ };
class Fire_engine: public Truck, protected Emergency { /* ... */ };
class Hook_and_ladder: public Fire_engine { /* ... */ };
  
```

Наследование — это связь наивысшего уровня, непосредственно представимая языком C++; на ранних стадиях проектирования она также является важнейшей. Часто возникает дилемма — выражать связь посредством наследования, или посредством включения (членства). Например, можно выдвинуть альтернативную концепцию транспортного средства экстренной службы — это транспортное средство, у которого имеется «мигалка». При этом классовая иерархия упрощается, так как класс **Emergency** заменяется на соответствующее поле данных в классе **Vehicle** (eptr — указатель на «экстренность»):



Теперь класс *Emergency* — это тип полей данных в классах, которым нужно действовать в качестве транспортных средств экстренных служб:

```
class Emergency { /* ... */ };
class Vehicle { protected: Emergency* eptr; /* ... */ };
class Car: public Vehicle { /* ... */ };
class Truck: public Vehicle { /* ... */ };
class Police_car: public Car { /* ... */ };
class Ambulance: public Car { /* ... */ };
class Fire_engine: public Truck { /* ... */ };
class Hook_and_ladder: public Fire_engine { /* ... */ };
```

Здесь транспортное средство относится к экстренной службе, если указатель *Vehicle::eptr* не равен нулю (соответственно, у простых автомобилей этот член инициализируется нулем). Например:

```
Car::Car() // конструктор Car (легковой автомобиль вообще)
: eptr(0)
{
}
Police_car::Police_car() // конструктор Police_car (полицейский автомобиль)
: eptr(new Emergency)
{
}
```

При этом

```
void f(Vehicle* p)
{
    delete p->eptr;
    p->eptr = 0; // теперь это не транспортное средство экстренной службы
    // ...
    p->eptr = new Emergency; // снова транспортное средство экстренной службы
}
```

Итак, какой же вариант иерархии классов лучше? Ответ на этот вопрос таков: «тот вариант лучше, что лучше соответствует концепциям реального мира». То есть выбирая между моделями в условиях достижения эффективности и простоты, нужно стремиться к большей реалистичности. В нашем конкретном случае простота преобразования экстренного транспортного средства в обычное кажется мне неестественной. Пожарные машины и кареты скорой помощи имеют специфические конструкции, укомплектованы обученным персоналом и работают в связке с диспетчером, что требует специального коммуникационного оборудования. Отсюда вытекает, что транспортное средство экстренной службы должно моделироваться как фундаментальное понятие, напрямую представимое в программе, что будет способствовать проверке типов и применению автоматизированных инструментов. Если бы мы моделировали другую задачу, где роли автомобилей менее четко определены, где обычные автомобили могли бы перевозить персонал экстренных служб, а устройства связи представлялись простыми сотовыми телефонами, там альтернативная схема моделирования была бы вполне адекватной (достаточно реалистичной).

Для тех, кому пример с моделированием уличного движения кажется надуманным или экзотичным, спешим сообщить, что альтернатива «наследование-включение (членство)» неизменно встречается при проектировании самых разных задач из разных предметных областей. В качестве еще одного примера в §24.3.3 рассматривается полоса прокрутки (scrollbar).

24.3.2.1. Зависимости внутри иерархии классов

Естественно, что производный класс зависит от своего базового класса. Не так часто отмечают и обратную зависимость. Если у класса есть виртуальная функция, значит он полагается на помощь производного класса в реализации части его функциональности. Если же какой-либо член базового класса вызывает одну из виртуальных функций, то в результате базовый класс становится зависимым от производного в своей реализации. Аналогично, если класс использует защищенные члены, то он опять же зависит от производных классов в своей реализации. Рассмотрим пример:

```
class B
{
    // ...
protected:
    int a;
public:
    virtual int f();
    int g() {int x=f(); return x-a; }
};
```

Что делает функция **g()**? Ответ самым критичным образом зависит от определения **f()** в производных классах. Вот версия, которая гарантирует возврат единицы функцией **g()**:

```
class D1: public B
{
    int f() {return a+1; }
};
```

А вот версия, в которой **g()** выведет "Hello, world!" и вернет назад 0.

```
class D2: public B
{
    int f() {cout<<"Hello, world!\n"; return a; }
};
```

Приведенный пример просто иллюстрирует формальный синтаксис виртуальных функций. Что в этом глупого? Кто-нибудь вполне может написать что-нибудь в этом роде. На самом деле, проблема здесь в том, что виртуальные функции являются частью открытого интерфейса базового класса, который предположительно может использоваться без каких-либо знаний о производных классах. Нужно описать предполагаемое поведение объектов базового класса, чтобы можно было разрабатывать программы, ничего не зная о производных классах. Каждый производный класс, замещающий виртуальную функцию, должен реализовать вариант этого поведения. Например, виртуальная функция **rotate()** класса **Shape** вращает объект этого класса

(фигуру). Функции *rotate()* из производных классов *Circle* и *Triangle* должны вращать круги (circles) и треугольники (triangles), соответственно; в противном случае нарушается фундаментальное предположение (представление) о классе *Shape*. Такого важного предположения о поведении класса *B* и производных классов *D1* и *D2* сделано не было; в результате, пример бессмыслен. Даже имена *B*, *D1*, *D2*, *f* и *g* выбраны так, что понять их предназначение невозможно. Спецификация поведения виртуальных функций — важнейший аспект классового дизайна. Выбор хороших имен для классов и их функций также очень важен (но не всегда легко).

Зависимость от неизвестного (например, еще не написанного) производного класса — это хорошо или плохо? Тут все зависит от намерений программиста. Если нужно изолировать класс от всех внешних влияний так, чтобы можно было гарантировать его специфическое поведение — то виртуальных функций и защищенных членов лучше избегать. Если же целью является построение исходного кодового каркаса, к которому позже другим (или тем же самым) программистом будет добавляться новый код, то виртуальные функции являются элегантным механизмом реализации этой идеи; защищенные члены поддерживают этот механизм. Эта технология положена в основу библиотеки потокового ввода/вывода (§21.6); она также иллюстрируется окончательной версией иерархии *Ival_box* (§12.4.2).

Если виртуальная функция предназначена лишь для косвенного использования производным классом, ее можно объявить закрытой. Рассмотрим, например, простой шаблон буфера:

```
template<class T> class Buffer
{
public:
    void put (T);           // вызывает overflow(T) при переполнении буфера
    T get ();              // вызывает underflow(), если буфер пуст
    // ...

private:
    virtual int overflow (T);
    virtual int underflow ();
    // ...
};
```

Функции *put()* и *get()* вызывают виртуальные функции *overflow()* и *underflow()*, соответственно. Пользователь может теперь реализовать разные типы буферов, заместив *overflow()* и *underflow()*:

```
template<class T> class Circular_buffer: public Buffer<T>
{
    int overflow (T);       // при заполнении возвращаемся на начало буфера
    int underflow ();
    // ...
};

template<class T> class Expanding_buffer: public Buffer<T>
{
    int overflow (T);       // при заполнении увеличиваем размер буфера
    int underflow ();
```

Если бы производному классу нужно было обращаться к *overflow()* и *underflow()* напрямую, их следовало сделать защищенными, а не закрытыми.

24.3.3. Агрегация (отношение включения)

Агрегация предполагает использование некоторым классом полей данных, связанных двумя способами с иным классовым типом *X*:

1. Член класса имеет тип *X*.
2. Член класса имеет тип *X** или *X&*.

Если значение указателя никогда не изменяется, эти варианты равносильны (за исключением эффективности и написания конструкторов и деструкторов):

```
class X
{
public:
    X(int) ;
    // ...
};

class C
{
    X a;
    X* p;
    X& r;

public:
    C(int i, int j, int k) : a(i), p(new X(j)), r(*new X(k)) {}
    ~C() { delete p; delete &r; }
};
```

В таких случаях включение самого объекта, то есть *C*: *a* в данном примере, эффективнее по быстродействию и памяти (и даже по нажатиям клавиш). Это и надежнее, ибо связь между внешним (содержащим) и внутренним (содержащимся) объектами определяется правилами конструирования и уничтожения (§10.4.1, §12.2.2, §14.4.1). См. также §24.4.2 и §25.7.

Указатель следует применять тогда, когда за время жизни «содержащего» объекта требуется изменить значение указателя на «содержащийся» объект. Например:

```
class C2
{
    X*p;

public:
    C2(int i) : p(new X(i)) {}
    ~C2() { delete p; }

    X* change(X* q)
    {
        X* t = p;
        p = q;
        return t;
    }
};
```


Естественно, что управление совместно используемым объектом требует повышенного внимания — особенно в параллельных системах.

24.3.4. Агрегация и наследование

С учетом важности отношений наследования не покажется удивительным, что ими часто злоупотребляют. Когда класс *D* открыто наследует классу *B*, говорят, что *D* есть *B* (*D is a B*):

```
class B { /* ... */ };
class D: public B { /* ... */ }; // D - это разновидность B
```

Говорят даже, что наследование есть *отношение "is-a" (is-a relationship)* или несколько точнее, что *D есть разновидность B (D is a kind of B)*. Если же класс *D* содержит член типа *B*, говорят, что *D агрегирует* (содержит — *contains*) *B*. Например:

```
class D                                // D содержит (агрегирует) B
{
public:
    B b;
    // ...
};
```

Про этот тип взаимосвязи говорят, что это *отношение типа "иметь" (has-a relationship)*.

Для заданных классов *B* и *D*, как выбрать между наследованием и агрегацией? Рассмотрим для примера классы *Airplane* (самолет) и *Engine* (двигатель). Новичкам часто приходит в голову сделать *Airplane* производным от *Engine*. Это плохая идея, поскольку самолет не является двигателем, он содержит (агрегирует, имеет) двигатель. Подумайте, может ли самолет иметь несколько двигателей? Поскольку это возможно, нам следует применить агрегирование, а не наследование. Во многих других случаях стоит задать вопрос — может ли быть несколько экземпляров одной сущности у другой сущности? Неосозаемая природа программ заставляет нас задавать такие вопросы. Если бы все классы были столь наглядными, как классы *Airplane* и *Engine*, было бы легко избежать таких тривиальных ошибок, когда *Airplane* делают производным от *Engine*. Однако в реальной практике подобные ошибки случаются весьма часто, особенно у людей, считающих наследование просто механизмом комбинирования свойств на уровне конструкций языка программирования. Несмотря на определенные преимущества наследования в виде краткости и удобства записи, его следует применять исключительно для выражения идеи общности/подтипизации, отраженной в проектных спецификациях. Рассмотрим пример:

```
class B
{
public:
    virtual void f();
    void g();
};

class D1                                // D1 содержит B
```

```

public:
    B b;
    void f() ;           // не замещение b.f()
};

void h1 (D1* pd)
{
    B* pb = pd;         // error. нет преобразования от D1* к B*
    pb = &pd->b;
    pb->g() ;            // вызывается B::g()
    pd->g() ;            // error: у D1 нет члена-функции g()
    pd->b.g() ;
    pb->f() ;            // вызывается B::f()
    pd->f() ;            // вызывается D1::f()
}

```

Заметьте, что не существует неявных преобразований от типа класса, содержащего член некоторого другого класса, к этому другому классу, и что первый из перечисленных классов не замещает виртуальных функций второго. Это резко контрастирует со случаем открытого наследования:

```

class D2: public B
{
public:
    void f() ;          // замещает B::f()
};

void h2 (D2* pd)
{
    B* pb = pd;         // ok: неявное приведение от D2* к B*
    pb->g() ;            // вызывается B::g()
    pd->g() ;            // вызывается B::g()
    pb->f() ;            // виртуальный вызов: вызывается D2::f()
    pd->f() ;            // вызывается D2::f()
}

```

Удобство и большая краткость записи в случае класса **D2** настолько превосходит таковое в случае класса **D1**, что этим легко злоупотребить. Стоит помнить, что за удобство записи приходится платить большей степенью зависимости между **B** и **D2** (см. §24.3.2.1). В частности, стоит помнить о неявном преобразовании из **D2** в **B**. Если такое преобразование не является приемлемой частью семантики ваших классов, открытого наследования стоит избегать. Когда же класс отражает некоторую сущность, а наследование от него используется в проекте для отражения зависимостей типа "is-a", то указанное неявное преобразование часто оказывается именно тем, что нужно.

Наконец, бывают случаи, когда наследование вполне подходит, но неявного преобразования нужно избежать. Рассмотрим класс **Cfield**, который помимо прочего обеспечивает динамическое (во время выполнения программы) управление доступом для другого класса **Field**. На первый взгляд, сделать **Cfield** производным от **Field** вполне разумно:

```

class Cfield: public Field { /* ... */ };

```

Таким образом, с помощью наследования выражается тот факт, что **Cfield** является разновидностью **Field**, повышается удобство записи при использовании функциями **Cfield** общих с **Field** членов, и — самое важное — позволяет классу **Cfield** замещать виртуальные функции класса **Field**. Подвох, однако, заключается в том, что неявное преобразование от **Cfield*** к **Field***, вытекающее из определения типа **Cfield**, сводит на нет все попытки контролировать доступ к **Field**:

```
void g (Cfield* p)
{
    *p = "asdf"; // доступ к Field при контроле со стороны операции присв-ния из Cfield:
                // p->Cfield::operator=("asdf")

    Field* q = p; // неявное приведение от Cfield* к Field*
    *q = "asdf"; // OOPS! нет контроля
}
```

Другое решение — сделать так, чтобы **Cfield** агрегировал (содержал) член типа **Field**, но это не позволит **Cfield** замещать виртуальные функции класса **Field**. Наилучшим решением в таком случае будет *закрытое наследование (private derivation)*:

```
class Cfield: private Field { /* ... */ };
```

С проектной точки зрения закрытое наследование эквивалентно агрегации (включению) за существенным исключением касательно замещения. Важный пример применения такого подхода — открытое наследование от абстрактного базового класса, определяющего интерфейс, и одновременно закрытое или защищенное наследование от конкретного класса с целью предоставления реализации (§2.5.4, §12.3, §25.3). Поскольку закрытое или защищенное наследование является деталью реализации, не оказывающей влияния на тип производного класса, его иногда называют *наследованием реализации (implementation inheritance)*, что контрастирует с открытым наследованием, подразумевающим наследование интерфейса и неявное преобразование в базовый тип. Это часто называют *подтипизацией (subtyping)* или *наследованием интерфейса (interface inheritance)*.

Другим способом выразить то же самое является утверждение, что «объект производного класса можно использовать там, где допустим объект базового класса». Это часто называют «*принципом подстановки Liskov*» (§23.6[Liskov,1987]) (*Liskov substitution principle*). Различие между public/protected/private формами наследованиями ярко наблюдается для полиморфных типов, объекты которых адресуются через указатели или ссылки.

24.3.4.1. Альтернатива «включение/наследование»

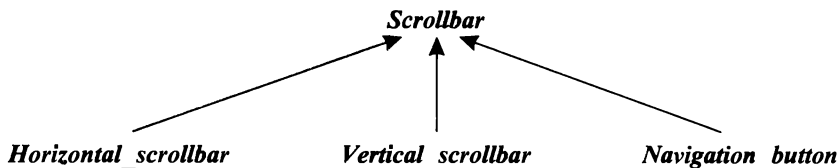
Дальнейшее изучение альтернативы «включение (агрегация)/наследование» будем проводить на примере графического интерфейса пользователя: зададимся вопросом, как представить полосу прокрутки (scrollbar) в интерактивной графической системе и как прикрепить ее к окну. Нам нужны два вида полос прокрутки — горизонтальная и вертикальная. Мы можем представить их двумя типами — **Horizontal_scrollbar** и **Vertical_scrollbar** — или одним типом **Scrollbar**, принимающим аргумент, определяющий пространственное расположение (горизонтальное или вертикальное). В первом случае нужен еще и третий тип — **Scrollbar** (полоса прокрутки вообще), выступающий в качестве базового класса для двух частных ти-

пов полос прокрутки. Во втором случае нужен дополнительный аргумент для задания типа полосы прокрутки и два его значения для идентификации этих вариантов полос. Например:

```
enum Orientation { horizontal, vertical } ;
```

Выбрав один из двух вариантов, мы сталкиваемся с необходимостью дальнейших расширений системы. Например, может обнаружиться необходимость ввести третий тип полосы прокрутки. Первоначально мы могли думать, что существует лишь две возможности для полос прокрутки («в конце концов, окно имеет лишь два измерения»). И, тем не менее, есть и другие возможности, требующие изменений в первоначальном проекте. Например, кому-то может прийти в голову использовать одну «навигационную кнопку» вместо двух полос прокрутки. Такая кнопка может осуществлять прокрутку в разных направлениях в зависимости от того, где пользователь нажал эту кнопку: если где-то вверху посередине — то прокрутка выполняется вертикально вверх; если слева — то прокрутка выполняется горизонтально влево; если в районе верхнего левого угла — то одновременная прокрутка горизонтально влево и вертикально вверх и т.д. Такие кнопки встречаются в реальной практике. Их можно рассматривать как развитие идей полос прокрутки.

Добавление навигационной кнопки в программу с иерархией трех классов полос прокрутки означает добавление нового класса, но это не влечет за собой никаких изменений в коде старых полос прокрутки:



Это очень приятный аспект иерархического решения.

Задание ориентации полосы прокрутки в качестве аргумента означает наличие полей типа в объектах полос прокрутки и наличие операторов **switch** в коде функций-членов. Таким образом, проектный выбор между двумя вариантами системы полос прокрутки означает выбор между объявлениями и лишним кодом. Первый вариант увеличивает степень статической проверки и увеличивает объем информации, доступной автоматизированным инструментам. Второй вариант откладывает выбор до момента исполнения программы и позволяет вносить изменения посредством модификации кода индивидуальных функций, не касаясь общей структуры системы, как она видится с точки зрения проверки типов и иных инструментальных средств. Лично я предпочитаю классовую иерархию, непосредственно моделирующую иерархические отношения между концепциями системы.

Решение с одним типом полосы прокрутки облегчает задачу хранения и передачи информации о типе полосы:

```
void helper (Orientation oo)
{
  // ...
  p = new Scrollbar (oo) ;
```

```
// ...
}

void me ()
{
    helper (horizontal) ;
    // ...
}
```

Такое решение может облегчить переориентацию полос прокрутки прямо во время выполнения программы. Вряд ли здесь это имеет большое значение, но во многих иных случаях это важно.

В общем, альтернатива есть, но выбор не прост.

24.3.4.2. Альтернатива «агрегация/наследование»

Теперь рассмотрим вопрос о том, как привязать полосу прокрутки к окну. Если мы будем считать класс *Window_with_scrollbar* чем-то таким, что одновременно является и окном, и полосой прокрутки, то получим нечто вроде

```
class Window_with_scrollbar: public Window, public Scrollba
{
    // ...
};
```

Это позволяет любому *Window_with_scrollbar* действовать и как *Scrollbar*, и как *Window*, но ограничивает нас одним типом полосы прокрутки.

С другой стороны, если мы будем рассматривать *Window_with_scrollbar* как окно, содержащее полосу прокрутки, то получим следующее:

```
class Window_with_scrollbar: public Window
{
    // ..
    Scrollbar* sb;

public:
    Window_with_scrollbar (Scrollbar* p, /* ... */ ) :
        Window (/* ... */ , sb (p) { /* ... */ }

    // ...
};
```

Это позволяет применить решение с иерархией полос прокрутки. Передача типа полосы в качестве аргумента позволяет окну не знать точного типа своей полосы прокрутки. Мы даже можем передать *Scrollbar* так же, как мы передавали ориентацию *Orientation* (§24.3.4.1). Если понадобится, чтобы *Window_with_scrollbar* действовал как полоса прокрутки, достаточно добавить операцию преобразования:

```
Window_with_scrollbar: : operator Scrollbar& ()
{
    return *sb;
}
```

На мой взгляд, предпочтительнее, чтобы окно содержало полосу прокрутки. Мне легче представить окно, имеющее полосу прокрутки, чем окно, которое по совместительству еще и полоса прокрутки. Моя излюбленная стратегия проектирования со-

стоит в том, что полоса прокрутки есть специальный вид окон, который содержится в окнах, нуждающихся в полосах прокрутки. Такая стратегия отдает предпочтение агрегации перед наследованием. Другой аргумент в пользу этого решения проистекает из эмпирического правила, заключающегося в вопросе «может ли окно содержать несколько полос прокрутки» (§24.3.4). Поскольку это действительно так (почему окно не может содержать, например, горизонтальную и вертикальную полосы прокрутки), класс *Window_with_scrollbar* не следует делать производным от *Scrollbar*.

Заметьте, что невозможно создать производный класс от неизвестного класса. Точный тип базового класса должен быть известен на этапе компиляции (§12.2). С другой стороны, если атрибут класса передается конструктору в качестве параметра, в классе где-нибудь должно быть представляющее его поле данных. Если тип этого поля (члена класса) — указатель или ссылка, мы можем передать объект класса, производного от типа, ассоциированного с указателем или ссылкой. В частности, поле данных *sb* типа *Scrollbar** из предыдущего примера может реально указывать на объекты полос прокрутки типа *Navigation_button*, неизвестные пользователям *Scrollbar**.

24.3.5. Отношение использования

Для проекта важно знать и понимать, как класс использует другие классы. Такие зависимости проявляются в языке C++ неявно. Класс может использовать только имена, которые были где-то объявлены. Для извлечения всех используемых имен требуется либо специальный инструмент, либо внимательное чтение файлов проекта. Способы, которыми класс *X* использует класс *Y*, можно классифицировать по-разному, например следующим образом:

- *X* использует имя *Y*.
- *X* использует *Y*.
 - *X* вызывает функцию-член *Y*.
 - *X* читает поле данных класса *Y*.
 - *X* записывает поле данных класса *Y*.
- *X* создает объект типа *Y*.
 - *X* выделяет память под *автоматические* или *статические* объекты *Y*.
 - *X* создает объект класса *Y* при помощи *new*.
- *X* принимает размеры *Y*.

Получение в качестве аргумента размеров объекта классифицируется отдельно, поскольку в этом случае требуется знакомство с объявлением класса, но нет зависимости от конструкторов. Пользование именем *Y* также классифицируется отдельно, поскольку используя *Y** или упоминая *Y* в объявлении внешней функции, мы не нуждаемся в знании объявления класса *Y* (§5.7):

```
class Y;           // Y - имя класса
Y* p;
extern Y f(const Y&);
```

Часто важно различать зависимости от интерфейса класса (объявления класса) и зависимости от реализации класса (определение классовых функций-членов).

Последние имеют еще много других зависимостей, менее интересных пользователю, чем зависимость от интерфейса класса (§24.4.2). Как правило, целью проектирования является минимизация зависимостей от интерфейса, ибо они становятся зависимостями пользователей класса (§8.2.4.1, §9.3.2, §12.4.1.1, §24.4).

Язык C++ не требует от программиста специфицировать, какие другие классы используются и каким образом. Отчасти это связано с тем, что число таких зависимостей может быть весьма велико, что при полном их описании чрезвычайно затруднит читабельность кода. И, вообще, это не вопрос языка программирования. Скорее, это вопрос проектирования системы, который может поддерживаться специальными инструментами.

24.3.6. Программируемые отношения

Язык программирования не может и не должен напрямую поддерживать любую и каждую концепцию проектирования. Подобным образом и методы, и технологии проектирования не должны поддерживать все особенности любого языка программирования. Язык проектирования обычно богаче и менее озабочен деталями, чем язык программирования. Язык программирования должен быть способен поддерживать множество идей (идиом) проектирования, или он потерпит неудачу в приспособлении к проекту.

Когда язык программирования не может напрямую поддержать некоторую идею проектирования, нужно косвенно отобразить проектную конструкцию в конструкции языка программирования. Пусть, например, в проекте введено понятие *делегирования* (*delegation*) посредством требования, чтобы все операции, не определенные в классе **A**, передавались (делегировались) для выполнения объекту класса **B**, адресуемому указателем *p*. Язык C++ не поддерживает идиому делегирования напрямую, но ее можно смоделировать в коде на C++ (то есть запрограммировать) следующим образом:

```
class B
{
    // ...
    void f();
    void g();
    void h();
};

class A
{
    B* p;                // делегируем через указатель p
    // ...
    void f();
    void ff();
    void g() {p->g();}    // делегируем g()
    void h() {p->h();}    // делегируем h()
};
```

Программисту ясно, что здесь класс **A** делегирует работу классу **B** через указатель **A**: *p*, но для языка программирования нет никакой разницы, как именно используется указатель типа **B***. Соответственно, такие «программируемые отношения» между классами плохо распознаются автоматизированными инструментами.

Взаимооднозначные соответствия между концепциями проекта и средствами языка программирования следует использовать всегда, когда только возможно. Они обеспечивают простоту и гарантируют, что проект точно отражен в коде, а это помогает в работе и программистам, и автоматизированным инструментам.

Операции преобразования обеспечивают языковой механизм для выражения программируемых отношений. Например, операция преобразования $X : \text{operator } Y()$ утверждает, что если допустим Y , то можно использовать и X (§11.4.1). Конструктор $Y : Y(X)$ выражает то же самое отношение. Отметим, что операция преобразования (и конструктор) производят новый объект, а не изменяют тип существующего. Объявляя операцию преобразования к Y , мы обеспечиваем способ неявного вызова функции, возвращающей Y . Поскольку неявное применение преобразований, программируемых с помощью соответствующих операций и конструкторов, может быть опасным, их следует тщательно и по отдельности проанализировать в проекте.

Важно также гарантировать, что графы преобразования типов не зацикливаются. В противном случае возникнут ошибки для некоторых комбинаций типов. Например:

```
class Rational;

class Big_int
{
public:
    friend Big_int operator+ (Big_int, Big_int) ;
    operator Rational() ;
    // ...
};

class Rational
{
public:
    friend Rational operator+ (Rational, Rational) ;
    operator Big_int() ;
    // ...
};
```

Типы **Rational** и **Big_int** взаимодействуют не так гладко, как кто-то надеялся:

```
void f(Rational r, Big_int i)
{
    g(r+i) ; // error, неоднозначность: operator+(r,Rational(i)) или operator+(Big_int(r),i)?
    g(r+Rational(i)) ; // одно явное разрешение
    g(Big_int(r)+i) ; // другое явное разрешение
}
```

Можно избежать таких взаимных преобразований, сделав некоторые из них явными. Например, преобразование **Big_int** в **Rational** можно определить (вместо операции) функцией **make_Rational()**, и тогда сложение разрешилось бы как $g(\text{Big_int}(r), i)$. Там, где не избежать взаимных операций преобразования, следует разрешать конфликты при помощи явных преобразований или предоставляя несколько версий таких операций, как $+$.

24.3.7. Отношения внутри класса

Класс может скрывать любое число деталей реализации и некоторое количество «грязи». Иногда ему это приходится делать. В то же время, объекты класса имеют вполне регулярную структуру и ими можно легко манипулировать. Объект класса — это набор некоторых подобъектов (часто называемых членами объекта), многие из которых являются указателями или ссылками на другие объекты. Таким образом, объект можно представить себе в виде корня дерева других объектов, в совокупности составляющих иерархию объектов, дополнительную к иерархии классов (§24.3.2.1). Например, рассмотрим очень простой класс *String*:

```
class String
{
    int sz;
    char* p;

public:
    String (const char* q) ;
    ~String () ;
    // ...
};
```

Объект класса *String* можно представить графически следующим образом:



24.3.7.1. Инварианты

Значения членов объекта и значения объектов, на которые могут указывать члены, в совокупности называются *состоянием объекта* или его *значением* (*object state* or *value*). Главной заботой при проектировании класса является необходимость привести объект в точно определенное состояние (инициализация/конструирование), поддерживать такое состояние в процессе выполнения операций с объектом и, наконец, уничтожить объект корректным и изящным образом. Свойство, делающее состояние объекта четко определенным, называется *инвариантом* (*invariant*).

Итак, целью проектирования является приведение объекта в состояние с выполняющимся инвариантом. Как правило, это делается конструктором. Любая операция над классом ожидает истинный инвариант на входе и обязана оставить инвариант в этом состоянии на выходе. В конце концов, деструктор отменяет инвариант, уничтожая объект. Например, конструктор *String::String (const char*)* гарантирует, что *p* указывает на массив по крайней мере из *sz+1* элементов, где *sz* имеет разумное значение и *p[sz] == 0*. Каждая строковая операция должна оставлять это утверждение истинным.

Искусство проектирования состоит в том, чтобы сделать класс простым и чтобы его реализация имела четко выраженный инвариант. Заявить это просто, но не так-то просто реально сформулировать полезный инвариант, который бы легко воспринимался и не налагал неприемлемых ограничений на эффективность операций класса. Под инвариантом мы будем здесь понимать фрагмент кода, который

можно выполнить для проверки состояния объекта. Можно также использовать и более строгое, математическое определение инварианта, и оно бывает уместно в некоторых случаях. Но мы будем придерживаться более простого понятия инварианта, как практической (пусть и логически неполной) проверки состояния объекта.

Понятие инварианта ведет свое начало от работ Флойда, Наура и Хоара (Floyd, Naug, Hoare) о предусловиях и постусловиях и присутствует практически во всех работах по абстрактным типам данных и верификации программ за последние 30 лет. Оно также широко применяется в процессе отладки программ, написанных на языке С.

Как правило, инвариант не поддерживается в процессе выполнения функций-членов. Функции, которые могут вызываться в этот момент, не должны быть частью открытого интерфейса класса (здесь лучше применять защищенные и закрытые функции).

Как выразить понятие инварианта в коде на C++? Простейший путь — определить функцию проверки инварианта и вставлять ее вызов в открытые операции. Например:

```
class String
{
    int sz;
    char* p;

public:
    class Range {}; // классы исключений
    class Invariant {};

    enum { TOO_LARGE = 16000 }; // предел длины
    void check (); // проверка инварианта

    String (const char* q);
    String (const String&);
    ~String ();

    char& operator [] (int i);
    int size () {return sz; }
    // ...
};

void String::check ()
{
    if (p==0 || sz<0 || TOO_LARGE<=sz || p[sz]) throw Invariant ();
}

char& String::operator [] (int i)
{
    check (); // проверка на входе
    if (i<0 || sz<=i) throw Range (); // работаем
    check (); // проверка на выходе
    return p [i];
}
```

Это хорошо работает и не представляет какой-либо сложности для программиста. В то же время, для такого простого класса, как **String**, проверки займут боль-

шую часть времени работы методов и, возможно, большую часть объема кода. Поэтому программисты часто выполняют проверку инварианта только во время отладки:

```
inline void String::check ()
{
    #ifndef NDEBUG
    if (p==0 || sz<0 || TOO_LARGE<=sz || p[sz]) throw Invariant ();
    #endif
}
```

Здесь макрос *NDEBUG* используется так же, как в стандартном макросе *assert()* языка C. Принято устанавливать *NDEBUG* в знак того, что отладка не производится.

Столь простая работа с инвариантами во время отладки оказывает бесценную помощь в получении правильно работающего кода и, что даже еще важнее, в четком отражении классами концепций проекта. Дело в том, что при определении инвариантов приходится рассматривать классы с иной точки зрения, в результате чего код получает полезную избыточность, увеличивая вероятность выявления ошибок, несовместимостей и оплошностей.

24.3.7.2. Утверждения

Инвариант — это особая форма *утверждения (assertion)*. Утверждение же в общем случае есть просто высказывание о том, что некоторое логическое условие выполняется. Вопрос здесь только один — что делать, если условие не выполняется.

Стандартная библиотека языка C — и стало быть C++ — предоставляет макрос *assert()* в заголовочном файле *<assert.h>* или *<cassert>*. Макрос *assert()* вычисляет значение аргумента и вызывает *abort()* в случае нулевого результата (*false*). Например:

```
void f(int* p)
{
    assert (p!=0);    // утверждается, что p!=0; abort() если p равно нулю
    // ...
}
```

Перед тем, как прекратить работу программы, *assert()* выводит имя своего исходного файла и номер строки, на которой это произошло. Это делает макрос *assert()* полезным отладочным средством. Обычно *NDEBUG* устанавливается с помощью опций компилятора отдельно для каждой единицы трансляции. Отсюда следует, что не стоит использовать *assert()* во встраиваемых и шаблонных функциях, поскольку их определения включаются в разные единицы трансляции, и нужно тщательно следить, чтобы *NDEBUG* был всюду установлен одинаково (§9.2.3). Как и все, что относится к магии макросов, применение *NDEBUG* следует признать весьма низкоуровневым приемом, довольно запутанным и подверженным ошибкам. Кроме того, иногда нужно оставлять диагностические средства и в хорошо работающей программе, а вызов *abort()* не подходит для готового кода промышленного уровня.

Альтернативой является применение шаблона *Assert()*, который генерирует прерывание, а не безусловное прекращение работы программы, так что его можно оста-

вить в окончательном варианте кода (если это необходимо). К сожалению, *Assert()* не входит в состав стандартной библиотеки, но он очень просто определяется:

```
template<class X, class A> inline void Assert (A assertion)
{
    if (!assertion) throw X();
}
```

Шаблон *Assert()* генерирует исключение *X()*, если условие *assertion* ложно. На-
пример:

```
class Bad_arg {};

void f(int* p)
{
    Assert<Bad_arg>(p!=0);
    // ...
}
```

Так как здесь условия утверждений формулируются явно, то можно явным образом указать необходимость проверки, например, только в условиях отладки:

```
void f2(int* p)
{
    Assert<Bad_arg>(NDEBUG || p!=0);
    // ...
}
```

Применение здесь операции `||` вместо `&&` может показаться удивительным. Тем не менее, *Assert<E>(a || b)* проверяет истинность `!(a || b)`, что эквивалентно `!a && !b`.

Указанное применение *NDEBUG* требует установки для него соответствующего значения в зависимости от того, осуществляем мы отладку, или нет. Поскольку реализации C++ не делают этого по умолчанию, лучше самим установить некоторое характерное значение. Например:

```
#ifdef NDEBUG
const bool ARG_CHECK = false;
#else
const bool ARG_CHECK = true;
#endif

void f3(int* p)
{
    Assert<Bad_arg>(!ARG_CHECK || p!=0);
    // ...
}
```

Если генерируемое в процессе проверки утверждения прерывание не перехватывается, то *Assert()* заканчивает выполнение программы вызовом *terminate()* (почти так же, как *assert()* заканчивает программу вызовом *abort()*). Обработчик прерывания может выполнить и другие, менее радикальные действия.

Обычно, при тестировании любой программы реальных (не учебных) размеров я включаю или выключаю проверочные утверждения группами. Применение *NDEBUG* — это простейший способ реализации такой техники. На ранних стадиях раз-

работки включается большинство проверочных утверждений, в то время как в окончательном коде их остается совсем немного (лишь критически важные проверки). Как видно из приведенного примера, такой стиль управления отладкой поддерживается разбиением утверждений на две части — одна часть является условием, которое управляет включением/выключением проверки (например, **ARG_CHECK**), а вторая содержит собственно проверяемое условие.

Если включающее/выключающее условие является константным выражением, то при его запрещающем значении все утверждение будет удалено компилятором. Однако включающее/выключающее условие может быть и переменной, значение которой можно динамически изменять во время выполнения программы в соответствии с нуждами отладки. Например:

```
bool string_check = true;

inline void String::check ()
{
    Assert<Invariant> ( !string_check || ( p && 0 <= sz && sz < TOO_LARGE && p [sz] == 0 ) );
}

void f ()
{
    String s = "wonder";
    // здесь строки проверяются
    string_check = false;
    // а здесь строки не проверяются
}
```

Естественно, в этом случае генерируется реальный код, так что стоит следить за тем, чтобы его проверочная часть не разбухла слишком сильно из-за интенсивного применения утверждений.

Отметим, что оператор

```
Assert<E> (a) ;
```

эквивалентен

```
if ( !a ) throw E () ;
```

В связи с этим возникает вопрос, зачем связываться с **Assert** (), а не просто написать этот оператор? Дело в том, что применение **Assert** () четко отражает намерение проектировщика. Оно явным образом говорит, что некоторое условие предполагается в этом месте программы истинным. Это не обычный программный код, а ценная дополнительная информация для любого человека, читающего программу. Еще одним (более приземленным) преимуществом является большая простота обнаружения **assert** () или **Assert** () в большом коде, нежели нахождение в нем условных выражений, генерирующих исключения.

Шаблон **Assert** () может быть обобщен так, чтобы он генерировал переменные исключения и исключения с аргументами:

```
template<class A, class E> inline void Assert (A assertion, E except)
{
    if ( !assertion ) throw except ;
}
```

```

struct Bad_g_arg
{
    int* p;
    Bad_g_arg(int* pp) : p(pp) {}
};

bool g_check = true;
int g_max = 100;

void g(int* p, exception e)
{
    Assert(!g_check || p!=0, e);
    Assert(!g_check || (0<*p&&*p<=g_max), Bad_g_arg(p));
    // ...
}

```

К сожалению, многие компиляторы не сумеют устранить избыточный код для обобщенного `Assert()` в случаях, когда условие вычисляется на этапе компиляции. Поэтому двухаргументные `Assert()` следует применять лишь в случае, когда исключение не представимо в форме `E()`, или когда код нужно генерировать независимо от значения утверждения.

В §23.4.3.5 рассматривались две наиболее распространенные формы реорганизации классовых иерархий — расщепление классов на два класса и выделение общей части двух классов в общий базовый класс. В обоих случаях большую помощь могут оказать грамотно спроектированные инварианты. В классе, созревшем для расщепления, хорошо видна избыточность большей части проверок инварианта, сосредоточенных в его операциях; после расщепления подмножества операций будут иметь дело с соответствующими подмножествами состояний объекта. В классах с общими частями будут просматриваться схожие инварианты, несмотря на различие в деталях реализации.

24.3.7.3. Предусловия и постусловия

Популярным применением утверждений является формулировка предусловий и постусловий для функций. Имеется ввиду проверка основных предположений относительно входа в функцию и проверка того, что при выходе функция оставляет мир в надлежащем состоянии. К сожалению, часто подобные утверждения формулируются на более высоком уровне, так что язык программирования не всегда в состоянии отразить их удобным образом. Например:

```

template<class Ran> void sort(Ran first, Ran last)
{
    Assert<Bad_sequence> (" [first, last) is a valid sequence" ); // псевдокод
    // ... сортирующий алгоритм ...
    Assert<Failed_sort> (" [first, last) is in increasing order" ); // псевдокод
}

```

Это фундаментальная проблема. Общие высказывания о программе лучше формулируются на высокоуровневом языке, близком к математическому, а не на алгоритмическом языке, на котором пишутся программы.

В связи с инвариантами требуется немалая доля интеллекта, чтобы перевести высказываемое нами идеализированное утверждение в нечто такое, что можно проверить алгоритмически. Например:

```

template<class Ran> void sort (Ran first, Ran last)
{
    // [first,last) - допустимая последовательность:
    Assert<Bad_sequence> (NDEBUG || first<=last) ;

    // ... сортирующий алгоритм ...

    // [first,last) находится в возрастающем порядке:
    Assert<Failed_sort> ( NDEBUG ||
        (last-first<2 || (*first<=last[-1] &&
            *first<=first[ (last-first) / 2] &&
            first[ (last-first) / 2] <=last[-1] ) ) ) ;
}

```

Несмотря на то, что писать обычные проверки аргументов и результатов вычислений намного проще, чем утверждения, я все же рекомендую обязательно попытаться выразить идеальные предусловия и постусловия (хотя бы в виде комментариев), прежде чем свести их к чему-либо менее абстрактному, что можно легко выразить на языке программирования.

Проверка предусловий может легко выродиться в тривиальную проверку значений аргументов вызова. Поскольку аргументы часто передаются многим функциям, такая проверка становится дорогой. Кроме того, проверка, например, всех аргументов-указателей на неравенство нулю может вызвать необоснованное успокоение и дать ложное чувство уверенности, особенно если оно выполняется лишь в отладочных вариантах программы. Вот почему я рекомендую сосредоточить внимание на инвариантах.

24.3.7.4. Инкапсуляция

Заметьте, что в языке C++ именно класс, а не объект класса, является единицей инкапсуляции. Например:

```

class List
{
    List* next;
public:
    bool on (List* ) ;
    // ...
};

bool List::on (List* p)
{
    if (p == 0) return false;
    for (List* q = this; q; q=q->next) if (p==q) return true;
    return false;
}

```

Доступ к закрытому указателю **List::next** разрешен, ибо функция **List::on()** имеет доступ к любым объектам класса **List**, так или иначе попадающим в зону ее действия. Там, где это неудобно, можно упростить ситуацию и без эксплуатации возможностей функций-членов иметь открытый доступ к объектам того же самого класса. Например:

```

bool List::on (List* p)
{
    if (p == 0) return false;

```

```
if (p == this) return true;  
if (next == 0) return false;  
return next->on (p) ;  
}
```

Здесь, однако, итерация превращается в рекурсию, а это может нанести удар по быстродействию в случае, если компилятор не способен на оптимизацию, превращающую рекурсию обратно в итерацию.

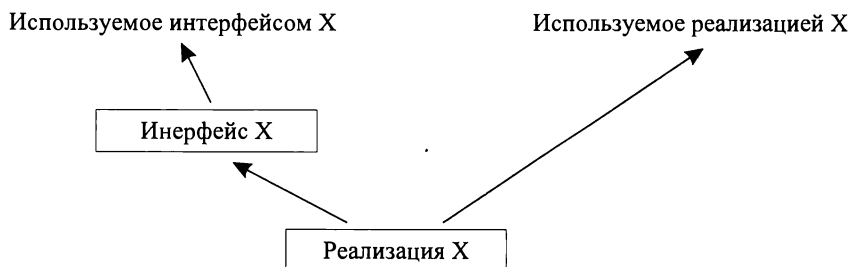
24.4. Компоненты

Единицей проектирования является набор классов, функций и т.д., а не отдельный класс. Это множество часто также называют библиотекой или каркасом (§25.8), и оно является предметом повторного использования (§23.5.1), сопровождения и т.д. Язык C++ предоставляет три возможности для выражения совокупности средств, объединенных неким единым логическим критерием:

1. Класс — коллекция данных, функций, шаблонов и типизированных членов.
2. Иерархия классов — коллекция классов.
3. Пространство имен — коллекция данных, функций, шаблонов и типизированных членов.

Класс обеспечивает средства, делающие удобным создание объектов типа, представляемого этим классом. В то же время, многие важные компоненты не могут быть описаны механизмом, удобным для создания объектов единственного типа. Иерархия классов предназначена для отражения понятия множества родственных классов. Однако составные части компонентов не всегда представимы именно классами, и не все классы можно вписать в осмысленные иерархии (§24.2.5). Поэтому самым непосредственным и универсальным воплощением понятия компонента в языке C++ является пространство имен. Еще раз отметим, что не все элементы компонента являются классами.

В идеале компонент должен описываться набором интерфейсов, которые он использует для собственной реализации, и набором интерфейсов, которые он предоставляет своим пользователям. Все прочее — это внутренние детали реализации, которые должны скрываться от «внешнего мира». Набор интерфейсов — это термины, в которых компонент описывает его проектировщик. Программист же вынужден отображать интерфейсы в объявления. Классы и иерархии классов определяют интерфейсы, а пространства имен позволяют сгруппировать их в наборы, которые компонент использует сам и которые он предоставляет пользователям:



При использовании техники, описанной в §8.2.4.1, это превращается в следующее:

```
namespace A
{
    // ...
}

namespace X                // интерфейс компонента X
{
    using namespace A;      // зависит от объявлений из A
    void f() ;
}

namespace X_impl           // средства, нужные реализации X
{
    using namespace X;
    // ...
}

void X: :f()
{
    using namespace X_impl; // зависит от объявлений из X_impl
    // ...
}
```

Основной интерфейс *X* компонента не должен зависеть от интерфейса реализации *X_impl*.

Компонент может содержать множество классов, не предназначенных для общего использования. Такие классы следует прятать внутри классов реализации или пространств имен:

```
namespace X_impl
{
    class Widget
    {
        // ...
    };
    // ...
}
```

Это гарантирует, что *Widget* не может использоваться из других частей программы. Однако классы, которые отражают общепользные понятия, являются хорошими кандидатами для многократного повторного использования, и их желательно включать в интерфейс компонента:

```
class Car
{
    class Wheel
    {
        // ...
    };

    Wheel flw, frw, rlw, rrw;
    ..
```

```
public :
```

```
    // ...
```

```
};
```

Во многих случаях колеса лучше спрятать ради чистоты абстрактной концепции автомобиля (нельзя использовать колеса в отрыве от главного понятия — автомобиля). Однако класс колес *Wheel* сам по себе кажется вполне пригодным для самостоятельного использования, так что возможно его все же лучше вынести из класса *Car*:

```
class Wheel
```

```
{
```

```
    // ...
```

```
};
```

```
class Car
```

```
{
```

```
    Wheel flw, frw, rlw, rrw;
```

```
    // ...
```

```
public :
```

```
    // ...
```

```
};
```

Решение оставить класс внутри или вынести наружу (оба способа широко применяются на практике) зависит от конкретных целей проекта и совокупности понятий, составляющих проект. По умолчанию класс локализуют как только могут до тех пор, пока не появляются серьезные причины сделать его более доступным.

У интересных функций и данных есть отвратительная тенденция всплывать в глобальное пространство имен, в широко используемые пространства имен или корневой класс иерархии. Это может ненароком приоткрыть детали реализации и привести к проблемам, характерным для глобальных функций и данных. Вероятность этого увеличивается в случае однокоренных иерархий и для проектов с малым числом пространств имен. В контексте классовых иерархий с этим можно бороться с помощью виртуальных базовых классов (§15.2.4). Проблема же малого числа пространств имен решается введением множества небольших реализационных пространств имен.

Заметьте, что заголовочные файлы обеспечивают мощный механизм предоставления разных видов на один и тот же компонент для разных групп пользователей, а также для устранения пользовательской информации о классах реализации (§9.3.2).

24.4.1. Шаблоны

С проектной точки зрения шаблоны служат двум, слабо связанным между собой целям:

- Обобщенному программированию.
- Политике параметризации.

На ранних стадиях проектирования операции — это просто операции. Но на более поздних стадиях, когда появляется необходимость описать типы их операндов, проявляется особая важность шаблонов для таких статически типизированных язы-

ков программирования, как C++. Без шаблонов стали бы повторяться определения функций, а иначе проверка типов была бы отнесена на стадию выполнения программ (§24.2.3). Наилучшим кандидатом на роль шаблонов являются операции, реализующие универсальные алгоритмы, пригодные для разных типов. Если все такие операнды относятся к одной иерархии классов, и особенно если нужно добавлять типы операндов на стадии выполнения программы, тогда тип операнда лучше всего представить классом (часто абстрактным классом). Если же типы операндов не вписываются в единственную иерархию классов и если особо важна эффективность, операцию лучше всего реализовать в виде шаблона. Стандартные контейнеры и поддерживающие их алгоритмы иллюстрируют ситуацию, когда несвязанные между собой типы операндов и требование высокой эффективности вынуждают использовать шаблоны (§16.2).

Чтобы предметнее проиллюстрировать альтернативу шаблоны/иерархия рассмотрим, как можно обобщить простую итерацию:

```
void print_all (Iter_for_T x)
{
    for (T* p = x.first(); p; p = x.next()) cout << *p;
}
```

Здесь предполагается, что *Iter_for_T* обеспечивает операции, возвращающие *T**.

Мы можем сделать итератор *Iter_for_T* параметром шаблона:

```
template<class Iter_for_T> void print_all (Iter_for_T x)
{
    for (T* p = x.first(); p; p = x.next()) cout << *p;
}
```

Это позволяет нам использовать самые разные итераторы до тех пор, пока они предоставляют операции *first()* и *next()* надлежащего смысла, и если во время компиляции мы знаем типы итераторов для каждого вызова *print_all()*. Стандартные контейнеры и алгоритмы базируются на этой идее.

С другой стороны, мы можем трактовать *first()* и *next()* как интерфейс к итераторам, и определить класс, представляющий этот интерфейс:

```
class Iter
{
public:
    virtual T* first() const = 0;
    virtual T* next() = 0;
};

void print_all2 (Iter& x)
{
    for (T* p = x.first(); p; p = x.next()) cout << *p;
}
```

Мы можем пользоваться любыми итераторами, производными от *Iter*. Как видно, реальный клиентский код не зависит от того, используем ли мы для аргументов параметры шаблона или классовую иерархию — различаются лишь эффективность и нюансы, связанные с перекомпиляцией. В частности, вот конкретный пример использования класса *Iter* в качестве аргумента шаблона:

```
void f(Iter& i)
{
    print_all(i);           // используется шаблон
    print_all2(i);
}
```

Следовательно, два рассматриваемых подхода могут и дополнять друг друга.

Часто шаблону требуются функции и классы в качестве части своей реализации. Многие из них для сохранения универсальности и эффективности сами должны быть шаблонами. В этом смысле алгоритмы становятся обобщенными по множеству типов. Такой стиль применения шаблонов называется *обобщенным программированием* (*generic programming*) (§2.7). Когда мы вызываем `std::sort()` для контейнера `vector`, элементы вектора становятся операндами операции `sort()`; можно сказать, что алгоритм `sort()` является обобщенным по типу элементов вектора. Кроме того, этот алгоритм является обобщенным и по типам контейнеров, поскольку задействуются лишь итераторы произвольных стандартных контейнеров (§16.3.1).

Еще алгоритм `sort()` параметризован по критерию сравнения (§18.7.1). С точки зрения проектировщика это отличается от простой параметризации операции по типам ее аргументов. Решение параметризовать алгоритм таким образом, чтобы влиять на особенности работы алгоритма, является проектным решением более высокого уровня. Тем самым проектировщик/программист получают дополнительный канал влияния на определение политики выполнения некоторых действий в алгоритме. Однако с точки зрения языка программирования никаких особенностей тут нет.

24.4.2. Интерфейсы и реализации

Идеальный интерфейс:

- предоставляет пользователю полный и согласованный набор концепций,
- согласован для всех частей компонента,
- не открывает деталей реализации пользователям,
- может реализовываться разными способами,
- статически типизирован,
- выражается с помощью типов уровня приложения,
- зависит ограниченно и строго определенным образом от других интерфейсов.

Отметив необходимость согласованности классов, представляющих интерфейс компонента остальному миру (§24.4), мы можем теперь упростить изложение, ограничившись единственным классом:

```
class Y { /* . . . */ };           // нужен для X
class Z { /* . . . */ };           // нужен для X

class X                             // пример плохого стиля интерфейса
{
    Y a;
    Z b;

public:
    void f(const char* . . . );
```

```
void g (int [ ] , int ) ;
void set_a (Y&) ;
Y& get_a () ;
};
```

У этого интерфейса есть несколько потенциальных проблем:

- Он использует типы *X* и *Y* таким образом, что для компиляции требуется знать их объявления.
- Функция *X::f()* принимает произвольное число аргументов неизвестного типа (возможно, под управлением «форматирующей строки», передаваемой в качестве первого аргумента; §21.8).
- Функция *X::g()* принимает аргумент типа *int []*. Это допустимо, но означает низкий уровень абстракции. Массив целых чисел не описывает себя сам, и неясно, сколько элементов он содержит.
- Функции *set_a()* и *get_a()* с большой вероятностью открывают детали представления класса *X*, разрешая прямой доступ к *X::a*.

Эти функции-члены предоставляют слишком низкоуровневый интерфейс. Обычно, классы со столь низкоуровневыми интерфейсами уместны в качестве деталей реализации частей более крупных компонентов (если вообще где-либо уместны). В идеале, функции интерфейса должны принимать самоописываемые аргументы.

Язык C++ разрешает программисту экспонировать часть реализации класса в качестве части интерфейса. Эта часть реализации может быть закрытой (*private* или *protected*), но она доступна компилятору, чтобы он мог выделять память под автоматические переменные, встраивать функции и т.д. Отрицательным эффектом такого решения является возможность порождения нежелательных зависимостей в случае, когда пользовательские типы (классы) присутствуют в реализации класса. На самом деле, порождают ли члены класса с типами *Y* и *Z* проблемы или нет, зависит от того, какими на самом деле типами являются *Y* и *Z*. Если это простые типы, вроде *list*, *complex* и *string*, их применение чаще всего вполне оправдано. Это весьма стабильные типы, а необходимость включения их объявлений не является чрезмерной нагрузкой для компилятора. Однако если *Y* и *Z* сами являются интерфейсными классами больших и значимых компонентов, таких как графическая система или система управления банковскими счетами, неразумно зависеть от них столь непосредственно. В подобных случаях лучше использовать указатели или ссылки:

```
class Y;
class Z;

class X    // X имеет доступ к Y и Z только через указатели и ссылки
{
    Y* a;
    Z& b;
    // ..
};
```

Это разъединяет определение *X* от определений *Y* и *Z* (остается зависимость *X* от имен *Y* и *Z*). Реализация *X* будет, конечно же, по-прежнему зависеть от определений *Y* и *Z*, но это напрямую не относится к пользователям класса *X*.

Это иллюстрирует важное положение: интерфейс, скрывающий значительную часть информации — как и должен делать полезный интерфейс — имеет намного меньше зависимостей, чем скрываемая им реализация. Например, определение класса *X* можно откомпилировать без доступа к определениям *Y* и *Z*. Однако определения функций-членов класса *X*, манипулирующие объектами *Y* и *Z*, нуждаются в определениях *Y* и *Z*. При анализе зависимостей зависимости интерфейса и реализации должны рассматриваться отдельно. В обоих случаях для упрощения системы желательно иметь ациклические графы зависимостей. Этот идеал гораздо более важен и достижим для интерфейсов, чем для реализации.

Отметим, что класс может определить три интерфейса:

```
class X
{
private:
    // доступен только для членов и друзей

protected:
    // доступен членам и друзьям, а также
    // членам и друзьям производных классов

public:
    // доступен широкой публике
};
```

Кроме того, друзья тоже относятся к открытому интерфейсу (§11.5).

Поля данных должны попадать в интерфейс в самом редком случае. Они должны быть закрытыми, если нет веских причин делать их более доступными. В последнем случае их желательно делать защищенными, если нет конкретных причин, по которым они должны стать открытыми. Функции и типы, входящие в открытый интерфейс класса, должны соответствовать концепции, которую класс представляет.

Отметим еще, что для большей степени сокрытия представления можно использовать абстрактные классы (§2.5.4, §12.3, §25.3).

24.4.3. «Жирные» интерфейсы

В идеале интерфейс должен предоставлять лишь те операции, которые имеют смысл и хорошо представимы любым производным классом, реализующим этот интерфейс. Это, однако, не всегда легко сделать. Рассмотрим списки, массивы, ассоциативные массивы, деревья и т.д. Как показано в §16.2.2, соблазнительно и иногда полезно обобщить эти типы до типа *контейнер* (*container*), который можно было бы использовать в качестве интерфейса ко всем перечисленным структурам данных. Как кажется, это могло бы освободить пользователя от необходимости работать с деталями многочисленных специализированных контейнеров. Однако определить интерфейс универсального контейнера не так-то легко. Допустим, мы хотим определить **Container** как абстрактный тип. Какими операциями его нужно снабдить? Мы можем ввести лишь операции, которые допустимы для любого из перечисленных специализированных контейнеров, то есть фактически пересечение множеств их операций (до смешного узкий интерфейс). Более того, во многих практических случаях такое пересечение представляет собой пустое множество.

В качестве иной крайности мы можем ввести объединение множество всех операций и выдавать сообщение об ошибке, если во время выполнения программы через такой интерфейс произошло обращение к операции, не поддерживаемой объектом. Интерфейс, реализующий объединение интерфейсов ко множеству концепций, называют *жирным интерфейсом* (*fat interface*). Рассмотрим универсальный контейнер объектов типа *T*:

```
class Container
{
public:
    struct Bad_oper      // класс исключений
    {
        const char* p;
        Bad_oper(const char* pp) : p(pp) {}
    };

    virtual void put(const T*) {throw Bad_oper("Container: : put");}
    virtual T* get() {throw Bad_oper("Container: : get");}

    virtual T* & operator[] (int) {throw Bad_oper("Container: : [] (int)");}
    virtual T* & operator[] (const char*) {throw Bad_oper("Container: : [] (char*)");}
    // ...
};
```

Тогда конкретные контейнеры можно объявлять следующим образом:

```
class List_container: public Container, private list
{
public:
    void put(const T*) ;
    T* get() ;
    // ... омысмыем operator[] ...
};

class Vector_container: public Container, private vector
{
public:
    T* & operator[] (int) ;
    T* & operator[] (const char*) ;
    // ... омысмыюм put() и get() ...
};
```

Пока соблюдаются предосторожности, все в порядке:

```
void f()
{
    List_container sc;
    Vector_container vc;
    // ...
    user(sc, vc);
}

void user(Container& c1, Container& c2)
{
    T* p1 = c1.get();
```

```

T* p2 = c2[3];
// не используйте c2.get() или c1[3]
// ...
}

```

Однако немногие структуры данных одновременно поддерживают и индексацию, и операции, характерные для списков. Поэтому идея объединить эти операции в одном интерфейсе вряд ли хороша. Это влечет либо интенсивный опрос типов во время выполнения (§15.4), либо приводит к генерации исключений (глава 14) ради устранения ошибок времени исполнения. Например:

```

void user2 (Container& c1, Container& c2)    // обнаружение просто, восстановление - нет
{
    try
    {
        T* p1 = c1.get();
        T* p2 = c2[3];
        // ...
    }

    catch (Container : : Bad_oper& bad)
    {
        // Oops!
        // И что теперь делать?
    }
}

```

или

```

void user3 (Container& c1, Container& c2)    // раннее обнаружение не просто;
                                           // восстановление также не просто
{
    if(dynamic_cast<List_container*>(&c1) &&
        dynamic_cast<Vector_container*>(&c2) )
    {
        T* p1 = c1.get();
        T* p2 = c2[3];
        // ...
    }
    else
    {
        // Oops!
        // И что теперь делать?
    }
}

```

В обоих случаях может пострадать быстродействие и код может быть излишне большим. В результате, программисты могут понадеяться на лучшее, то есть что пользователи будут работать с программой надлежащим образом и ошибок возникать не будет. К тому же исчерпывающее тестирование в данном случае труднодостижимо и слишком дорого.

Следовательно, жирных интерфейсов лучше избегать, когда производительность является важнейшим условием, когда требуется высокая надежность исполнения

и вообще, когда есть другая альтернатива. Применение жирных интерфейсов ослабляет степень соответствия между концепциями проекта и классами, и открывает простор для использования наследования исключительно ради удобства реализации.

24.5. Советы

1. Эволюционируйте в направлении абстракции данных и объектно-ориентированного программирования; §24.2.
2. Применяйте средства языка C++ и технологии программирования сообразно необходимости; §24.2.
3. Стремитесь к соответствию стиля программирования стилю проектирования; §24.2.1.
4. В проектировании фокусируйтесь на концепциях/классах, а не на функциях/вычислениях; §24.2.1.
5. Применяйте классы для представления концепций; §24.2.1, §24.3.
6. Применяйте наследование исключительно для отражения иерархических отношений между концепциями; §24.2.2, §24.2.5, §24.3.2.
7. Базируйте строгие гарантии касательно интерфейсов в терминах статических типов уровня приложения; §24.2.3.
8. Для решения четко определенных задач применяйте генераторы кода и другие средства визуального программирования; §24.2.4.
9. Избегайте тех генераторов кода и средств визуального программирования, что плохо стыкуются с языками программирования общего назначения; §24.2.4.
10. Не смешивайте между собой разные уровни абстракции; §24.3.1.
11. Фокусируйтесь на проектировании компонентов; §24.4.
12. Убедитесь, что виртуальные функции имеют четко определенный смысл, и что все замещающие функции реализуют надлежащее поведение; §24.3.4, §24.3.2.1.
13. Используйте открытое наследование для выражения отношения "is-a"; §24.3.4.
14. Используйте агрегацию для отражения отношения "has-a"; §24.3.4.
15. Для отражения отношения включения используйте поля данных с типом включаемого объекта, а не с типом указателя на этот объект; §24.3.3, §24.3.4.
16. Старайтесь делать отношения использования ясными и понятными, нециклическими и минимальными; §24.3.5.
17. Определяйте инварианты для всех классов; §24.3.7.1.
18. Предусловия, постусловия и иные утверждения выражайте с помощью утверждений (возможно, с помощью *Assert()*); §24.3.7.2.
19. Определяйте интерфейсы, раскрывая минимум необходимой информации; §24.4.

20. Минимизируйте зависимости интерфейса от других интерфейсов; §24.4.2.
21. Поддерживайте строгую типизацию интерфейсов; §24.4.2.
22. Выражайте интерфейсы на уровне типов приложения; §24.4.2.
23. Выражайте интерфейс таким образом, чтобы запрос мог передаваться на удаленный сервер; §24.4.2.
24. Избегайте «жирных» интерфейсов; §24.4.3.
25. Применяйте закрытые поля данных и функции-члены где только можно; §24.4.2.
26. Применяйте альтернативу *private/protected* для различения нужд разработчиков производных классов и обычных пользователей; §24.4.2.
27. Для обобщенного программирования применяйте шаблоны; §24.4.1.
28. Пользуйтесь шаблонами для параметризации политики выполнения алгоритмов; §24.4.1.
29. Применяйте шаблоны, если нужно, чтобы разрешение типов производилось на этапе компиляции; §24.4.1.
30. Используйте иерархии классов, когда разрешение типов необходимо на этапе выполнения; §24.4.1.

Роли классов

*Кое-что можно и поменять,
но фундаментальные вопросы должны быть неизблемыми.*
— Стивен Дж. Гульд

Разновидности классов — конкретные типы — абстрактные типы — узловые классы — изменение интерфейсов — объектный ввод/вывод — операции — интерфейсные классы — дескрипторные классы — прикладные среды разработки — советы — упражнения.

25.1. Разновидности классов

Класс в языке C++ — это конструкция, которая отвечает разнообразным проектным целям. Я нахожу, что многие запутанные проблемы проектирования часто решаются введением нового класса, представляющего понятие, которое в предыдущем варианте проекта не было выявлено явным образом (при этом иногда приходится и удалять классы). Множество ролей классов в проекте приводит ко множеству видов классов в программе, специально приспособленных для решения конкретных задач. В настоящей главе рассматривается несколько классов-архетипов с присущими им достоинствами и недостатками:

- §25.2 Конкретные типы
- §25.3 Абстрактные типы
- §25.4 Узлы
- §25.5 Операции
- §25.6 Интерфейсы
- §25.7 Дескрипторные классы (handles)
- §25.8 Прикладные среды разработки

Эти разновидности классов являются проектными понятиями, а не языковыми конструкциями. В идеале (наверное, недостижимом), хорошо бы иметь минималь-

ный набор простых и ортогональных друг другу классов, из которых можно было составлять любые полезные и эффективно работающие классы. Отметим, что в проектах могут пригодиться любые из этих разновидностей классов, и ни одна из них по своей природе не лучше других в общем случае. Большая путаница при обсуждении вопросов проектирования и программирования исходит от людей, предпочитающих лишь одну или две разновидности классов. Делается это, якобы, ради простоты, но на деле приводит лишь к неуклюжему и неестественному применению любимых разновидностей классов.

Мы здесь будем рассматривать чистые формы перечисленных разновидностей классов, хотя можно использовать и гибридные формы. Гибрид, однако, должен появляться в результате осознанного проектного решения, а не в результате отказа от четкого выбора. Откладывание решения выбора на потом часто является завуалированной формой «отказа думать». Новичкам лучше держаться подальше от гибридов и придерживаться стилей уже существующих компонентов, имеющих характеристики, похожие на те, что требуется получить от нового решения. Только опытным программистам по плечу писать компоненты и библиотеки общего назначения, предназначенные для длительного использования, и которые нужно хорошо документировать и поддерживать все это время. См. §23.5.1.

25.2. Конкретные классы (типы)

Такие классы, как *vector* (§16.3), *list* (§17.2.2), *Date* (§10.3) и *complex* (§11.3, §22.5), являются *конкретными* (*concrete*) в том смысле, что каждый из них представляет относительно простое понятие вместе с операциями, достаточными для поддержки этого понятия. Кроме того, в каждом из этих классов имеется взаимнооднозначное соответствие между интерфейсом и реализацией, и ни один из этих классов не предназначен для создания производных классов. Как правило, конкретные классы не вписываются в иерархии связанных классов. Каждый конкретный тип понятен сам по себе с минимальнейшей отсылкой к другим классам. Если конкретный тип хорошо реализован, то использующие его программы столь же малы по размеру и столь же эффективны, как и варианты программы, в которых программист самостоятельно вручную кодирует специализированные версии той же самой концепции. Аналогично, если реализация изменяется существенно, изменяется и интерфейс для отражения этих изменений. Во всем этом конкретный класс напоминает встроенные типы (естественно, все встроенные типы конкретны). Определяемые пользователем конкретные типы, такие как комплексные числа, матрицы, сообщения об ошибках и т.д., часто представляют собой фундаментальные понятия для определенных предметных областей.

Точная природа классового интерфейса определяет, какие изменения в реализации имеют существенное значение в данном контексте; более абстрактные интерфейсы оставляют больше простора для изменения в реализации, но могут приводить к снижению быстродействия. Хорошая реализация не зависит от других классов больше, чем это необходимо, в результате чего снижаются затраты на «притирку» с иными классами как во время компиляции программы, так и во время ее выполнения.

Таким образом, подводя итог, можно сказать, что конкретный класс нацелен на то, чтобы:

1. Как можно точнее соответствовать частной концепции и стратегии ее реализации.
2. Обеспечивать затраты памяти и быстродействие, сравнимые с таковыми со специализированными «ручными» вариантами кода, за счет применения встраивания и операций, использующих все конкретные особенности концепции и ее реализации.
3. В минимальной степени зависеть от других классов.
4. Быть понятным и пригодным к использованию независимо от других классов.

В результате достигается тесная связь между кодом реализации и кодом пользователя класса. Если реализация как-то меняется, пользовательский код придется перекомпилировать, поскольку пользовательский код почти всегда содержит обращение ко встраиваемым функциям или внутренним переменным конкретного класса.

Название «конкретный тип» было выбрано по контрасту с распространенным термином «абстрактный тип». Взаимосвязь между конкретными и абстрактными типами обсуждается в §25.3.

Конкретные типы непосредственным образом не выражают отношения общности. Например, *list* и *vector* предоставляют схожие наборы операций и могут взаимозаменяемо использоваться в шаблонах функций. Тем не менее, никакого отношения общности у классов *list<int>* и *vector<int>*, или между *list<Shape*>* и *list<Circle*>* (§13.6.3), нет, хотя мы сами-то и можем это сходство углядеть.

Для наивно спроектированных конкретных типов это приводит к тому, что код, использующий их схожим образом, будет выглядеть по-разному. Например, итерации по списку *List* при помощи функции *next()* совсем не похожи на итерации по типу *Vector* при помощи индексации:

```
void my (List& sl)
{
    for (T* p = sl.first () ; p ; p = sl.next () )    // естественная итерация по списку
    {
        // мой код
    }
    // ...
}

void your (Vector& v)
{
    for (int i = 0 ; i < v.size () ; i++)              // естественная итерация по вектору
    {
        // ваш код
    }
    //...
}
```

Разница в стиле итераций вполне естественна в том смысле, что операция «получить следующий элемент» органична для списков, но не для векторов, а опера-

ция индексации является фундаментальной для векторов (но не для списков). Наличие органичных для выбранной стратегии реализации контейнера операций критически важно с точки зрения достижения высокой эффективности и удобства записи.

Неприятность такого подхода состоит в том, что программы для фундаментально схожих операций, используемых в двух приведенных выше циклах, выглядят различно, и код, применяющий конкретные типы для логически эквивалентных операций, не взаимозаменяем. В итоге, в программах реальных размеров выявить такие схожести бывает очень сложно, и сложно бывает перепроектировать систему для эксплуатации обнаруженной общности. Удачным примером перепроектирования, позволившим задействовать сходство между конкретными типами, не пожертвовав при этом ни элегантностью, ни эффективностью, являются стандартные контейнеры и алгоритмы (§16.2).

Если функция принимает аргумент конкретного типа, уже не будет никакой возможности принять менее специфичный аргумент (как это бывает в отношениях наследования). Поэтому попытка задействовать сходство между конкретными типами с неизбежностью приводит к использованию шаблонов и обобщенного стиля программирования, как описано в §3.8. Для стандартной библиотеки итерации принимают следующий вид:

```
template<class C> void ours (C& c)
{
    for (typename C::iterator p=c.begin(); p!=c.end(); ++p) // итерации стандарт. биб-ки
    {
        // ...
    }
}
```

Здесь использовано фундаментальное сходство между контейнерами, открывающее дополнительные возможности, используемые стандартными алгоритмами (глава 18).

Чтобы эффективно применять конкретные типы, пользователь должен хорошо понимать его частные детали. В библиотеке конкретных классов нет никакого общего свойства для всех содержащихся в ней типов, так что пользователю приходится вникать в каждый класс по отдельности. Это плата за быстроедействие и компактность, причем эта плата может окупиться, а может и не окупиться. В то же время, часто легче понять и использовать индивидуальный конкретный класс, чем более общий и абстрактный. Это характерно для таких широко известных типов данных, как, например, массивы и списки.

В идеале лучше скрывать реализацию в максимально возможной степени, не допуская при этом серьезных ухудшений производительности. Здесь встраиваемые функции могут дать большой выигрыш.

Открывать поля данных класса, чтобы пользователь манипулировал ими напрямую или через специально предназначенные для этого функции-члены, редко когда бывает хорошей идеей (§24.4.2). Конкретные типы должны оставаться типами, а не мешками битов с несколькими функциями, добавленными лишь для удобства.

25.2.1. Многократное использование конкретных типов

Конкретные типы редко когда бывает полезным использовать в виде базовых классов. Каждый конкретный тип призван обеспечить четкое и эффективное представление единственной концепции. Маловероятно, чтобы класс, хорошо справляющийся с такой задачей, был хорошим кандидатом для создания других, родственных классов посредством открытого наследования. Такие классы лучше использовать в качестве типов полей данных других классов, или в качестве закрытых базовых классов. Тут они могут использоваться эффективно, не смешивая при этом свои интерфейсы и реализацию с интерфейсами и реализацией новых классов. Рассмотрим наследование от класса *Date*:

```
class My_date: public Date
{
    // ...
};
```

Нужно ли когда-нибудь использовать *My_date* вместо *Date*? Это, конечно, зависит от того, что именно представляет собой *My_date*, но по моему опыту конкретные классы редко бывают полезны в качестве базовых без серьезных модификаций.

Немодифицируемым образом конкретные классы «повторно используются» так же, как и встроенные типы, вроде *int* (§10.3.4). Например:

```
class Date_and_time
{
private:
    Date d;
    Time t;

public:
    // ...
};
```

Такая форма использования (повторного?) проста, эффективна и продуктивна.

Может, это ошибка проектирования привела к тому, что класс *Date* не пригоден к повторному использованию посредством наследования от него производных классов? Иногда слышны утверждения, что *каждый класс* должен проектироваться с учетом последующей модификации посредством замещения и посредством использования его членов функциями-членами производных классов. Эта точка зрения привела бы к следующему варианту класса *Date*:

```
class Date2
{
public:
    // публичный интерфейс, состоящий преимущественно из виртуальных функций

protected:
    // другие детали реализации (возможно, включая часть представления)

private:
    // представление и иные детали реализации
};
```

Часть представления класса сделана защищенной с целью облегчить замещение функций в производных классах. Это позволяет сделать тип **Date2** весьма податливым к модификациям посредством наследования, оставляя его открытый пользовательский интерфейс неизменным. Цена такого решения следующая:

1. *Базовые операции менее эффективны.* Обращение к виртуальным функциям в C++ немного медленнее, чем к обычным функциям; виртуальные функции редко когда бывает разумным делать встраиваемыми; объекты классов с виртуальными функциями требуют добавочного расхода памяти.
2. *Необходимость динамического использования свободной памяти.* Цель введения класса **Date2** — позволить использовать взаимозаменяемым образом объекты классов, производных от **Date2**. Поскольку размеры этих объектов разные, их нужно очевидным образом размещать в динамически выделяемой свободной памяти, обращаться к которой придется с помощью указателей или ссылок. Локализация данных класса при этом резко уменьшается.
3. *Неудобство для пользователей.* Выгоды от полиморфизма, обеспечиваемого виртуальными функциями, достигаются лишь при манипулировании объектами класса через указатели или ссылки.
4. *Слабая инкапсуляция.* Виртуальные функции можно замещать, и защищенными данными можно манипулировать из производных классов (§12.4.1.1).

Рассмотренная цена решения в общем случае не столь уж и велика, и часто это решение оказывается именно тем, что нужно (§25.3, §25.4). В то же время, для таких простых типов, как **Date2**, цена явно излишняя и совершенно необязательная.

Наконец, четко спроектированный конкретный тип часто подходит для реализации (представления) более гибких (податливых) типов. Например:

```
class Date3
{
public:
    // публичный интерфейс, состоящий преимущественно из виртуальных функций

private:
    Date d;
};
```

Именно этот способ следует использовать при необходимости встроить конкретные типы в иерархию классов. См. также §25.10[1].

25.3. Абстрактные типы

Простейшим способом ослабить связь между пользователями класса и его разработчиками, а также между создающим объекты кодом и кодом, использующим объекты, является введение абстрактного класса, предоставляющего интерфейс к набору реализаций некоторой общей концепции. Рассмотрим шаблон **Set** (множество):

```
template<class T> class Set
{
public:
```



```

virtual void insert (T*) = 0;
virtual void remove (T*) = 0;

virtual int is_member (T*) = 0;

virtual T* first () = 0;
virtual T* next () = 0;

virtual ~Set () {}
};

```

Это интерфейс ко множеству со встроенной концепцией итерации по его элементам. Типично отсутствие конструкторов и присутствие виртуального деструктора (§12.4.2). Допускаются разные реализации (§16.2.1). Например:

```

template<class T> class List_set: public Set<T>, private list<T>
{
    // ...
};

template<class T> class Vector_set: public Set<T>, private vector<T>
{
    // ...
};

```

Абстрактный класс предоставляет интерфейс к любым реализациям. Это означает, что мы можем применять **Set**, не зная, какая из реализаций используется. Например:

```

void f( Set<Plane*> & s)
{
    for (Plane** p = s.first () ; p; p = s.next () )
    {
        // мой код
    }
    // ...
}

List_set<Plane*> sl;
Vector_set<Plane*> v (100) ;

void g ()
{
    f(sl) ;
    f(v) ;
}

```

Для конкретных типов требуется перепроектирование классов реализации с целью отражения общности и применение шаблонов для эксплуатации этой общности. Здесь же общность отражается интерфейсом (то есть **Set**), а от классов реализации не требуется никакой другой общности, кроме способности реализовывать интерфейс.

Более того, пользователи **Set** не знают объявлений **List_set** и **Vector_set**, и поэтому пользователи не зависят от этих объявлений и не нуждаются в перекомпиляции в случае, если изменяются **List_set** или **Vector_set**, или даже если вводится новая ре-

лизация *Set* — скажем, *Tree_set*. Все зависимости сосредоточены в функциях, которые явным образом используют классы, производные от *Set*. В частности, предполагая традиционное использование заголовочных файлов, программисту, пишущему *f(Set&)*, нужно включить лишь *Set.h*, а не *List_set.h* или *Vector_set.h*. «Реализационный» заголовочный файл нужен лишь там, где создаются *List_set* или *Vector_set*. Пользовательский код можно еще сильнее изолировать от классов реализации, введя абстрактный класс, обрабатывающий запросы на создание объектов (так называемая «фабрика»; §12.4.4).

Отделение интерфейса от реализации означает отсутствие доступа к операциям, свойственным конкретной реализации, но не достаточно общим, чтобы стать частью интерфейса. Например, поскольку множество неупорядочено по своей сути, мы не можем включить в интерфейс операцию индексации даже в том случае, когда для реализации применяется частное представление в виде массива. Отсюда вытекает некоторая потеря эффективности за счет запрета ручной оптимизации. Кроме того, встраивание функций, как правило, недостижимо (кроме локального контекста, когда компилятор знает истинный тип), а все интересные операции интерфейса представлены виртуальными функциями. Как и в случае с конкретными типами, затраты на абстрактные типы иногда окупаются, а иногда нет. Подводя итог, можно сказать, что абстрактные типы предназначены для того, чтобы:

1. Определять одну концепцию с разными реализациями, сосуществующими в пределах программы.
2. Обеспечивать приемлемые быстродействие и затраты памяти посредством использования виртуальных функций.
3. Минимизировать зависимость каждой реализации от других классов.
4. Иметь возможность самоописания.

Абстрактные типы не лучше конкретных типов; они просто другие. Разработчики библиотек часто предоставляют и те, и другие типы, оставив выбор за пользователем. Попытки ограничить общность абстрактного типа ради конкуренции с конкретными типами в быстродействии как правило оканчиваются неудачей; это компрометирует возможность использовать взаимозаменяемые реализации без значительной перекомпиляции после их модификации. Также неудачей оканчиваются и попытки увеличить универсальность конкретных типов; при этом снижается их эффективность и удобство использования. Конкретные и абстрактные типы могут сосуществовать — на самом деле они *должны* сосуществовать, ибо конкретные типы обеспечивают реализацию абстрактных типов. Однако не следует смешивать эти типы хаотическим образом.

Часто, абстрактные типы не предназначены для порождения производных классов, отличных от классов непосредственной реализации. Однако из абстрактного класса можно сконструировать новый интерфейс, породив расширенный производный класс. Этот новый интерфейс (новый абстрактный класс) также нужно реализовать посредством неабстрактных производных классов (§15.2.5).

Почему же мы сразу не породили *List* и *Vector* от *Set*, избежав введения классов *List_set* и *Vector_set*? Иными словами, зачем нам потребовались конкретные типы вместо абстрактных типов? Здесь нужно учитывать следующие соображения:

1. *Эффективность.* Нам могут потребоваться конкретные типы, вроде **vector** и **list**, без затрат на отделение интерфейса от реализации (сопутствуют стилю абстрактных классов).
2. *Повторное использование.* Нам нужен механизм приспособления спроектированных вовне типов (вроде **vector** и **list**) в новые библиотеки или программы путем предоставления для них нового интерфейса (а не путем переписывания их кода).
3. *Множественные интерфейсы.* Использование одного класса в качестве базового для многих классов приводит к жирным интерфейсам (§24.4.3). Для новых задач лучше снабдить класс новым интерфейсом (вроде интерфейса **Set** для **vector**), чем приспособливать старый интерфейс под новые задачи.

Естественно, эти положения связаны между собой. Подробно они обсуждались на примере **Ival_box** (§12.4.2, §15.2.5), а также в контексте проектирования контейнеров (§16.2).

25.4. Узловые классы

Часто классовые иерархии используют наследование не в том ключе, как в случае абстрактного класса, предоставляющего интерфейс к реализациям. Здесь класс рассматривается в качестве фундамента, на котором строится часть системы. Даже, если этот класс абстрактный, он все равно имеет некоторое представление, обеспечивающее услуги своим производным классам. Примерами узловых классов служат **Polygon** (§12.3), первоначальный вариант **Ival_slider** (§12.4.1) и **Satellite** (§15.2).

В типичном случае, класс иерархии представляет собой некоторую концепцию, для которой производные классы могут рассматриваться как специализации этой концепции. *Узловой класс (node class) иерархии* использует сервисы базового класса для того, чтобы предоставлять свои собственные сервисы; для этого он вызывает функции-члены базового класса. Типичный узловой класс не только реализует интерфейс, описанный базовым классом (как это для абстрактного типа делает класс реализации), но и добавляет свои собственные функции, обеспечивая тем самым расширенный интерфейс. Рассмотрим класс **Car** из примера моделирования уличного движения в §24.3.2:

```
class Car: public Vehicle
{
public:
    Car(int passengers, Size_category size, int weight, int fc)
        : Vehicle(passengers, size, weight), fuel_capacity(fc) { /* . . . */ }

    // замещаем виртуальные функции из Vehicle:
    void turn(Direction) ;
    // ..
    // добавляем специфические автомобильные функции:
    virtual void add_fuel(int amount) ;    // заправка топливом
    //
```

Здесь важны конструкторы, через которые программист задает основные моделируемые свойства, и виртуальные функции, позволяющие моделирующим уличное движение операциям манипулировать транспортным средством, не зная его точного типа. Класс *Car* можно использовать следующим образом:

```
void user ()
{
    // ...
    Car* p = new Car (3, economy, 1500, 60) ;
    drive (p, bs_home, MH) ;    // входим в общую схему имитации уличного движения
    // ...
}
```

Обычно узловому классу нужны нетривиальные конструкторы. Этим он отличается от абстрактных типов, которые редко когда имеют конструкторы.

Операции над классом *Car* в своих реализациях будут, как правило, пользоваться операциями базового класса *Vehicle*. Кроме того, пользователь класса *Car* также рассчитывает на услуги базового класса. Например, базовый класс *Vehicle* предоставляет функции, имеющие дело с весом и размерами, так что классу *Car* нет нужды этим заниматься:

```
bool Bridge::can_cross (const Vehicle& r)
{
    if (max_weigh < r.weight ()) return false;
    // ...
}
```

Это позволяет программистам создавать из узлового класса *Vehicle* новые классы, такие как *Car* и *Truck*, специфицируя и реализуя лишь то, что должно быть отличным от *Vehicle*. Это часто называют «программированием отличий».

Как и многие узловые классы, класс *Car* является хорошей кандидатурой для порождения производных классов. Например, *Ambulance* (скорая помощь) нуждается в дополнительных данных и функциях, связанных с экстренностью службы:

```
class Ambulance: public Car, public Emergency
{
public:
    Ambulance () ;

    // замещаем виртуальные функции из Car:
    void turn (Direction) ;
    // ...

    // замещаем виртуальные функции из Emergency:
    virtual void dispatch_to (const Location&) ;
    // ...

    // добавляем специфические для Ambulance функции:
    virtual int patient_capacity () ;    // количество носилок
    // ...
}
```

Подводя итог, можно сказать, что узловой класс:

1. Опирается на базовые классы как с целью реализации, так и для предоставления услуг своим пользователям.
2. Обеспечивает более широкий интерфейс (с большим числом открытых функций-членов) по сравнению с базовыми классами.
3. В своем открытом интерфейсе опирается в первую очередь (но не исключительно) на виртуальные функции.
4. Зависит (прямо или косвенно) от всех своих базовых классов.
5. Может быть понят только в контексте своих базовых классов.
6. Полезен для дальнейшего порождения классов наследованием.
7. Используется для создания объектов.

Не все узловые классы будут удовлетворять пунктам 1, 2, 6 и 7, но большинство будет. Класс, не удовлетворяющий пункту 6, напоминает конкретный тип, так что его можно назвать *конкретным узловым классом* (*concrete node class*). Например, конкретный узловой класс может использоваться для реализации абстрактного класса (§12.4.2), а его объекты могут создаваться статически и в стеке. Такой класс также называют *листовым классом* (*leaf class*). Отметим, что любой код, оперирующий указателем или ссылкой на класс с виртуальными функциями, должен учитывать возможность появления дальнейших производных классов (или предположить, без прямой языковой поддержки, что дальнейших классов не будет). Класс, не удовлетворяющий пункту 7, напоминает абстрактный тип и может быть назван *абстрактным узловым классом* (*abstract node class*). По сложившейся традиции (неудачной) многие узловые классы имеют ряд защищенных членов, чтобы не слишком сильно ограничивать интерфейс для производных классов (§12.4.1.1).

Пункт 4 подразумевает, что для компиляции узлового класса программист должен включить объявления всех его прямых и непрямых базовых классов, а также все объявления, от которых зависят последние. В этом отношении узловой класс опять-таки отличается от абстрактных классов — пользователь последних не зависит от классов реализации и может не включать для компиляции своего кода информацию об этих классах.

25.4.1. Изменение интерфейсов

По определению, узловой класс является частью классовой иерархии. Не все классы иерархии должны предоставлять один и тот же интерфейс. В частности, производные классы могут предоставлять дополнительные функции-члены, а классы одного и того же уровня иерархии (*sibling classes*) — совершенно иной набор функций. С проектной точки зрения операция *dynamic_cast* (§15.4) представляется механизмом опроса объектов на предмет, обеспечивают ли они данный интерфейс.

Для примера рассмотрим простую систему ввода/вывода объектов. Пользователи хотят читать объекты из потока, определять, относятся ли они к ожидаемому типу, после чего использовать эти объекты. Например:

```
void user ()
{
    // открываем файл предположительно с фигурами типа Shape и прикрепляем
    // ss в качестве istream для этого файла
```


Ассоциативный массив типа *map*, названный *io_map*, содержит пары из имен, классов и функций, способных создавать объекты этих классов.

Мы могли бы определить класс *Shape* обычным образом, кроме того, что его нужно сделать производным от *Io_obj*, как того требует функция *user()*:

```
class Shape: public Io_obj
{
    // ...
};
```

Однако интереснее (и реалистичнее) воспользоваться ранее определенным классом *Shape* (§2.6.2) без изменений:

```
class Io_circle: public Circle, public Io_obj
{
public:
    Io_circle* clone() const {return new Io_circle (*this) ; }
    Io_circle(istream&) ;      // инициализация из потока ввода
    static Io_obj* new_circle(istream& s) {return new Io_circle(s) ; }
    // ...
};
```

Этот пример показывает, насколько легче вписать класс в иерархию при помощи абстрактного класса, чем если бы мы в первую очередь рассматривали его в качестве узлового (§12.4.2, §25.3).

Конструктор *Io_circle(istream&)* инициализирует объект данными из своего аргумента типа *istream*. Функция *new_circle()* помещается в ассоциативный массив *io_map*, чтобы дать знать системе объектного ввода/вывода о новом классе. Например:

```
io_map["Io_circle"] = &Io_circle::new_circle;
```

Другие фигуры создаются таким же образом:

```
class Io_triangle: public Triangle, public Io_obj
{
    // ...
};
```

Это может и утомить, так что лучше определить шаблон:

```
template<class T> class Io: public T, public Io_obj
{
public:
    Io* clone() const {return new Io (*this) ; }      // замечаем Io_obj::clone()
    Io(istream&) ;      // инициализация из потока ввода
    static Io* new_io(istream& s) {return new Io(s) ; }
    //...
};
```

и уже с его помощью определить *Io_circle*:

```
typedef Io<Circle> Io_circle;
```

Впрочем, нам по-прежнему нужно явно определить *Io<Circle>::Io(istream&)*, поскольку при этом требуется знать подробности о *Circle*.

Шаблон *Io* служит примером того, как вписать конкретные типы в классовую иерархию с помощью класса, являющегося узловым в этой иерархии. Шаблон наследует от типа его параметра, чтобы обеспечить приведение типа от *Io_obj*. К сожалению, из-за этого становится невозможным использовать *Io* со встроенными типами:

```
typedef Io<Date> Io_date;      // обертка для конкретного типа
typedef Io<int> Io_int;        // error: наследование от встроенного типа невозможно
```

Эту проблему можно уладить, введя отдельный шаблон для встроенных типов, или воспользовавшись классом, представляющим встроенный тип (§25.10[1]).

Эта простая система объектного ввода/вывода не может делать все на свете, но ее код почти уместается на одной странице, а ее ключевые механизмы имеют множество применений. Например, их можно использовать для вызова функции, строка с именем которой предоставляется пользователем, и для манипулирования объектами неизвестного типа при помощи интерфейсов, выявляемых средствами динамического определения типа.

25.5. Операции

Самый простой и наиболее очевидный способ определить какую-нибудь операцию в языке C++ — это написать соответствующую функцию. Однако если операция должна быть отложенной (*delayed*), или ее нужно передать куда-либо для выполнения, или она должна вовлекать в вычисления собственные данные или комбинироваться с другими операциями (§25.10[18,19]) и т.д. — то возникает желание оформить ее в виде класса, который может выполнить эту операцию, а также предоставить множество других услуг. Очевидным примером здесь являются классы функциональных объектов (классы объектов-функций), используемые совместно со стандартными алгоритмами (§18.4), а также манипуляторы, применяемые с потоками *iostream* (§21.4.6). В первом случае собственно действие вызывается операцией «круглые скобки», а во втором — операциями << и >>. В случае класса *Form* (§21.4.6.3) и *Matrix* (§22.4.7) классы-композиторы (*compositor classes*) позволяют отложить операцию до тех пор, пока не будет собрано достаточно информации для ее эффективного выполнения.

Простейшая форма класса-операции обычно содержит одну виртуальную функцию (называемую по традиции что-нибудь вроде «do_it» — «сделай это»):

```
struct Action
{
    virtual int do_it (int) =0;
    virtual ~Action () {}
};
```

Теперь мы можем написать код, например меню, который сможет сохранять действия (операции) для их последующего выполнения, не используя указателей на функции, не имея информации о вызванных объектах, и даже не зная имени операции, подлежащей выполнению. Например:

```
class Write_file: public Action
{
    File& f;
```



```

public:
    int do_it(int) {return f.write() .succeed();}
};

class Error_response: public Action
{
    string message;
public:
    Error_response(const string& s) : message(s) {}
    int do_it(int);
};

int Error_response::do_it(int)
{
    Response_box db(message.c_str(), "continue", "cancel", "retry");
    switch(db.get_response())
    {
        case 0:
            return 0;
        case 1:
            abort();
        case 2:
            current_operation.redo();
            return 1;
    }
}

Action* actions[] = { new Write_file(f),
                     new Error_response("you blew it again"),
                     // ...
};

```

Пользователь *Action* может ничего не знать о производных классах, таких как *Write_file* и *Error_response*.

Описанная техника чрезвычайно мощная и людям с привычкой к функциональной декомпозиции ею нужно пользоваться с известной осторожностью. Если слишком много классов начинают напоминать *Action*, это может свидетельствовать об ухудшении проекта и крене в нечто излишне функциональное.

Наконец, класс может кодировать операцию для выполнения на другом сетевом (удаленном — remote) компьютере, или для сохранения ее на диске с целью выполнения когда-либо в будущем (§25.10[18]).

25.6. Интерфейсные классы

Одними из самых важных являются скромные интерфейсные классы, на которые часто смотрят свысока. Интерфейсный класс мало что делает — если бы делал, то не был бы тогда интерфейсным классом. Он просто приспособливает подачу услуг под «местные» требования.

Чистейшая форма интерфейса даже не вызывает генерации кода. Рассмотрим специализацию шаблона *Vector* из §13.5:

```

template<class T> class Vector<T*>: private Vector<void*>
{
public:
    typedef Vector<void*> Base;

    Vector() {}
    Vector(int i) : Base(i) {}

    T* & operator[] (int i) {return reinterpret_cast<T*>(Base::operator[] (i));}
    // ...
};

```

Эта (частичная) специализация превращает небезопасный *Vector<void*>* в гораздо более полезное семейство векторных классов, безопасных с точки зрения типов. Встраиваемые функции позволяют сделать интерфейсные классы более приемлемыми. Например, здесь встраиваемая функция выполняет всего лишь подгонку типа, так что важно, что она не влечет дополнительных накладных расходов (ни по памяти, ни по производительности).

Естественно, абстрактный базовый класс, отражающий абстрактную концепцию и реализуемый конкретными типами (§25.2), тоже является в некотором смысле интерфейсным классом, как и дескрипторные классы из §25.7. Однако здесь мы фокусируемся на классах, которые не выполняют никаких специфических функций, кроме подгонки интерфейсов.

Рассмотрим проблему слияния двух иерархий с применением множественного наследования. Что делать, если имеет место конфликт имен, например, если виртуальные функции, выполняющие совершенно разные операции, имеют совпадающие имена? Рассмотрим, например, игру в «Дикий Запад», где взаимодействие с пользователем осуществляется посредством общего оконного класса:

```

class Window
{
    // ...
    virtual void draw();
};

class Cowboy
{
    // ...
    virtual void draw();
};

class Cowboy_window: public Cowboy, public Window
{
    // ...
};

```

Cowboy_window представляет анимацию ковбоя, и через этот класс пользователь осуществляет управление ковбоем. Мы предпочли множественное наследование агрегации членов с типами *Cowboy* и *Window*, поскольку имеется множество вспомогательных сервисов, определенных как для *Window*, так и для *Cowboy*. Хотелось бы передавать таким функциям *Cowboy_window*, не требуя от программиста никаких дополнительных действий. Это, однако, приводит к проблеме определения в классе *Cowboy_window* версий функций *Cowboy::draw()* и *Window::draw()*.

В *Cowboy_window* может быть только одна функция *draw()*. Но поскольку сервисные функции манипулируют объектами *Window* и *Cowboy*, ничего не зная о *Cowboy_window*, последний должен заместить *draw()* и для класса *Cowboy*, и для класса *Window*. Замещение их обеих единственной функцией будет неправильным, так как, несмотря на одинаковое название это разные функции. Наконец, хотелось бы получить однозначно различающиеся имена для унаследованных в *Cowboy_window* функций *Cowboy::draw()* и *Window::draw()*.

Для решения проблемы введем по дополнительному классу для *Cowboy* и для *Window*. Эти классы введут два новых имени для функций *draw()*, а также заместят *draw()* при наследовании от *Cowboy* и *Window* так, что вызываться будут функции с новыми именами:

```
class CCowboy: public Cowboy    // интерфейс к Cowboy, переименовывающий draw()
{
public:
    virtual int cow_draw() = 0;
    void draw() { cow_draw(); } // замещаем Cowboy::draw()
};

class WWindow: public Window    // интерфейс к Window, переименовывающий draw()
{
public:
    virtual int win_draw() = 0;
    void draw() { win_draw(); } // замещаем Window::draw()
};
```

Теперь мы можем составить *Cowboy_window* из интерфейсных классов *CCowboy* и *WWindow*, и заместить *cow_draw()* и *win_draw()*, чтобы добиться желаемого эффекта:

```
class Cowboy_window: public CCowboy, public WWindow
{
    // ...
    void cow_draw();
    void win_draw();
};
```

Заметим, что проблема оказалась серьезной по той причине, что две функции с одинаковыми именами имели аргументы одинакового типа. Если бы их аргументы различались, обычные правила перегрузки гарантировали бы различение функций с одинаковыми именами.

Для каждого способа применения интерфейсных классов можно было бы, в принципе, представить некоторое расширение языка, которое выполняло бы необходимое приспособление чуть более элегантно и эффективно. Однако каждый вариант применения интерфейсных классов встречается не столь уж и часто, так что сопоставление каждому из них дополнительных языковых конструкций внесло бы в язык ненужную сложность. Кроме того, столкновение имен при слиянии двух классовых иерархий тоже нельзя назвать обычным явлением (по сравнению, например, с тем, как часто программист пишет классы), и, к тому же, оно имеет место лишь при объединении иерархий из разных предметных областей — таких как игры и системы оконного интерфейса. Слияние непохожих друг на друга иерархий — во-

обще столь непростая задача, что разрешение конфликтов имен далеко не единственная возникающая при этом проблема; есть еще разные способы инициализации, разные стратегии обработки ошибок и управления памятью и т.д. Мы здесь обсуждаем разрешение конфликтов имен лишь потому, что рассмотренный прием с введением интерфейсных классов, имеющих функции с переадресацией вызовов (forwarding functions), сам по себе интересен и имеет множество иных применений. Его можно применять не только для изменения имен функций, но и для изменения типов их аргументов и возврата, для введения проверок на стадии выполнения и др.

Поскольку функции **CCowboy::draw()** и **WWindow::draw()** переадресуют свой вызов виртуальным функциям, их нельзя оптимизировать встраиванием. Но если компилятор распознает их в качестве переадресующих функций, он сможет оптимизировать код, убрав их из цепочки вызовов.

25.6.1. Подгонка интерфейсов

Основное применение интерфейсных функций состоит в подгонке интерфейса под ожидания пользователей так, чтобы сконцентрировать в рамках интерфейса весь код, который иначе был бы разбросан по всей программе. Например, отсчет индексов в стандартном контейнере **vector** начинается с нуля. Если пользователи не хотят, чтобы индексы занимали диапазон от **0** до **size-1**, они должны приспособить вектора для своих нужд. Например:

```
void f()
{
    vector v<int> (10);           // диапазон [0:9]
    // делаем вид, что диапазон [1:10]:
    for (int i = 1; i<=10; i++)
    {
        v[i-1] = 7;              // подправляем индекс
        // ...
    }
}
```

Еще лучше снабдить вектор произвольными границами:

```
class Vector: public vector<int>
{
    int lb;
public:
    Vector (int low, int high) : vector<int> (high-low+1) {lb=low; }
    int& operator[] (int i) {return vector<int>::operator[] (i-lb); }
    int low() {return lb; }
    int high() {return lb+size()-1; }
};
```

Класс **Vector** можно использовать следующим образом:

```
void g()
{
    Vector v (1, 10);             // диапазон [1:10]
```

```
for (int i = 1; i<=10; i++)
{
    v[i] = 7;
    //...
}
}
```

Это не приводит к дополнительным по сравнению с предыдущим примером затратам. В то же время, вариант с *Vector* намного нагляднее, его легче и читать, и писать, так что он менее подвержен ошибкам.

Интерфейсные классы обычно невелики и (по определению) мало что делают. Однако они становятся необходимы, когда нужно объединить программные компоненты, написанные согласно разным традициям. Например, интерфейсные классы часто используются для предоставления C++-интерфейса к коду, написанному на других языках программирования, а также для изоляции пользовательского кода от деталей библиотеки (оставляя, тем самым, возможность замены библиотеки).

Другое важное применение интерфейсных классов — предоставление проверяемых или ограничительных интерфейсов. Например, нередко требуется обеспечить ограничение целых значений заданным диапазоном. Это можно гарантировать (во время выполнения программы) с помощью простого шаблона:

```
template<int low, int high> class Range
{
    int val;

public:
    class Error { };           // класс исключений

    Range (int i) { Assert<Error> (low<=i&&i<high) ; val = i; }    // см. §24.3.7.2

    Range operator= (int i) { return *this=Range (i) ; }
    operator int () { return val; }
    // ...
};

void f (Range<2, 17>) ;
void g (Range<-10, 10>) ;

void h (int x)
{
    Range<0, 2001> i = x;    // может сгенерировать Range::Error
    int i1 = i;

    f(3) ;
    f(17) ;                  // генерируется Range::Error
    g(-7) ;
    g(100) ;                 // генерируется Range::Error
}
```

Шаблон *Range* легко расширяется для работы с диапазонами произвольных скалярных типов (§25.10[7]).

Часто интерфейсный класс, контролирующий доступ к другому классу или выполняющий подгонку его интерфейса, называют *оберткой* (*wrapper*).

25.7. Дескрипторные классы (handles)

Абстрактный класс помогает эффективно разделить интерфейс и его реализации. Однако если следовать §25.3, то получается неразрывная связь между интерфейсом, предоставляемым абстрактным типом, и его реализацией, обеспечиваемой конкретным типом. Например, невозможно отвязать абстрактный итератор от одного источника (скажем, множества) и привязать его к другому (например, потоку) по мере исчерпания первого источника.

Кроме того, если манипулировать объектами, реализующими абстрактный класс, не через указатели или ссылки, то теряются выгоды, предоставляемые виртуальными функциями. Пользовательский код становится зависимым от деталей класса реализации, ибо невозможно выделить память под эти объекты статически или в стеке, не зная точного размера объектов. Использование же указателей или ссылок означает, что бремя управления памятью ложится на пользователя.

Распространенный способ решения этих проблем состоит в разделении одного объекта на два разных: *дескриптора* (*handle*), обеспечивающего пользовательский интерфейс, и *представления* (*representation*), содержащего все или большую часть состояния объекта. Соединяет их указательное поле в дескрипторе. В типичном случае класс дескриптора содержит чуть больше данных, чем один лишь указатель на представление, но не намного. Это означает, что расположение в памяти объектов дескрипторного класса остается неизменным, даже если представление изменяется, и что дескрипторные объекты достаточно малы, чтобы их можно было безболезненно передавать в функции по значению, так что отпадает нужда в указателях или ссылках.



Дескрипторы иллюстрируются классом *String* из §11.12; дескрипторный класс обеспечивает интерфейс, контроль за доступом и управления памятью для представления. В данном случае и дескриптор, и представление задаются конкретными классами, но часто представлению соответствуют абстрактные классы.

Рассмотрим абстрактный тип *Set* (множество) из §25.3. Как можно ввести для него дескриптор и какие выгоды это даст (и какой ценой)? Отталкиваясь от класса *Set*, можно определить дескрипторный класс, просто перегрузив операцию $\rightarrow$:

```

template<class T> class Set_handle
{
    Set<T>* rep;

public:
    Set<T>* operator->() {return rep;}
    Set_handle(Set<T>* pp) : rep(pp) {}
};
  
```

Это не меняет существенно то, как используются множества; просто всюду вместо *Set*& или *Set** передаются *Set_handle*. Например:

```

void f(Set_handle<int> s)
{
    for (int* p = s->first(); p; p = s->next())
    {
        // ...
    }
}

void user()
{
    Set_handle<int> sl(new List_set<int>);
    Set_handle<int> v(new Vector_set<int>(100));

    f(sl);
    f(v);
}

```

Часто требуется, чтобы дескриптор не ограничивался лишь предоставлением доступа. Например, если бы класс *Set* и дескриптор *Set_handle* проектировались совместно, то можно было бы легко обеспечить подсчет ссылок, включив счетчики в объекты *Set*. Но в общем случае, нам не хотелось бы проектировать дескрипторы совместно с управляемыми ими объектами, чтобы не хранить совместно используемые данные в отдельных объектах. Иными словами, помимо интрузивных дескрипторов хотелось бы иметь и неинтрузивные. Вот пример дескрипторного класса, удаляющего объект по исчерпанию ссылок на него:

```

template<class X> class Handle
{
    X* rep;
    int* pcount;

public:
    X* operator->() { return rep; }

    Handle(X* pp) : rep(pp), pcount(new int(1)) {}
    Handle(const Handle& r) : rep(r.rep), pcount(r.pcount) { (*pcount)++; }

    Handle& operator=(const Handle& r)
    {
        if(rep == r.rep) return *this;
        if(--(*pcount) == 0)
        {
            delete rep;
            delete pcount;
        }

        rep = r.rep;
        pcount = r.pcount;
        (*pcount)++;
        return *this;
    }

    ~Handle() { if(--(*pcount) == 0) { delete rep; delete pcount; } }
}

```

Такие дескрипторы можно свободно передавать:

```
void fl (Handle<Set>) ;

Handle<Set> f2 ()
{
    Handle<Set> h (new List_set<int>) ;
    // ...
    return h ;
}

void g ()
{
    Handle<Set> hh=f2 () ;
    fl (hh) ;
    // ...
}
```

Множество, созданное в *f2()*, будет автоматически удалено на выходе из функции *g()*, если только *fl()* не создает ссылку на него (программисту здесь этого даже знать не надо).

Естественно, такое удобство не дается даром, но для большинства прикладных программ цена хранения и обслуживания счетчика ссылок вполне приемлема.

Иногда нужно извлечь указатель из дескриптора и использовать его напрямую. Например, если нужно передать указатель функции, которая ничего не знает о дескрипторе. Это неплохо работает, если, конечно, функция не уничтожит переданный ей объект. Также может быть полезной операция привязки дескриптора к новому представлению:

```
template<class X> class Handle
{
    // ...
    X* get_rep () {return rep ; }

    void bind (X* pp)
    {
        if (pp != rep)
        {
            if (-- (*pcount) == 0)
            {
                delete rep ;
                *pcount = 1 ;           // обновить pcount
            }
            else
                pcount = new int (1) ;   // новый pcount

            rep = pp ;
        }
    }
};
```

Заметим, что наследование новых классов от *Handle* не особенно полезно. Это конкретный тип без виртуальных функций. Идея состоит в том, чтобы иметь один дескрипторный класс для семейства классов, определяемых одним базовым клас-

сом. Наследование от этого базового класса является мощной технологией. Она применима и к узловым классам, и к абстрактным типам.

Как уже сказано, **Handle** не предназначен для порождения производных классов. Чтобы получить класс, действующий как истинный указатель с подсчетом ссылок, нужно **Handle** скомбинировать с **Ptr** из §13.6.3.1 (см. §25.10[2]).

Дескриптор, предоставляющий интерфейс, почти идентичный классу, для которого он обеспечивает доступ, часто называют *заместителем (proxy)*. Это типично для дескрипторов, ссылающихся на удаленные объекты (объекты на удаленных компьютерах).

25.7.1. Операции в дескрипторных классах

Перегрузка операции `->` позволяет дескриптору получать управление и выполнять некоторую работу при каждом обращении к объекту. Например, можно собрать статистику о числе обращений к объекту через дескриптор:

```
template <class T> class Xhandle
{
    T* rep;
    int no_of_accesses;

public:
    T* operator->() { no_of_accesses++; return rep; }
    // ...
};
```

Дескрипторы, которым нужно выполнить работу и *до*, и *после* обращения, требуют более изощренного программирования. Рассмотрим, к примеру, множество, для которого нужно выполнять блокировку при вставках или удалениях элементов. Существенно, что интерфейс класса представления должен быть повторен в дескрипторном классе:

```
template<class T> class Set_controller
{
    Set<T>* rep;
    Lock lock;
    // ...

public:
    void insert (T* p) { Lock_ptr x(lock); rep->insert(p); }           // см. §14.4.1
    void remove (T* p) { Lock_ptr x(lock); rep->remove(p); }
    int is_member (T* p) { return rep->is_member(p); }

    T get_first() { T* p = rep->first(); return p ? *p : T(); }
    T get_next() { T* p = rep->next(); return p ? *p : T(); }

    T first() { Lock_ptr x(lock); T tmp = *rep->first(); return tmp; }
    T next() { Lock_ptr x(lock); T tmp = *rep->next(); return tmp; }
    // ...
};
```

Введение всех этих переадресующих функций довольно утомительно, но не слишком сложно и не очень дорого с точки зрения влияния на производительность программы.

Отметим, что далеко не все функции *Set* нуждаются в блокировке. По моему опыту, это типично для большинства программ, нуждающихся в пред- и пост-действиях. Блокировка абсолютно всех действий в некоторых системных мониторах приводит к ухудшению параллельного исполнения.

Важной причиной для тщательного определения операций дескриптора при перегрузке в нем операции $\rightarrow$ является возможность наследования от *Set_controller*. Однако многие выгоды от дескрипторов становятся сомнительными, когда в производный класс добавляются поля данных. В частности, объем контролируемых дескриптором общих данных снижается относительно объема данных, сосредоточенных в каждом дескрипторном объекте.

25.8. Прикладные среды разработки (application frameworks)

Компоненты, построенные из классов, рассмотренных в §25.2 — §25.7, поддерживают проектирование и повторное использование кода, предоставляя как строительные блоки, так и способы их комбинирования; прикладной программист строит каркас программы, в который эти общеупотребительные строительные блоки включаются. Альтернативным (и часто более амбициозным) подходом к поддержке проектирования и повторного использования является автоматическое построение *каркаса приложения* в рамках *прикладных сред разработки (application frameworks)*, когда программист лишь добавляет к каркасу приложения специфические блоки собственного изготовления. Классы, из которых автоматически строится общий каркас приложения, обладают столь «жирными» интерфейсами, что вряд ли являются классами в обычном понимании. Они приближают построение законченной программы, если не считать того, что они ничего специфического (характерного для нового приложения) не делают. Все конкретные действия предоставляются непосредственно прикладным программистом.

В качестве примера рассмотрим фильтр, то есть программу, которая считывает поток ввода, выполняет какие-то действия с введенными данными, выводит что-то в поток вывода и (возможно) выдает некоторый окончательный результат. Простая среда разработки таких программ обеспечивает набор операций, которые могут потребоваться прикладному программисту:

```
class Filter
{
public:
    class Retry
    {
    public:
        virtual const char* message() {return 0;}
    };

    virtual void start() {}
    virtual int read() = 0;
    virtual void write() {}
    virtual void compute() {}
    virtual int result() = 0;
```

```

virtual int retry (Retry& m) { cerr << m.message() << '\n'; return 2; }
virtual ~Filter() {}
};

```

Функции, которые должен определять производный класс, объявлены чисто виртуальными; остальные функции определены таким образом, что просто ничего не делают.

Среда разработки также обеспечивает главный цикл и рудиментарный механизм обработки ошибок:

```

int main_loop (Filter* p)
{
    for ( ; ; )
    {
        try
        {
            p->start() ;
            while (p->read())
            {
                p->compute() ;
                p->write() ;
            }
            return p->result() ;
        }
        catch (Filter: :Retry& m)
        {
            if (int i = p->retry(m) ) return i;
        }
        catch ( . . . )
        {
            cerr << "Fatal filter error\n";
            return 1;
        }
    }
}

```

Я (в качестве прикладного программиста) свою собственную часть программы мог бы написать следующим образом:

```

class My_filter: public Filter
{
    istream& is;
    ostream& os;
    int nchar;

public:
    int read() { char c; is.get(c) ; return is.good() ; }
    void compute() { nchar++; }
    int result() { os << nchar << " characters read\n"; return 0; }

    My_filter(istream& ii, ostream& oo) : is(ii) , os(oo) , nchar(0) { }
}

```

и запустить все это на выполнение:

```
int main ()  
{  
    My_filter f(cin.cout);  
    return main_loop (&f);  
}
```

Естественно, чтобы быть по-настоящему полезной, среда разработки должна предоставлять больше структурных возможностей и намного больше сервисов, чем наш простой пример. В частности, среды разработки обычно являются иерархиями узловых классов. Благодаря тому, что прикладному программисту остается добавлять «листовые» классы (классы-листья), которые прорастают глубоко внутри иерархии, достигается изрядная общность прикладных программ между собой и высокая степень повторного использования служебных сервисов среды разработки. Среды разработки обычно дополняются библиотеками классов, которые нужны прикладному программисту при написании специфического кода «работающих» классов.

25.9. Советы

1. Делайте осознанные решения по способам использования классов (осознанные как проектировщиком, так и пользователем); §25.1.
2. Учитывайте достоинства и недостатки применения разных видов классов; §25.1.
3. Используйте конкретные типы для представления простых независимых концепций; §25.2.
4. Используйте конкретные типы для представления концепций, для которых важна близкая к оптимальной эффективность; §25.2.
5. Не наследуйте от конкретного класса; §25.2.
6. Используйте абстрактные классы для представления интерфейсов в случаях, когда реализации объектов могут варьироваться; §25.3.
7. Используйте абстрактные классы для представления интерфейсов в случаях, когда разные реализации объектов должны сосуществовать; §25.3.
8. Используйте абстрактные классы для предоставления новых интерфейсов к уже существующим типам; §25.3.
9. Используйте узловые классы там, где схожие концепции разделяют большую часть реализации; §25.4.
10. Используйте узловые классы для постепенного расширения реализации; §25.4.
11. Используйте RTTI для выявления интерфейса объекта; §25.4.1.
12. Используйте классы для ассоциации действий с состоянием (данными); §25.5.
13. Используйте классы для представления действий (операций), которые нужно сохранять, передавать или откладывать их выполнение; §25.5.

14. Используйте интерфейсные классы для адаптирования классов к новым способам их использования (без модификации самих классов); §25.6.
15. Используйте интерфейсные классы, чтобы добавить проверки; §25.6.1.
16. Применяйте дескрипторы, чтобы избежать непосредственного использования указателей или ссылок; §25.7.
17. Применяйте дескрипторы для управления разделяемыми представлениями; §25.7.
18. Применяйте прикладные среды разработок в тех областях, где особенности программ допускают их стандартную структуру; §25.8.

25.10. Упражнения

1. (*1) Шаблон *Io* из §25.4.1 не работает для встроенных типов. Исправьте эту ситуацию.
2. (*1.5) Шаблон *Handle* из §25.7 не отражает отношений наследования между классами, которыми он управляет. Сделайте так, чтобы он отражал эти зависимости. То есть чтобы можно было присваивать *Handle<Circle>* объектам типа *Handle<Shape>*, но не наоборот.
3. (*2.5) Отталкиваясь от класса *String*, определите другой класс, реализующий представление строк с помощью виртуальных функций. Сравните производительность двух классов. Постарайтесь определить разумный класс, который оптимально реализовывался бы в форме открытого наследования от строкового класса с виртуальными функциями.
4. (*4) Изучите пару широко используемых библиотек. Классифицируйте их классы в терминах конкретных типов, абстрактных типов, узловых классов, дескрипторных классов и интерфейсных классов. Используются ли в этих библиотеках абстрактные узловые и конкретные узловые классы? Можно ли классы этих библиотек классифицировать как-то иначе? Используются ли жирные интерфейсы? Какие средства применяются для идентификации типов на стадии выполнения? Какова стратегия управления памятью?
5. (*2) Воспользуйтесь средой разработки *Filter* (§25.8) для реализации программы, которая устраняет смежные дублированные слова в потоке ввода, и перенаправляет исправленный поток слов в поток вывода.
6. (*2) Воспользуйтесь средой разработки *Filter* для реализации программы, которая подсчитывает частоту слов в потоке ввода и помещает в поток вывода пары (слово, частота) в порядке возрастания частоты.
7. (*1.5) Напишите шаблон *Range*, который принимает тип и диапазон значений элементов в качестве параметров шаблона.
8. (*1) Напишите шаблон *Range*, который принимает диапазон значений в качестве параметров конструктора.
9. (*2) Напишите простой строковый класс, который не обрабатывает никаких ошибок. Напишите другой класс, который проверяет доступ к первому. Обсудите все «за» и «против» разделения базовых функций и проверки ошибок.

10. (*2.5) Реализуйте систем
у объектного ввода/вывода из §25.4.1 для нескольких типов, включая целые числа, строки и классовую иерархию по вашему выбору.
11. (*2.5) Определите класс *Storable* в качестве абстрактного базового класса с виртуальными функциями *write_out()* и *read_in()*. Для простоты предположите, что символьная строка описывает место для долговременного хранилища. Воспользуйтесь классом *Storable*, чтобы написать средство, позволяющее сохранять на диске и читать с него объекты классов, производных от *Storable*. Протестируйте это средство с парой классов по вашему выбору.
12. (*4) Разработайте базовый класс *Persistent* с операциями *save()* и *no_save()*, которые определяют, записываются ли объекты в долговременное хранилище деструктором. Кроме указанных операций какие еще полезные операции могли бы предоставлять класс *Persistent*? Протестируйте класс *Persistent* с парой классов по вашему выбору. Является ли *Persistent* узловым классом, конкретным типом или абстрактным типом? Почему?
13. (*3) Напишите класс *Stack*, для которого можно сменить представление на стадии выполнения программы. Подсказка: «любую проблему можно решить, введя еще один уровень косвенности».
14. (*3.5) Определите класс *Oper*, содержащий идентификатор типа *Id* (*string* или *C*-строка) и операцию (указатель на функцию или функциональный объект). Определите класс *Cat_object*, который содержит список элементов типа *Oper* и *void**. Снабдите *Cat_object* операциями *add_oper(Oper)*, которая добавляет *Oper* к списку; *remove_oper(Id)*, которая удаляет *Oper*, идентифицируемую с помощью *Id* из списка; *operator()* (*Id, arg*), которая активизирует *Oper*, идентифицируемую с помощью *Id*. Реализуйте стек котов посредством *Cat_object*. Напишите небольшую программу для тестирования этих классов.
15. (*3) Определите шаблон *Object* на базе класса *Cat_object*. Используйте *Object* для реализации стека строк. Напишите небольшую программу для тестирования этого шаблона.
16. (*2.5) Определите вариант класса *Object* под именем *Class*, который гарантирует, что объекты с одинаковыми операциями совместно разделяют список операций. Напишите небольшую программу для тестирования этого шаблона.
17. (*2) Определите шаблон *Stack*, который предоставляет традиционный и безопасный по типу интерфейс к стеку, реализованному шаблоном *Object*. Сравните этот стек с вариантами стековых классов из предыдущих упражнений. Напишите небольшую программу для тестирования этого шаблона.
18. (*3) Напишите класс для представления операций, которые нужно отправлять для исполнения на удаленный компьютер. Протестируйте этот класс, либо отправляя команды на другой компьютер, либо при помощи записи команд в файл с последующим их чтением и исполнением.
19. (*2) Напишите класс для композиции операций, представленных объектами функциональных классов (объектами-функциями). Для двух объектов *f* и *g*, *Compose(f, g)* должно порождать объект, который может быть активизирован с аргументом *x*, пригодным для *g*, и с возвратом *f(g(x))*, при условии, что возврат *g()* является приемлемым аргументом для *f()*.